IPAGE SOFTWARE

ETE ESCOLA TÉCNICA ESTADUAL

Microsoft® Visual Basic 6.0

APOSTILA DO PROJETO

CADASTRO DE PRODUTOS

CRUD COM ARQUIVOS DE TAMANHO FIXO

Desenvolvimento de um sistema de cadastro de produtos em Visual Basic 6.0, utilizando arquivos de tamanho fixo.





Microsoft® Visual Basic 6.0

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA
Fone/whatsApp: (081) 98615-2352

IPAGE SOFTWARE
Soluções em Software para o seu negócio.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Esta apostila foi elaborada para fins educacionais e disseminação do conhecimento em programação com Visual Basic 6.0.

É proibida a reprodução, distribuição, publicação ou comercialização, total ou parcial, deste material, por qualquer meio, sem autorização prévia e expressa do autor, exceto nos casos permitidos pela legislação vigente.

O conteúdo, os exemplos de código e as explicações apresentados nesta apostila destinam-se ao aprendizado, ao estudo e ao desenvolvimento de soluções de software.

Autor: Diógenes Dias
Organização: Ipage Software
Última Revisão: quinta-feira 22/10/2026 09:10
GitHub: https://github.com/ipagesoftware
Fontes de pesquisa: Programando em Visual Basic 4.0 Para Iniciantes (José Roberto Varela), Microsoft Visual Basic 4 For Windows DATABASE KNOW-HOW (Marco Antônio Gutierrez), Usando o Visual Basic 6 – O Guia Definitivo (Brian Siler & Jeff Spotts), Aplicações em Microsoft Visual Basic 5.0 (Adilson da Silva Lima), Microsoft Visual Basic 5 – Dicas e Soluções de Problemas (Daniel Burd), Manipulando Banco de Daos com Visual Basic 4 (Juano Nuñes del Prado), Aprenda em 21 dias ADO 2.5 (Christoph Wile & Christian Kioller), Dominando a Programação de Banco de Daos com Visual Basic 6 “A BÍBLIA” (Evangelos Petroutsos).
Ferramentas utilizadas: Microsoft Word, PhotoShop, Microsoft Paint.
IA’s: ChatGPT, Gemini.

Este material é fruto da experiência prática e da dedicação ao desenvolvimento de software e ao compartilhamento do conhecimento.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

SOBRE O AUTOR



Diógenes Dias de Souza Júnior é profissional de desenvolvimento de software com mais de 30 anos de experiência na área de tecnologia, dedicando grande parte de sua trajetória à criação, manutenção e evolução de sistemas computacionais.

Ao longo de sua carreira, desenvolveu conhecimentos em programação, automação de processos, construção de interfaces gráficas e gerenciamento de dados, com destaque para o

Visual Basic 6.0, linguagem com a qual trabalhou no desenvolvimento de aplicações comerciais e soluções voltadas às necessidades do dia a dia das empresas.

À frente da Ipage Software, atua no desenvolvimento e na modernização de sistemas, buscando aliar experiência prática, eficiência e simplicidade na construção de soluções tecnológicas.

Além da atuação profissional, acredita que compartilhar conhecimentos é uma das formas mais importantes de contribuir para o crescimento da comunidade de tecnologia. Por esse motivo, dedica parte de seu tempo ao estudo, à orientação e ao apoio de estudantes que estão iniciando sua jornada na programação, especialmente aqueles que encontram dificuldades no aprendizado.

Esta apostila nasceu dessa combinação entre experiência prática e propósito educacional. Ao apresentar um projeto de cadastro de produtos desenvolvido em Visual Basic 6.0, utilizando arquivos de tamanho fixo e as operações CRUD (**Create, Read, Update e Delete**), o autor busca demonstrar como conceitos fundamentais da programação podem ser aplicados na construção de um sistema funcional.

Mais do que documentar um projeto, a proposta é compartilhar conhecimento, estimular o raciocínio lógico e mostrar que a compreensão dos fundamentos continua sendo essencial para quem deseja se tornar um bom desenvolvedor.

Sua filosofia: a tecnologia evolui constantemente, mas a experiência, a lógica de programação e a disposição para aprender e compartilhar conhecimentos permanecem como pilares da boa formação profissional.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

MIT LICENSE

Copyright (c) 1995-2026 Diogenes Dias

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

The software is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. in no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the software or the use or other dealings in the software.

EXCEPTION FOR VISUAL ASSETS AND BRAND IDENTITY

The MIT License above applies to the FlatFile source code and other project materials, EXCEPT for FlatFile's visual assets and brand identity, including the application icon, document and file-type icons, splash-screen images, installer images, logos, and the FlatFile name and trademarks.

These visual assets and brand identity elements are NOT licensed under the MIT License. They are licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License (CC BY-NC-ND 4.0): https://creativecommons.org/licenses/by-nc-nd/4.0/

Use, sharing, and redistribution of these assets must comply with the terms of that Creative Commons license, including its attribution, non-commercial, and no-derivatives requirements.

THIRD-PARTY COMPONENTS

Third-party components and materials included in or used by FlatFile, including icons, images, libraries, and other assets, remain subject to their respective licenses and the rights of their authors. Please consult the applicable license notices and documentation. This project license does not replace or modify third-party licenses.

PROJECT REFERENCES

- WEBSITE: https://www.ipage.com.br
- FLATFILE source repository: https://github.com/ipagesoftware/projeto-Banco-de-Dados-com-FlatFile-em-vb6

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

LICENÇA MIT

É concedida, por meio deste documento, permissão gratuita a qualquer pessoa que obtenha uma cópia deste software e dos arquivos de documentação associados (o "Software"), para utilizar o Software sem restrições, incluindo, sem limitação, os direitos de usar, copiar, modificar, mesclar, publicar, distribuir, sublicenciar e/ou vender cópias do Software, e permitir que as pessoas a quem o Software for fornecido o façam, sujeito às seguintes condições:

O aviso de direitos autorais acima e este aviso de permissão devem ser incluídos em todas as cópias ou partes substanciais do Software.

O software é fornecido "no estado em que se encontra", sem garantia de qualquer tipo, expressa ou implícita, incluindo, mas não se limitando a, garantias de comerciabilidade, adequação a um determinado propósito e não violação. em nenhuma hipótese os autores ou titulares de direitos autorais serão responsáveis por qualquer reivindicação, danos ou outra responsabilidade, seja em uma ação contratual, delitual ou de outra natureza, decorrente de, relacionada a ou em conexão com o software, seu uso ou outras formas de manuseio do software.

EXCEÇÃO PARA ATIVOS VISUAIS E IDENTIDADE DE MARCA

A Licença MIT acima aplica-se ao código-fonte do FlatFile e a outros materiais do projeto, EXCETO aos ativos visuais e à identidade de marca do FlatFile, incluindo o ícone do aplicativo, ícones de documentos e tipos de arquivo, imagens de tela de abertura (splash screen), imagens do instalador, logotipos, bem como o nome FlatFile e suas marcas registradas.

Estes ativos visuais e elementos de identidade de marca NÃO estão licenciados sob a Licença MIT. Eles estão licenciados sob a licença Creative Commons Atribuição-NãoComercial-SemDerivações 4.0 Internacional (CC BY-NC-ND 4.0):
https://creativecommons.org/licenses/by-nc-nd/4.0/

O uso, compartilhamento e redistribuição desses ativos devem estar em conformidade com os termos da referida licença Creative Commons, incluindo suas exigências de atribuição, uso não comercial e vedação à criação de obras derivadas.

COMPONENTES DE TERCEIROS

Componentes e materiais de terceiros incluídos no FlatFile ou por ele utilizados — incluindo ícones, imagens, bibliotecas e outros recursos — permanecem sujeitos às suas respectivas licenças e aos direitos de seus autores. Consulte os avisos de licença e a documentação aplicáveis. A licença deste projeto não substitui nem modifica as licenças de terceiros.

REFERÊNCIAS DO PROJETO

- SITE: https://www.ipage.com.br
- Repositório de código-fonte do FLATFILE: https://github.com/ipagesoftware/projeto-Banco-de-Dados-com-FlatFile-em-vb6

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

PROJETO EM VISUAL BASIC 6.0 PARA OPERAÇÕES DE CRUD COM ARQUIVO TEXTO DEMILITADO – TAMANHO FIXO

INTRODUÇÃO AO PROJETO

O projeto consiste no desenvolvimento de uma aplicação desktop utilizando **VB 6.0**, com o objetivo de demonstrar, de forma prática, os principais conceitos envolvidos na construção de um sistema de gerenciamento de dados.

A aplicação foi estruturada com base no modelo CRUD (**Create, Read, Update e Delete**), que representa as quatro operações fundamentais de manipulação de informações: inclusão, consulta, atualização e exclusão de registros.

A interface inicial do projeto, denominada **frmProduto**, permite ao usuário pesquisar produtos, visualizar seus dados, navegar entre os registros e acessar as operações de manutenção por meio de uma barra de ferramentas.

O sistema também possui telas auxiliares, como **frmInsert**, responsável pela inclusão de novos produtos, e **frmEdit**, destinada à edição de registros existentes.

Além disso, disponibiliza uma rotina de confirmação para exclusões e um relatório com a listagem dos produtos ordenados por código.

O projeto utiliza uma estrutura de armazenamento de dados em arquivo, representada pelo arquivo **produtos.dat**, permitindo demonstrar os princípios de persistência das informações e a integração entre a interface gráfica e a camada responsável pelo gerenciamento dos registros.

OBJETIVO GERAL

Desenvolver e documentar um sistema de cadastro de produtos em Visual Basic 6.0, demonstrando a implementação das operações fundamentais de um CRUD, a organização dos formulários, a interação com o usuário e o gerenciamento das informações armazenadas.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

OBJETIVOS ESPECÍFICOS.

Implementar a inclusão de novos produtos, com preenchimento e validação dos campos.

- Permitir a consulta e a localização de produtos por meio de um campo de pesquisa.
- Apresentar os registros em uma lista interativa, facilitando a seleção e a visualização dos dados.
- Implementar a alteração das informações de produtos já cadastrados.
- Disponibilizar a exclusão de registros com confirmação prévia, reduzindo o risco de operações acidentais.
- Permitir a navegação entre os registros, incluindo primeiro, anterior, próximo e último.
- Gerar relatórios com os produtos cadastrados, ordenados por código.
- Demonstrar o uso de componentes do VB 6.0, como formulários, caixas de texto, botões, menus, listas e barras de ferramentas.
- Evidenciar a utilização da propriedade **ItemData** de uma **ListBox** para associar o item apresentado na interface ao identificador correspondente do produto.
- Organizar o código e os componentes de maneira que facilitem a compreensão, a manutenção e a evolução futura da aplicação.

FINALIDADE EDUCACIONAL E PRÁTICA.

Além de atender às necessidades básicas de gerenciamento de produtos, o projeto serve como referência prática para estudantes e desenvolvedores interessados em compreender a estrutura de aplicações comerciais tradicionais.

Por meio deste exemplo, é possível estudar a relação entre formulários, eventos, controles visuais, validação de dados, operações de leitura e gravação, navegação entre registros e geração de relatórios.

Dessa forma, o Cadastro de Produtos demonstra como os conceitos fundamentais de programação podem ser aplicados na construção de uma aplicação funcional, organizada e adequada ao gerenciamento de informações em um ambiente desktop.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O QUE VOCÊ APRENDERÁ COM ESTE PROJETO?

- Introdução ao Visual Basic 6.0
- Módulo (bas)
- Breve introdução Orientação Objeto com classe (cls)
- formulário SDI (frm)
- Documentar o código do projeto com Rem ou (‘)
- Nomenclaturas
- Menus
- Eventos
- Variáveis de memória
- Sub rotinas
- Funções
- Componentes
- API’s do Windows

UMA BREVE HISTÓRIA SOBRE A LINGUAGEM VISUAL BASIC 6.0

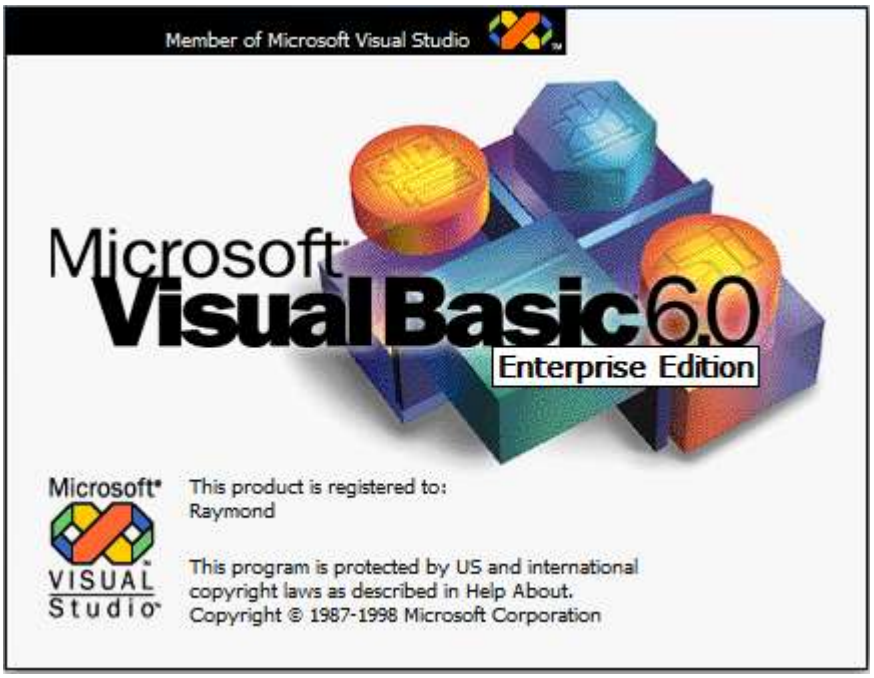


Figura 1

O Microsoft VB 6.0, lançado em 1998, é uma das ferramentas de desenvolvimento mais emblemáticas e influentes da história da tecnologia.

Projetado com o foco no desenvolvimento rápido de aplicações (RAD - Rapid Application Development), o VB 6.0 revolucionou a indústria ao introduzir um ambiente visual intuitivo no qual telas e interfaces gráficas para o Windows podiam ser desenhadas em minutos via arrastar e soltar, combinando uma sintaxe clara e altamente acessível.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FACILIDADE DE APRENDIZADO E EXPANSÃO GLOBAL.

A curva de aprendizado suave do Visual Basic 6.0 permitiu que programadores iniciantes, estudantes e profissionais de diversas áreas fora da T.I. criassem softwares funcionais com extrema rapidez. Essa simplicidade rompeu as barreiras do desenvolvimento tradicional de software da época, transformando o VB 6.0 em um fenômeno global.

A ferramenta impulsionou o crescimento de uma das maiores comunidades de desenvolvedores do planeta, alcançando a marca histórica de mais de 3,5 milhões de programadores ao redor do mundo. Durante o final dos anos 1990 e início dos anos 2000, o ecossistema VB 6.0 dominou a criação de sistemas corporativos, componentes ActiveX e automações comerciais de todos os portes.

ONDE O VB 6.0 SE DESTACA ATUALMENTE?

Apesar de ter sido lançado há mais de duas décadas, o Visual Basic 6.0 continua se destacando em nichos estratégicos da engenharia de software:

Sistemas Legados Críticos: Milhares de empresas globais — incluindo bancos, indústrias, operadoras de logística e pequenas e médias empresas — continuam rodando sistemas em VB 6.0 devido à alta estabilidade das regras de negócio e ao custo/risco elevado de reescrita completa.

Automação Comercial e Hardware: Mantém forte presença em sistemas de Ponto de Venda (PDV), controle de estoque e integração com periféricos (leitores de código de barras, impressoras fiscais e balanças).

Alta Compatibilidade e Desempenho: Por ser nativamente compilado para o modelo COM (Component Object Model) e ter baixíssimo consumo de memória RAM. Executáveis compilados em VB 6.0 continuam rodando nativamente com rapidez no Windows 10 e Windows 11 (64-bit).

Interoperabilidade Moderna: Atua em arquiteturas híbridas através de pontes de comunicação (COM Interop com .NET, proxies locais e middlewares de nuvem) para atender exigências atuais de segurança, como criptografia TLS 1.2/1.3 e consumo de APIs REST.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O QUE POSSO APRENDER COM ESTA LINGUAGEM?

Estudar o VB 6.0 vai muito além de aprender uma tecnologia; é uma verdadeira imersão nos fundamentos da engenharia de software, na arquitetura de sistemas e na lógica por trás das aplicações corporativas que sustentam o mundo real há décadas.

Abaixo estão os principais aprendizados práticos e conceituais que o VB 6.0 proporciona a qualquer desenvolvedor:

O PODER E A PRATICIDADE DO RAPID APPLICATION DEVELOPMENT(RAD).

O VB 6.0 foi pioneiro em mostrar como entregar software funcional com extrema rapidez. Com ele, você aprende a pensar de forma centrada no usuário: desenhar a interface visual, associar eventos de clique ou alteração de campos e conectar os dados de forma ágil, entendendo o ciclo direto entre a tela (User Interface - UI) e a lógica de negócios.

MANIPULAÇÃO DE DADOS DE BAIXO NÍVEL (FLAT FILES E UDTS)

Como o VB 6.0 lida diretamente com arquivos planos (.dat) e estruturas de dados personalizadas (UDTs — User-Defined Types), você aprende como os dados são organizados na memória e no disco byte a byte, sem a "blindagem" ou a dependência de ORMs modernos (como Entity Framework ou Hibernate). Isso ensina:

Como funcionam chaves e registros em acesso aleatório (Random Access).
A importância do alinhamento de tipos de dados primitivos.
Lógica pura de serialização e persistência de arquivos.

ENCAPSULAMENTO E POO PRÁTICA COM MÓDULOS DE CLASSE (.cls)

O VB 6.0 possui uma implementação pragmática de Orientação a Objetos. Ao criar classes no VB 6.0, você aprende a construir os pilares da POO do zero:

O uso de variáveis privadas para proteger o estado interno.
A criação manual de portas de acesso com Property Get e Property Let.

O controle rigoroso do ciclo de vida dos objetos através de `Class_Initialize` e `Class_Terminate`.

GESTÃO DE ESCOPO E ORGANIZAÇÃO DE CÓDIGO (.bas)

A separação clara entre a interface visual (.frm) e a lógica global de negócio (.bas) treina o desenvolvedor a estruturar projetos limpos. Você aprende a gerenciar o escopo de variáveis globais (Public), rotinas utilitárias reutilizáveis e o ponto de entrada de uma aplicação via Sub Main.

LINK PARA DOWNLOAD E INSTALAÇÃO DO VB 6.0 PORTABLE:

https://github.com/ipagesoftware/Visual-Basic-6-Portable-Instalacao_e_Configuracao_Automatica

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

VISUAL BASIC 6.0 PORTABLE

O Visual Basic 6.0 Portable é uma versão empacotada e modificada da clássica IDE do VB6, projetada para rodar diretamente de uma pasta ou dispositivo removível (como um pen drive), sem a necessidade de uma instalação tradicional no sistema operacional Windows.

O CONTEXTO: POR QUE O VB6 "PORTABLE" EXISTE?

Como o Visual Basic 6.0 original foi lançado pela Microsoft em 1998, o seu instalador foi concebido para o ecossistema do Windows 98/XP. Tentar instalar a versão original em sistemas modernos (como o Windows 10 ou Windows 11) costuma ser uma tarefa frustrante, exigindo modos de compatibilidade, dezenas de cliques manuais e ajustes complexos para evitar erros nas bibliotecas do sistema.

Para contornar esse problema, desenvolvedores da comunidade empacotaram a IDE junto com as dependências essenciais em um único pacote compactado que roda de forma isolada.

PRINCIPAIS VANTAGENS

1. Zero Instalação: Não altera o Registro do Windows de forma invasiva e não espalha arquivos pelas pastas protegidas do sistema (System32 ou SysWOW64).
2. Pronto para Uso (Plug and Play): Basta descompactar a pasta em qualquer diretório (ou colocar num pen drive) e executar o arquivo .exe para começar a programar imediatamente.
3. Compatibilidade com Windows Modernos: A maioria das versões portáveis já vem pré-configurada para funcionar sem travamentos nas versões mais recentes do Windows.
4. Ideal para Manutenção de Legados: Excelente para programadores que precisam dar manutenção rápida em sistemas antigos (legacy systems) sem precisar formatar uma máquina virtual antiga.

CUIDADOS E LIMITAÇÕES IMPORTANTES

Apesar da praticidade, por se tratar de uma solução não oficial (a Microsoft jamais lançou uma versão "Portable" do VB6), é preciso ter atenção a alguns pontos:

- Origem Confiável: Como essas versões são distribuídas por terceiros em fóruns ou sites de download, é fundamental baixar de fontes seguras para evitar malwares ou versões adulteradas da IDE.
- Problemas com Componentes Avançados (.ocx / .dll): Embora o ambiente básico funcione perfeitamente, alguns controles ActiveX de terceiros ou referências muito específicas do Windows podem falhar se exigirem registros complexos no sistema que o pacote portable não consegue isolar sozinho.
- Desenvolvimento Comercial vs. Testes: Para projetos educacionais, testes rápidos e suporte a legados, o portable quebra um galho imenso. No entanto, para ambientes corporativos rígidos, muitos desenvolvedores ainda preferem usar máquinas virtuais (com Windows XP ou 7) rodando a instalação oficial para garantir 100% de estabilidade e fidelidade ao ambiente de homologação.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Você poderá baixar e instalar a versão portátil no link abaixo:

https://github.com/ipagesoftware/Visual-Basic-6-Portable-Instalacao_e_Configuracao_Automatica

ARQUITETURA DE JANELAS: SDI e MDI

Trabalhar com interfaces no VB 6.0 expõe diretamente os conceitos clássicos de design de sistemas de desktop:

SDI (Single Document Interface): Gerenciamento de janelas independentes, modais e não-modalees.

MDI (Multiple Document Interface): Confinamento de telas filhas (MdiChild = True) dentro de um container pai e fusão dinâmica de menus (Menu Merging).

ENGENHARIA REVERSA, RESILIÊNCIA E MODERNIZAÇÃO (Sistemas Híbridos)

Lidar com um sistema legado ensina a lição mais valiosa para um engenheiro de software: como dar novo fôlego a tecnologias antigas sem precisar reescrever tudo do zero. Você aprende a:

Criar pontes de interoperabilidade (COM Interop para conectar o VB 6.0 com bibliotecas .NET).

Desenvolver proxies locais e middlewares para superar limitações de segurança modernas (como a ausência de suporte nativo a TLS 1.2/1.3).

Analisar código sem documentação prévia, entendendo regras fiscais e operacionais complexas acumuladas ao longo de anos.

CONCLUSÃO

O programador que compreende o ecossistema do VB 6.0 ganha uma visão sistêmica profunda. Ele deixa de depender exclusivamente de frameworks automáticos modernos e passa a entender como o software realmente interage com o sistema operacional, a memória RAM e o armazenamento local, tornando-se um profissional muito mais analítico e preparado para resolver problemas complexos em qualquer tecnologia.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O AMBIENTE DE DESENVOLVIMENTO DO VISUAL BASIC

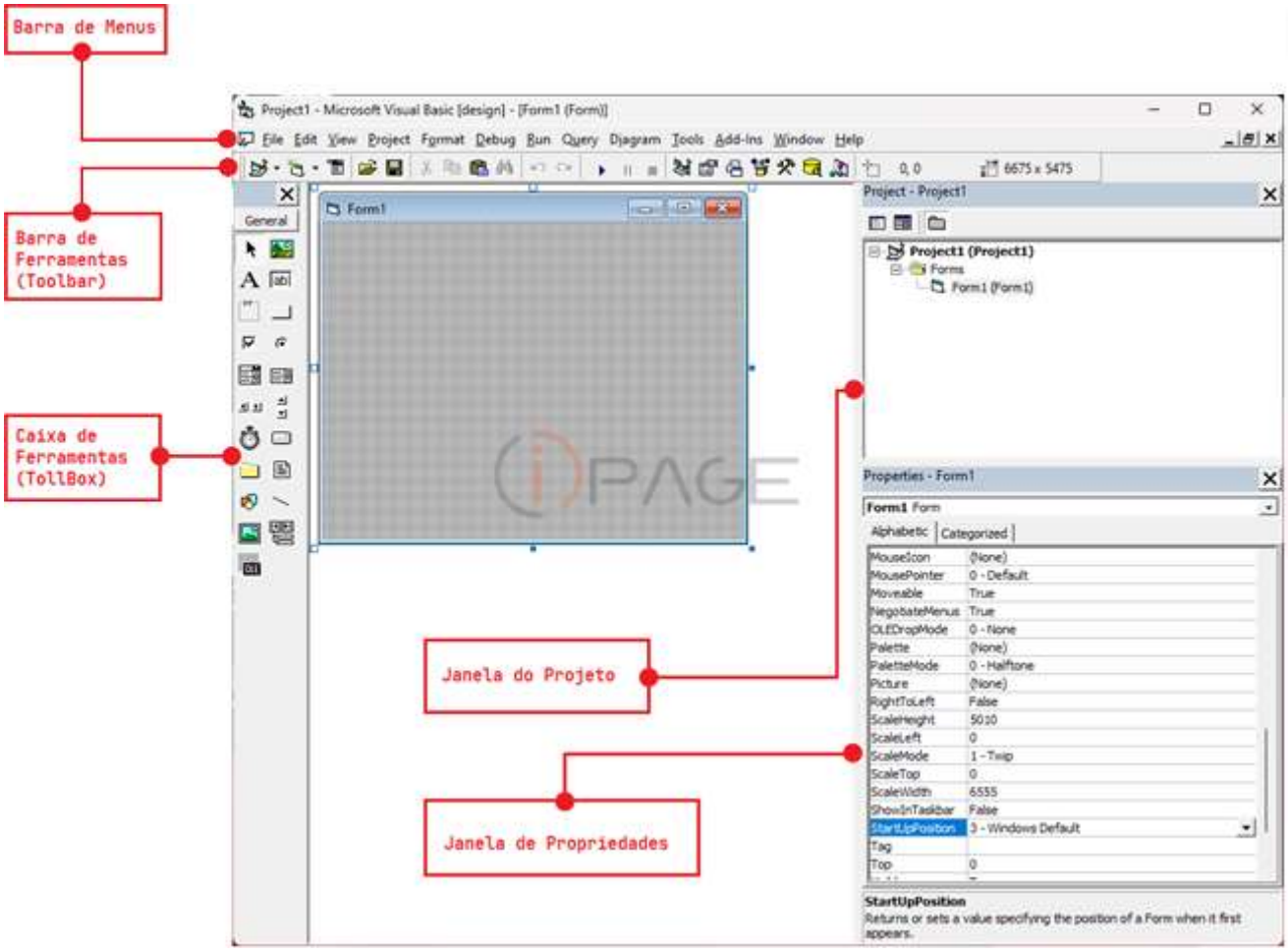


Figura 2

BARRA DE MENUS

A Barra de Menus (Menu Bar) do Visual Basic 6.0 é o painel superior localizado logo abaixo da barra de títulos da IDE (Integrated Development Environment). Ela centraliza os principais comandos, ferramentas de navegação, opções de compilação, gerenciamento de janelas e o acesso ao editor de menus visuais.

Para quem está aprendendo ou ensinando desenvolvimento, a barra de menus do VB 6.0 funciona como o "painel de controle" do ambiente de desenvolvimento.

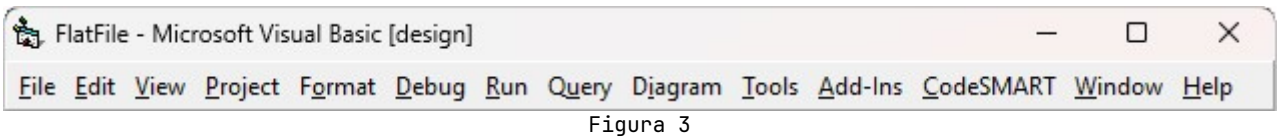


Figura 3

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

MENUS PRINCIPAIS E SUAS FUNÇÕES

A barra de menus padrão do VB 6.0 é dividida nos seguintes menus principais:

1. File (Arquivo)

Gerencia os arquivos e projetos que compõem o sistema.

- New Project/New File: Cria um novo projeto (Standard EXE, ActiveX, etc.) ou novos componentes (Form, MDIForm, Module .bas, Class .cls).
- Open Project/Save Project: Abre ou salva o projeto e suas extensões (.vbp, .frm, .bas).
- Make Project (.exe): É o comando mais importante para a entrega final — compila todo o código e gera o executável nativo do Windows (.exe) e componentes associados.

2. Edit (Editar)

Comandos básicos de edição de texto e código-fonte.

- Undo/Redo: Desfaz ou refaz alterações no código ou no design visual.
- Cut, Copy, Paste, Find, Replace: Ferramentas clássicas de manipulação de texto no editor de código.

3. View (Exibir)

Controla quais janelas e painéis do ambiente de desenvolvimento estão visíveis na tela.

- Code (F7): Abre a janela de código do formulário ou módulo selecionado.
- Object (Shift+F7): Abre o editor visual (a tela em si).
- Project Explorer (Ctrl+R): Mostra a árvore de arquivos do projeto (formulários, módulos, classes).
- Properties Window (F4): Exibe as propriedades do componente selecionado.
- ToolBox: Abre a caixa de ferramentas com os controles visuais.

4. Project (Projeto)

Adiciona novos elementos estruturais e gerencia as referências externas do projeto.

- Add Form/Add Module/Add Class Module: Atalhos para inserir novos arquivos no projeto atual.
- References: Permite adicionar bibliotecas externas (como COM components, ADO para banco de dados ou integração com o Windows).
- Components (Ctrl+T): Abre a janela para habilitar controles ActiveX adicionais (.ocx) na caixa de ferramentas (como barra de progresso, calendários, grids).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

5. Format (Formatar)

Ferramentas cruciais de alinhamento visual para organizar os botões e caixas de texto nas telas (muito útil antes de existirem layouts responsivos automáticos).

- Align (Left, Right, Top, Bottom): Alinha múltiplos controles selecionados entre si.
- Make Same Size (Width/Height): Padroniza o tamanho de vários botões ou campos.
- Center in Form: Centraliza componentes horizontalmente ou verticalmente na tela.

6. Debug (Depuração)

Ferramentas essenciais para encontrar e corrigir erros (bugs) no código.

- Toggle Breakpoint (F9): Marca uma linha de código para pausar a execução do programa quando o fluxo passar por ela.
- Step Into (F8)/Step Over (Shift+F8): Executa o código linha por linha para acompanhar o comportamento das variáveis.
- Add Watch/Quick Watch: Permite monitorar o valor de variáveis em tempo real enquanto o programa está pausado.

7. Run (Executar)

Controla o estado de execução da aplicação direto na IDE.

- Start (F5): Compila temporariamente e executa a aplicação para testes.
- Break (Ctrl+Break): Pausa a execução de um programa que está rodando.
- End: Encerra abruptamente a execução e retorna ao modo de design.

8. Tools, Add-Ins e Window

Tools: Permite configurar opções gerais do ambiente de desenvolvimento.

- Add-Ins: Gerencia suplementos e ferramentas auxiliares instaladas no VB 6.0.
- Window: Lista e organiza as janelas abertas dentro da IDE (organizar em cascata, lado a lado, etc.).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O EDITOR DE MENUS VISUAIS (Menu Editor)

Um recurso muito importante atrelado à barra de menus no VB 6.0 é o Menu Editor (acessado pelo atalho Ctrl+E ou pelo menu Tools -> Menu Editor).

Com ele, o desenvolvedor pode criar facilmente a barra de menus superior de suas próprias aplicações (como os menus Arquivo, Editar, Relatórios, Sair de um sistema comercial), estruturando submenus e teclas de atalho (Shortcuts) sem precisar escrever códigos complexos de interface do zero.

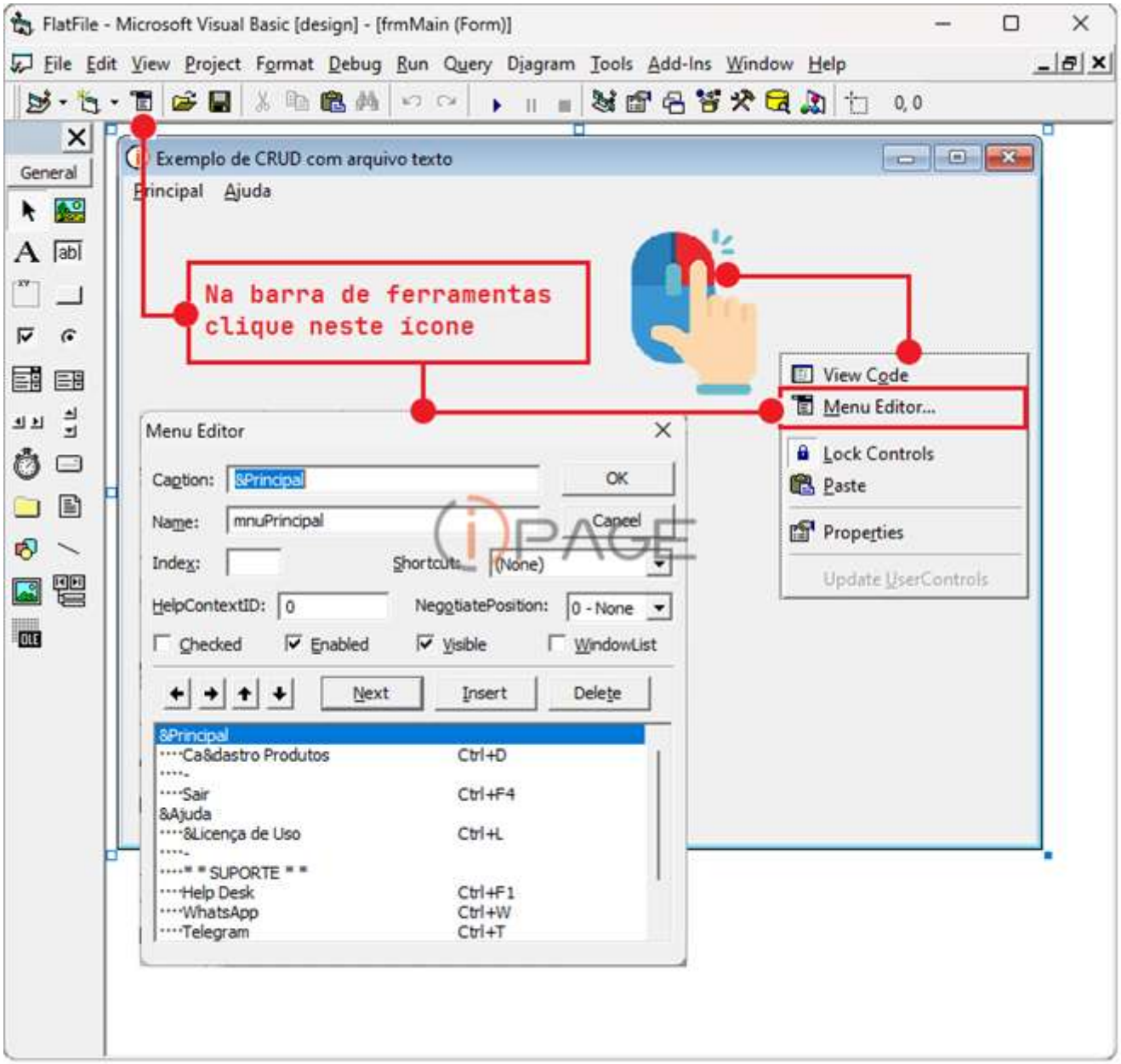


Figura 4

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

BARRA DE FERRAMENTAS (ToolBar)

A Toolbar (ou barra de ferramentas) é um controle gráfico interativo que exibe botões com ícones visuais, separadores e menus suspensos, permitindo que o usuário execute comandos frequentes com um único clique.



Figura 5

Na documentação da tela de Cadastro de Produtos que estamos construindo para o seu repositório GitHub, a Toolbar desempenha um papel central: ela concentra todas as ações essenciais do CRUD (navegação de registros, atualização, inclusão, alteração, exclusão, impressão e saída) através de ícones intuitivos no topo da janela (frmProduto).

COMO FUNCIONA A TOOLBAR NO VB 6.0?

Por padrão, a Toolbar no VB 6.0 não vem nativamente visível na caixa de ferramentas simples (ToolBox). Ela faz parte de um componente ActiveX avançado fornecido pela Microsoft.

- A Biblioteca Necessária: Para utilizá-la, é preciso carregar o componente Microsoft Windows Common Controls (geralmente o arquivo MSComctl.ocx) acessando o menu Project -> Components (ou pressionando Ctrl+T) e marcando a opção correspondente.
- O Controle (Toolbar): Uma vez habilitado, ele aparece na caixa de ferramentas. Ao arrastá-lo para o formulário, ele automaticamente se alinha à parte superior da tela (como uma barra horizontal).
- Associação com ImageList: Para exibir os ícones coloridos nos botões, a Toolbar exige a associação com outro componente chamado ImageList, onde os ícones (arquivos .ico ou .bmp) são armazenados em lote e referenciados por índice ou chave.

RESUMO DIDÁTICO

O que é?
Uma barra horizontal repleta de ícones que acelera o uso do sistema.

Vantagem: Melhora a experiência do usuário (User experience), tornando operações complexas (como deletar, atualizar ou imprimir) acessíveis com apenas um clique visual.

No Projeto Atual (Cadastro de Produtos): É exatamente a barra superior de frmProduto que comanda toda a dinâmica de navegação e as ações do CRUD (Recarregar, Novo, Editar, Excluir, Imprimir e Sair).



Figura 6

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

CAIXA DE FERRAMENTAS (ToolBox)

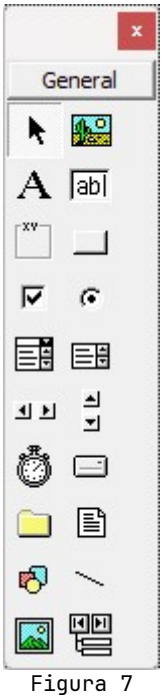


Figura 7

A ToolBox (Caixa de Ferramentas) do VB 6.0 é a janela flutuante ou acoplada que contém todos os componentes visuais e funcionais (controles) que você pode adicionar às suas telas (Forms).
Para quem está a aprender ou a ensinar programação, a ToolBox é literalmente a prateleira de peças de construção da aplicação.

ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO.



Imagine que o Formulário é a tela em branco de um quadro e a ToolBox é a caixa de ferramentas de um artesão. Nela estão organizados todos os instrumentos prontos a usar: botões, caixas de texto, listas, imagens e rótulos.
Para criar o seu programa, basta escolher a ferramenta certa na caixa e colocá-la no formulário.

OS CONTROLES PADRÃO DA TOOLBOX (Controles Intrínsecos)

Por predefinição, a caixa de ferramentas do VB 6.0 traz uma série de controles nativos essenciais para o desenvolvimento de aplicações desktop. Entre os mais utilizados, destacam-se:

- Pointer (Seta): Não é um controle, mas sim a ferramenta padrão para selecionar, mover e redimensionar os componentes já colocados no formulário.
- TextBox (txt): Caixa de texto editável onde o utilizador pode digitar dados (como o nome de um produto ou uma palavra-passe).
- CommandButton (btn ou cmd): O clássico botão de comando que o utilizador clica para executar uma ação (ex: Salvar, Excluir).
- Label (lbl): Um rótulo de texto estático usado para exibir mensagens informativas ou títulos de campos na tela (ex: NOME PRODUTO:).
- ListBox (lst) e ComboBox (cmb): Listas de seleção de itens (a primeira exibe uma lista aberta de opções; a segunda é a famosa caixa de combinação suspensa).
- CheckBox e OptionButton: Controles de marcação para opções booleanas (caixas de verificação e botões de rádio/opção exclusiva).
- PictureBox e Image: Utilizados para carregar e exibir imagens, ícones ou gráficos na interface.
- Shape e Line: Ferramentas puramente visuais para desenhar retângulos, círculos e linhas separadoras na tela.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMO FUNCIONA O USO DA TOOLBOX NA PRÁTICA?

O fluxo de trabalho utilizando a ToolBox é o conceito principal do RAD (Rapid Application Development):

- Seleção: O programador clica num ícone dentro da ToolBox (por exemplo, o CommandButton).
- Posicionamento (Drag & Drop): Clica e arrasta o mouse sobre o formulário para desenhar o botão no local e tamanho desejados.
- Configuração: Imediatamente após soltar, o controle ganha um nome automático (ex: Command1) e suas propriedades podem ser ajustadas na janela de Properties (como mudar o Caption para "Salvar" e o Name para btnSalvar).

EXPANDINDO A TOOLBOX (Controles ActiveX/.ocx)

A ToolBox do VB 6.0 não se limita apenas aos controles padrão. Através do menu Project -> Components (ou pressionando o atalho Ctrl+T), é possível abrir um catálogo gigantesco de componentes externos da Microsoft ou de terceiros.

Ao marcar caixas de seleção nessa janela (como o Microsoft Windows Common Controls), novos ícones especializados são injetados diretamente na ToolBox, tais como:

- ProgressBar: Barras de progresso para downloads ou carregamentos.
- Slider: Controles deslizantes de volume ou ajuste numérico.
- MonthView/DTPicker: Calendários visuais interativos.
- MSFlexGrid: Tabelas e planilhas de dados avançadas para exibição de linhas e colunas.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

BARRA DE PROPRIEDADES

A Barra de Propriedades (Properties Window), geralmente acessada pela tecla de atalho F4 no Visual Basic 6.0, é o painel onde você controla o comportamento, o visual e a identificação de cada elemento que compõe o seu sistema (seja um formulário, um botão, uma caixa de texto ou qualquer outro controle).

Pense nela como o painel de ajustes finos de cada peça do seu projeto. Quando você clica em um botão na tela, a janela de propriedades muda instantaneamente para mostrar apenas as características daquele botão específico.

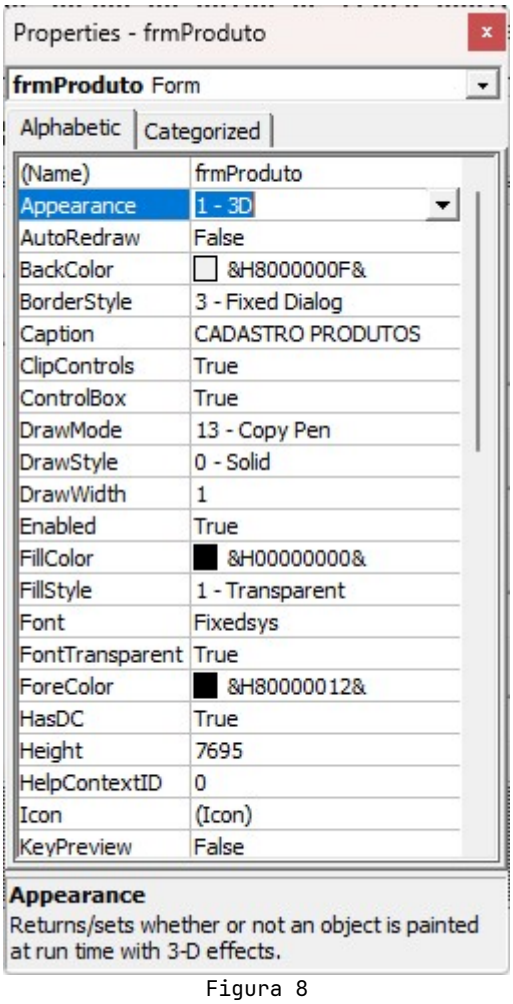


Figura 8

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO.



Imagine que você está a decorar uma casa. O formulário é a parede da sala, e o botão que você colocou lá é um quadro. A Barra de Propriedades é a folha de especificações onde você decide se a moldura do quadro é de madeira ou metal: Qual o seu tamanho? Ele está pendurado mais à esquerda ou à direita?
Qual é o nome escrito na etiqueta de identificação dele?

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMO A JANELA DE PROPRIEDADES É ORGANIZADA?

A interface da janela de propriedades divide-se essencialmente em duas colunas principais e um menu de seleção superior:

1. A Caixa de Seleção Superior (Object Box):

Um menu suspenso no topo que mostra o nome e o tipo do controle que está selecionado no momento (ex: cmdSalvar do tipo CommandButton). Se você tiver muitos controles, pode escolhê-los diretamente por ali sem precisar clicar na tela.

2. As Abas de Organização (Categorized/Alphabetic):

- Categorized (Categorizada): Agrupa as propriedades por temas lógicos (ex: Appearance para cores e bordas, Behavior para comportamento, Font para letras, Position para tamanho e coordenadas).
- Alphabetic (Alfabética): Coloca todas as propriedades em ordem de A Z. É a favorita dos programadores mais experientes que já sabem o nome exato da propriedade que querem alterar.

3. As Colunas de Configuração:

- Coluna da Esquerda: O nome da propriedade (ex: BackColor, Caption, Enabled, Visible).
- Coluna da Direita: O valor atual dessa propriedade, que você pode alterar livremente digitando um texto, escolhendo uma cor em uma paleta ou alternando entre True e False.

AS PROPRIEDADES MAIS IMPORTANTES NO VB 6.0

Embora existam dezenas de propriedades para cada componente, algumas são fundamentais no dia a dia do desenvolvimento:

- Name (O Identificador): É o nome técnico que você usa para chamar o controle dentro do código-fonte (ex: txtNomeCliente, btnExcluir). Nunca deve conter espaços ou acentos.
- Caption ou Text (O Rótulo/Conteúdo):
    - Caption define o texto visível para o usuário em botões, rótulos (Label) e formulários (ex: "Salvar Registro").
    - Text define o conteúdo editável dentro de caixas de texto (TextBox).

- Enabled (Habilitado – True/False): Controla se o usuário pode interagir com o controle. Se estiver como False, o botão ou caixa de texto fica "acinzentado" e bloqueado.
- Visible (Visível – True/False): Define se o controle aparece na tela ou se fica totalmente oculto para o utilizador.
- Font (A Fonte): Permite alterar o tipo de letra, o tamanho, o negrito e o itálico do texto exibido no componente.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

EXEMPLO PRÁTICO DE USO

Se você colocar um botão (CommandButton) em um formulário, a Barra de Propriedades permite transformá-lo completamente sem escrever uma única linha de código:

1. Altera a propriedade Name de Command1 para btnConfirmar.
2. Altera a propriedade Caption de Command1 para &Confirmar Pedido (o caractere & cria automaticamente uma tecla de atalho no teclado, sublinhando a letra C).
3. Altera a propriedade Font para colocar o texto em Negrito (Bold) e tamanho 10.

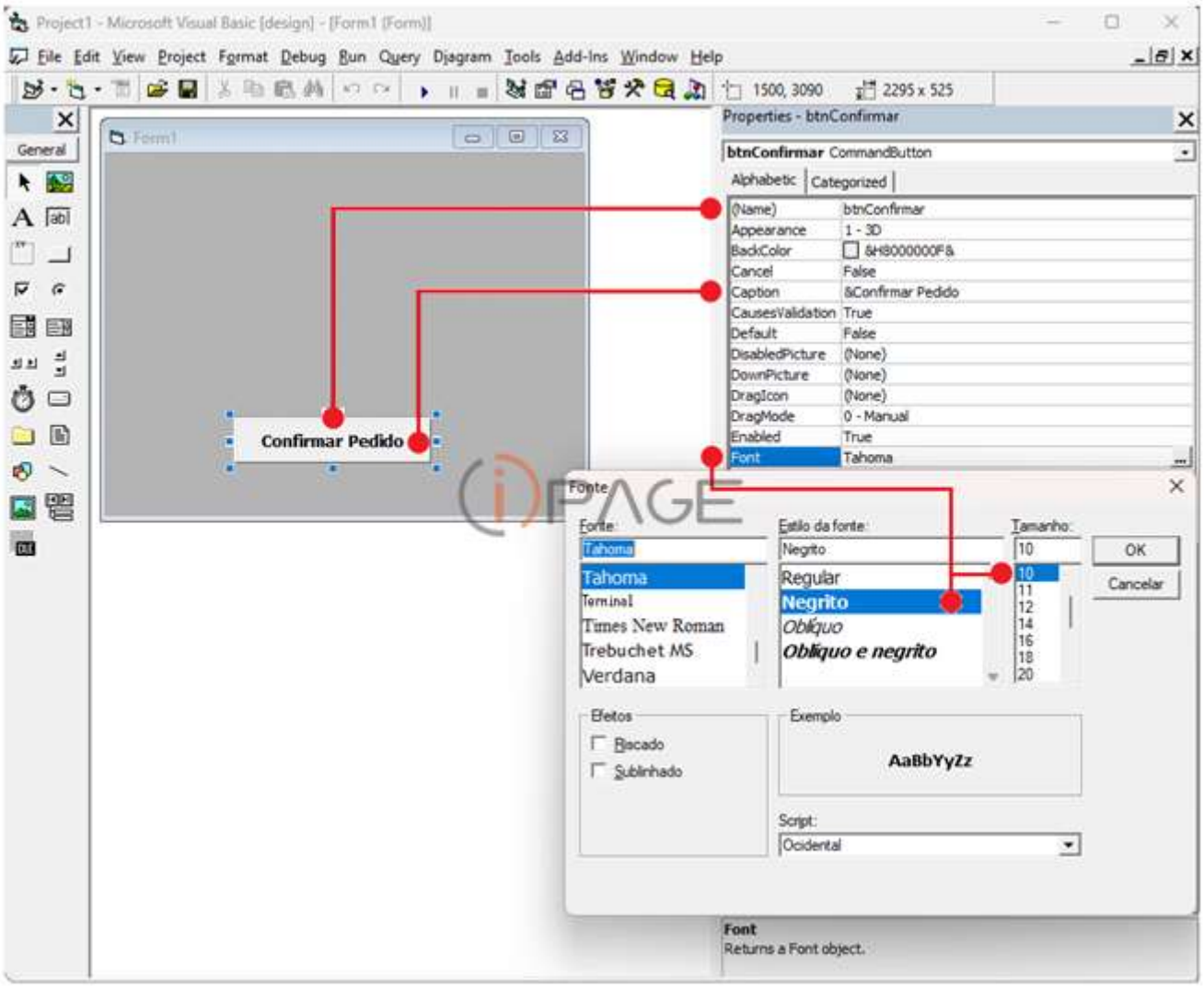


Figura 9

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O QUE SÃO CONTROLES?

São os componentes visuais e funcionais (também conhecidos como elementos de interface ou widgets) que você arrasta da caixa de ferramentas (ToolBox) e solta em cima de um formulário (Form) para construir a interface gráfica da aplicação e interagir com o usuário.

Pense nos controles como as peças de montar de uma tela: botões, caixas de texto, listas, rótulos e imagens que dão forma e utilidade ao software.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO



Imagine que o Formulário é a bancada de trabalho vazia e os Controles são as ferramentas de uma oficina (martelo, chave de fenda, medidores, botões de liga/desliga). Você pega essas ferramentas prontas na caixa, coloca na bancada e programa o que acontece quando o usuário aperta ou interage com cada uma delas.

OS TRÊS TIPOS PRINCIPAIS DE CONTROLES NO VB 6.0

No ecossistema do VB 6.0, os controles dividem-se em três categorias básicas:

1. Controles Intrínsecos (Padrão):
   - Já vêm embutidos nativamente na caixa de ferramentas padrão do VB 6.0, prontos para uso imediato.
   - Exemplos: CommandButton (botão), TextBox (caixa de texto), Label (rótulo/texto fixo), ListBox (lista), ComboBox (caixa de combinação/seleção), CheckBox (caixa de marcação), OptionButton (botão de rádio) e PictureBox/Image (para exibir imagens).

2. Controles ActiveX/Externos (.ocx):
   - São componentes adicionais fornecidos pela Microsoft ou por terceiros que podem ser adicionados ao projeto quando você precisa de recursos avançados.
   - Exemplos: O controle de calendário (MonthView), barras de progresso avançadas (ProgressBar), controle de árvore (TreeView) ou grades de dados mais complexas (MSFlexGrid).

3. Controles de Forma e Desenho (Shapes e Lines):
   - Elementos puramente visuais usados para desenhar linhas, retângulos ou círculos estáticos diretamente no formulário para organizar o layout visual.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

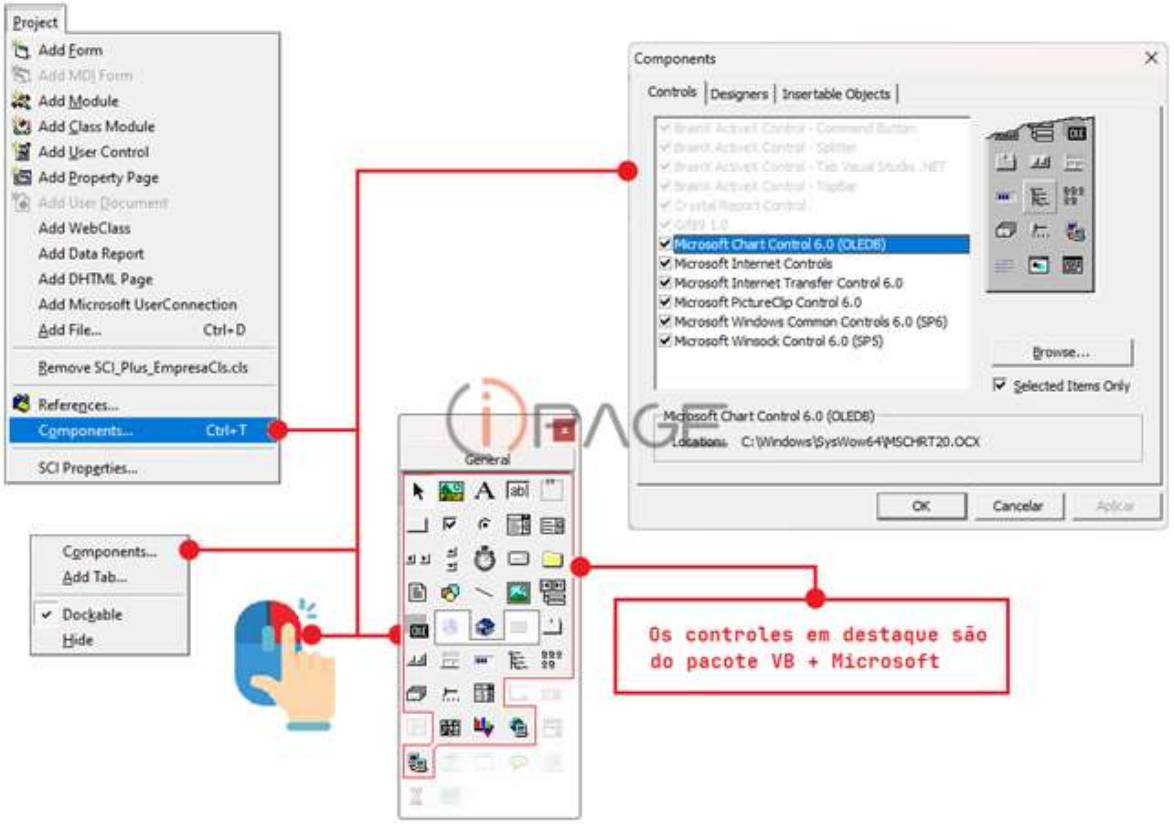


Figura 10

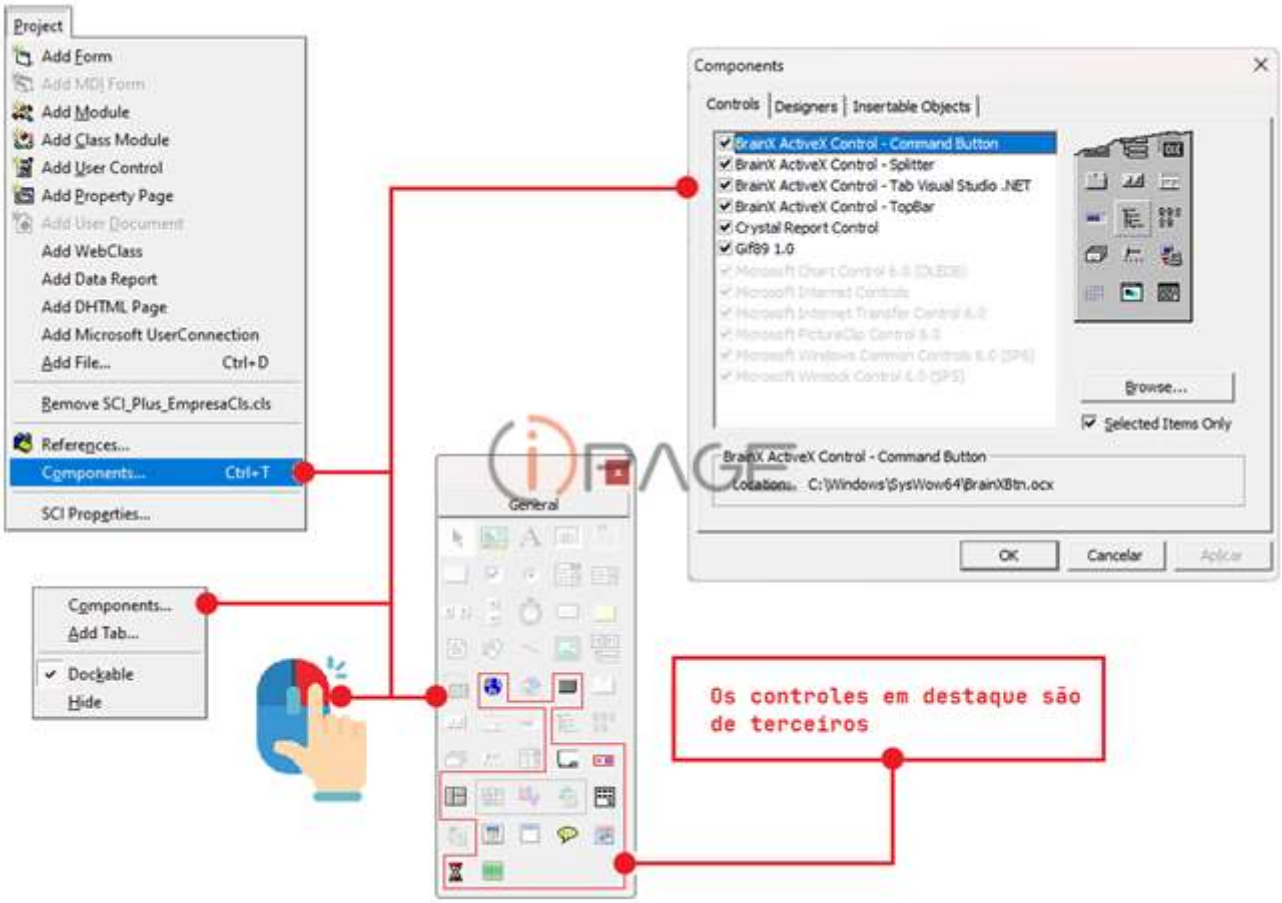


Figura 11

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMO FUNCIONA UM CONTROLE NA PRÁTICA?

Cada controle em VB 6.0 possui uma tríade fundamental para o desenvolvimento:

1. Propriedades (O aspecto visual e identificação):

Define como o controle se parece e como é chamado no código.

- Name: O identificador interno (ex: txtNomeCliente, btnSalvar).
- Caption ou Text: O texto que aparece escrito para o usuário.
- Visible/Enabled: Controla se o componente está visível ou habilitado para clique.

2. Eventos (A reação às ações do usuário):
É o gatilho disparado quando o usuário interage com o controle.

- Exemplo: O evento Click de um botão (CommandButton1_Click()) executa o código programado assim que o usuário clica nele com o mouse.

3. Métodos (As ações que o controle pode executar):
Comandos que você chama via código para manipular o controle em tempo de execução.

- Exemplo: O método SetFocus foca o cursor em uma caixa de texto, ou Refresh atualiza a exibição de uma lista.

MÓDULO BAS

Um Módulo BAS (cujo nome vem da extensão de arquivo .bas) é um módulo de código padrão no VB 6.0.



Figura 12

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Para entender de forma simples, pense em um projeto em VB 6.0 como uma construção:

Os Formulários (.frm) são as telas visíveis com as quais o usuário interage (botões, caixas de texto, imagens).

O Módulo BAS (.bas) é os bastidores da aplicação: um arquivo invisível para o usuário que guarda códigos, funções, cálculos, regras de negócio e variáveis que precisam ser acessados por qualquer tela ou parte do sistema.

POR QUE USAR UM MÓDULO BAS?

Imagine que o seu sistema tem 10 telas diferentes e todas elas precisam:

- Testar se um CPF é válido.
- Formatar um valor numérico para moeda (R$ 0,00).
- Conectar ao banco de dados ou ler o arquivo .dat.

Se você colocasse essas funções dentro de cada formulário, teria que duplicar o mesmo código 10 vezes. Se precisasse mudar uma regra, teria que alterar tela por tela.

Com o Módulo BAS, você escreve a função uma única vez e todas as telas do projeto conseguem usá-la.

O QUE COSTUMA FICAR DENTRO DE UM MÓDULO BAS?

1. Funções e Procedimentos Globais (Public Sub/Public Function)

Rotinas reaproveitáveis que podem ser chamadas de qualquer lugar:

Bas:

```
' Função global para verificar se um número é par
Public Function EhPar(ByVal Numero As Long) As Boolean
    EhPar = (Numero Mod 2 = 0)
End Function
```

2. Variáveis Globais (Public)

Variáveis que mantêm seu valor durante toda a execução do programa e são enxergadas por todas as telas (ex: dados do usuário logado, caminho do banco de dados):

Bas:

```
Public g_NomeUsuarioLogado As String
Public g_CaminhoBancoDeDados As String
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

3. Definições de UDTs (User-Defined Types)

Estruturas de dados personalizadas (como a estrutura de produtos ou clientes):

Bas:

```
Public Type ProdutoRecord
    Codigo As String * 5
    Nome As String * 40
    PrecoCompra As Double
End Type
```

4. Declarações de APIs do Windows (Declare Sub/Function)

Chamadas para funções nativas do próprio Windows (DLLs do sistema):

Código Em VB 6.0

Bas:

```
Public Declare Sub Sleep Lib "kernel32" (ByVal dwMilliseconds As Long)
```

5. O Procedimento Inicial do Sistema (Sub Main)

Um Módulo BAS pode conter a rotina especial Sub Main(). Em muitos sistemas profissionais em VB 6.0, em vez de abrir uma tela diretamente ao iniciar o programa, a execução começa na Sub Main(), onde o sistema carrega configurações, conecta ao banco de dados, verifica licenças e só depois abre a tela principal.

RESUMO DAS CARACTERÍSTICAS

CARACTERÍSTICA	MÓDULO BAS (.bas)	FORMULÁRIO (.frm)
Interface Visual?	Não (apenas código puro)	Sim (telas, botões, ícones)
Instanciação na memória?	Não precisa (carregado automaticamente)	Precisa ser carregado (Load/Show)
Escopo das funções	Globais por padrão (Public)	Locais ao formulário por padrão
Uso Principal	Regras globais, utilitários, conexão e inicialização	Interação do usuário com a tela

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ORIENTAÇÃO OBJETO(POO)

Para ensinar orientação a objetos em VB 6.0 de forma clara, o segredo é mostrar que uma Classe é o molde (ou projeto) e o Objeto é o item real construído a partir desse molde.



Figura 13

No VB 6.0, uma classe fica em um arquivo separado com a extensão .cls (Class Module). Ela permite encapsular dados e comportamentos, elevando o nível de organização do código em comparação com os módulos .bas.

1. A Analogia Didática Para Compreensão



Imagine que a Classe é o desenho técnico (a planta) de um carro. A planta em si não anda na rua. Mas quando você usa a planta para fabricar um carro de verdade, você cria uma Instância (um Objeto). A partir de uma única planta (Classe), você pode criar vários carros (Objetos) com cores e donos diferentes.

2. Os Quatro Pilares de uma Classe em VB 6.0

Uma classe em VB 6.0 é composta essencialmente por quatro elementos:

Variáveis Privadas: Guardam os dados internos do objeto de forma protegida (encapsulamento).

Propriedades (Property Let/Property Get): Funcionam como as portas de entrada e saída para ler ou alterar esses dados com validações.

Métodos (Public Sub/Public Function): São as ações que o objeto pode executar.

Eventos de Ciclo de Vida (Class_Initialize/Class_Terminate): O que acontece quando o objeto "nasce" e quando "morre" na memória.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

3. Exemplo Prático de Código (clsContaBancaria.cls)

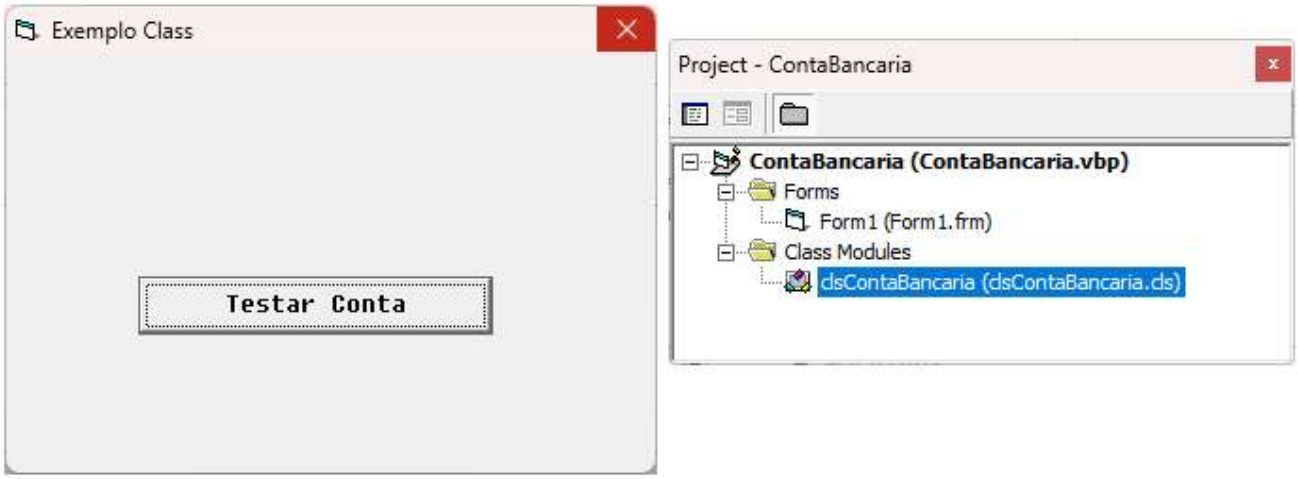


Figura 14

Crie no VB 6.0 um Class Module e salve com o nome clsContaBancaria.cls:

Class:

```
' ========================================================
' 1. VARIÁVEIS PRIVADAS (Estado Interno)
' ========================================================
Private m_Titular As String
Private m_Saldo As Double
' ========================================================
' 2. CICLO DE VIDA DO OBJETO
' ========================================================
' Executado automaticamente quando o objeto é instanciado (Set ... = New)
Private Sub Class_Initialize()
    m_Titular = "Não informado"
    m_Saldo = 0
End Sub
' Executado quando o objeto é destruído da memória (Set ... = Nothing)
Private Sub Class_Terminate()
    ' Limpeza de memória ou registro de logs, se necessário
End Sub

' ========================================================
' 3. PROPRIEDADES (Encapsulamento com Validação)
' ========================================================
' Leitura do Titular
Public Property Get Titular() As String
    Titular = m_Titular
End Property

' Escrita do Titular
Public Property Let Titular(ByVal Value As String)
    m_Titular = Trim(Value)
End Property
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

```
' Leitura do Saldo (Apenas leitura externamente, sem Property Let para o Saldo!)
Public Property Get Saldo() As Double
    Saldo = m_Saldo
End Property

' ======================================================
' 4. MÉTODOS (Comportamentos/Ações)
' ======================================================
' Método para depositar dinheiro
Public Sub Depositar(ByVal Valor As Double)
    If Valor > 0 Then
        m_Saldo = m_Saldo + Valor
    Else
        MsgBox "O valor do depósito deve ser positivo!", vbExclamation, "Atenção"
    End If
End Sub

' Método para sacar dinheiro
Public Function Sacar(ByVal Valor As Double) As Boolean
    If Valor > 0 And Valor <= m_Saldo Then
        m_Saldo = m_Saldo - Valor
        Sacar = True ' Saque realizado com sucesso
    Else
        MsgBox "Saldo insuficiente ou valor inválido!", vbCritical, "Erro"
        Sacar = False ' Falha no saque
    End If
End Function
```

4. Como Instanciar e Usar o Objeto em um Formulário (frmPrincipal.frm)

Vejamos como criar e manipular os objetos no formulário principal:

Form:

```
Private Sub btnTestarConta_Click()
    ' 1. Declaração da variável do tipo da Classe
    Dim minhaConta As clsContaBancaria
    ' 2. Instanciação (Criação do objeto na memória -> dispara Class_Initialize)
    Set minhaConta = New clsContaBancaria
    ' 3. Uso das Propriedades e Métodos
    minhaConta.Titular = "Guilherme Santos"
    minhaConta.Depositar 500.00

    MsgBox "Titular: " & minhaConta.Titular & vbCrLf & _
           "Saldo Inicial: R$ " & minhaConta.Saldo, vbInformation, "Conta Criada"

    ' Tentando realizar um saque
    If minhaConta.Sacar(150.00) Then
        MsgBox "Saque de R$ 150,00 realizado! Saldo atual: R$ " & minhaConta.Saldo
    End If
    ' 4. Destruição do objeto (Dispara Class_Terminate e libera a RAM)
    Set minhaConta = Nothing
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

RESUMO DAS CARACTERÍSTICAS

CONCEITO	MÓDULO BAS (.bas)	MÓDULO DE CLASSE (.cls)
Abordagem	Programação procedural	Programação Orientada a Objeto (POO)
Instanciação	Não é instanciado (código fixo)	Precisa de `Set obj = New NomeClasse`
Encapsulamento	Variáveis Public ficam expostas sem controle	Uso de `Property Get/Let` para proteger os dados
Múltiplas Cópias	Existe apenas um estado na memória para todo o sistema	Você pode criar centenas de objetos a partir da mesma classe
Limpeza	Fica na memória até o programa fechar	Liberado da RAM imediatamente ao fazer `Set obj = Nothing`

FORMULÁRIOS(FORMS)

O QUE SÃO FORMULÁRIOS?

No VB 6.0, os formulários (Forms) são as janelas físicas e visuais que compõem a interface do usuário (User Interface - UI). Eles funcionam como a tela em branco onde você desenha os botões, caixas de texto e tabelas, além de abrigarem o código-fonte responsável por reagir aos cliques e ações do utilizador.

Para quem está no início da programação visual, entender os tipos de formulários e o seu ciclo de vida é o primeiro passo para estruturar sistemas desktop profissionais.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO.



Imagine que o Formulário é uma folha de papel em branco. Em cima dela, você cola recortes (os controles). Dependendo de como você configura esse formulário, ele pode ser a janela principal que inicia o programa, uma caixa de diálogo que obriga o espectador a responder antes de continuar, ou uma janelinha interna que fica presa dentro de uma mesa de trabalho.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ESTILOS DE BORDAS DE UM FORMULÁRIO

Ao adicionar um formulário ao seu projeto, podemos modificar o estilo do mesmo modificando a propriedade **BorderStyle**.

Esta propriedade determina o tipo de borda, o comportamento de redimensionamento e o visual da barra de título de um formulário no VB 6.0.

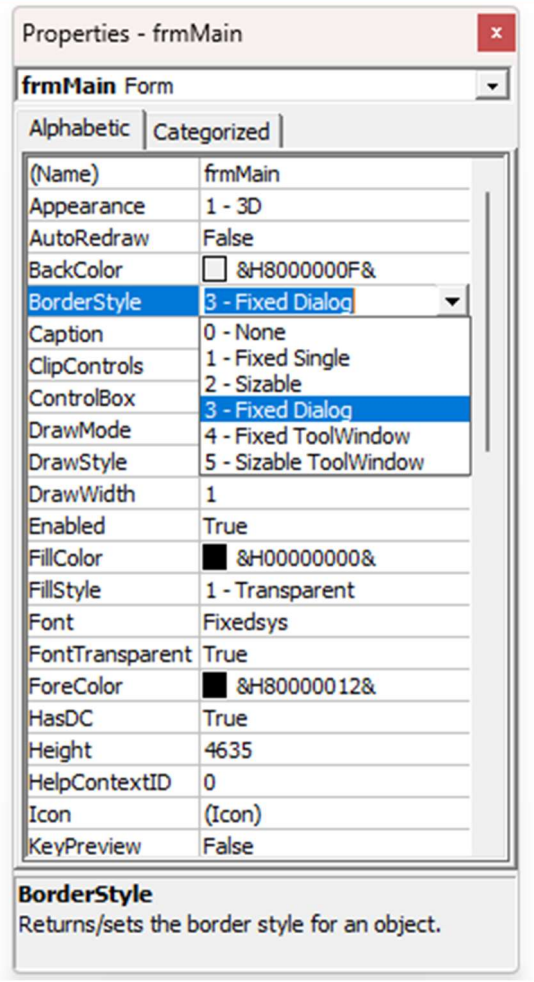


Figura 15

Abaixo estão detalhados os 6 tipos de janelas e estilos de borda disponíveis nessa seleção:

0 - None: O formulário é exibido completamente sem bordas e sem barra de título superior. É muito utilizado para criar telas de abertura (Splash Screens) ou interfaces personalizadas onde o desenvolvedor desenha os próprios elementos visuais de fechamento.

1 - Fixed Single: O formulário possui uma borda fina e fixa. O utilizador não consegue redimensionar a janela arrastando as extremidades com o mouse.

2 - Sizable: É o comportamento padrão do Windows para janelas normais. O formulário pode ser redimensionado livremente pelo utilizador, contando com os botões tradicionais de minimizar, maximizar e fechar.

3 - Fixed Dialog: Projetado especificamente para caixas de diálogo. A janela tem bordas fixas (não redimensionáveis) e exibe uma barra de título mais limpa, geralmente contendo apenas o botão de fechar.

4 - Fixed ToolWindow: Cria uma janela de ferramentas com uma barra de título mais estreita e letras menores. A janela não pode ser redimensionada e é ideal para paletas flutuantes ou painéis de opções secundários.

5 - Sizable ToolWindow: Mantém o formato compacto de janela de ferramentas (com a barra de título fina), mas permite que o utilizador altere o tamanho da janela arrastando as bordas livremente.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

TIPOS DE FORMULÁRIOS

FORMULÁRIO SDI (frm)

Em VB 6.0, SDI é a sigla para Single Document Interface (Interface de Documento Único).

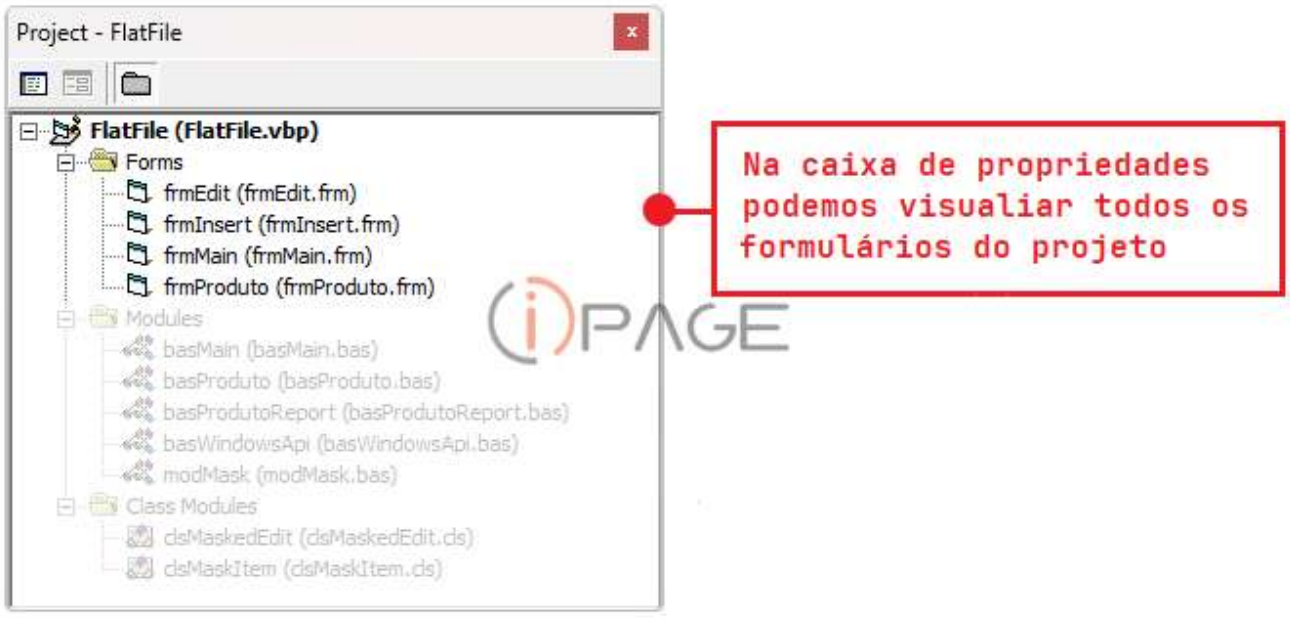


Figura 16

É o modelo tradicional de interface gráfica em que cada janela (formulário) da aplicação existe de forma independente e autônoma na área de trabalho do sistema operacional ou dentro do sistema.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO.



Imagine o Bloco de Notas (Notepad) do Windows. Se você abrir três arquivos de texto ao mesmo tempo, o Windows vai abrir três janelas separadas do Bloco de Notas, cada uma com sua própria barra de tarefas, seu próprio menu e seus botões de fechar, minimizar e maximizar. Esse é o modelo SDI.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMPARAÇÃO: SDI vs. MDI

Para entender bem o SDI, vale a pena compará-lo com seu irmão, o MDI (Multiple Document Interface):

CARACTERÍSTICA	SDI (Single Document Interface)	MDI (Multiple Document Interface)
Estrutura	Janelas independentes/avulsas.	Uma janela "mãe" (*MDIForm*) que contém várias janelas "Filhas" (*MDIChild*) presas dentro dela.
Exemplo do Windows	Bloco de Notas, Calculadora, Paint.	Photoshop antigo, Excel antigo, AutoCAD.
Comportamento	Cada formulário pode ser movido livremente pela tela fora dos limites dos outros.	Os formulários filhos ficam restritos à área interna do formulário mãe.
Barra de Tarefas	Pode exibir um ícone/botão para cada janela aberta.	Exibe apenas um ícone principal para o aplicativo.

COMO FUNCIONA O SDI NO VB 6.0?

No VB 6.0, todo formulário padrão (Form) criado a partir do menu Add Form é por padrão um formulário SDI.

Para verificar ou configurar a propriedade no formulário:

Clique no formulário no Designer.
Na janela de Properties (Propriedades), procure por MDIChild.
Se estiver definido como False (padrão), o formulário é SDI.

Para verificar ou configurar a propriedade no formulário:

1. Clique no formulário no Designer.
2. Na janela de Properties (Propriedades), procure por MDIChild.
3. Se estiver definido como False (padrão), o formulário é SDI.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

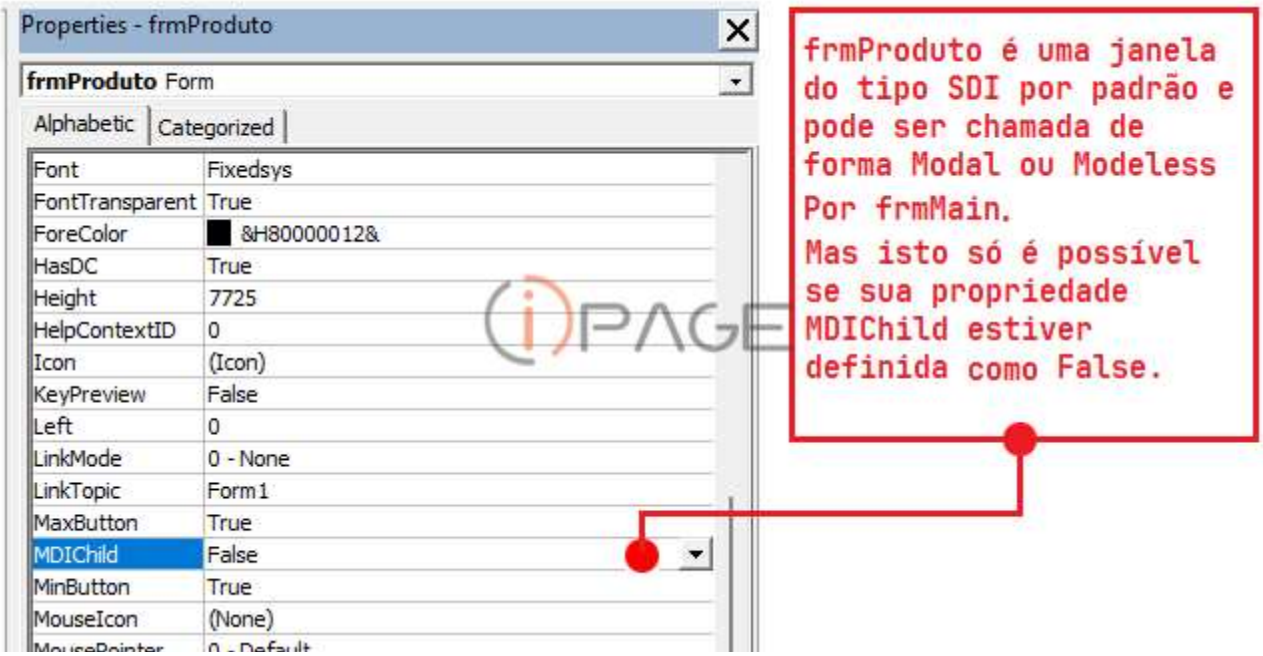


Figura 17

Form:

```
' Exemplo de chamada de um formulário SDI a partir do formulário principal:

Private Sub btnAbrirCadastro_Click()
    ' Exibe o formulário de cadastro como uma janela SDI independente
    frmProduto.Show vbModal
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

MODOS DE EXIBIÇÃO NO SDI:

O formulário SDI pode ser aberto de duas formas:

- Modal (frmCadastro.Show vbModal): Trava a navegação nas outras janelas do sistema até que a janela atual seja fechada (ideal para caixas de diálogo, confirmações ou telas de inserção como o frmInsert).
- Non-Modal (frmCadastro.Show vbModeless ou apenas frmCadastro.Show): Permite que o usuário continue clicando e interagindo com outras janelas do sistema enquanto a mesma permanece aberta.

O QUE VOCÊ PRECISA SABER DO MODELO SDI?

Vantagens:

- Simplicidade de Desenvolvimento: Muito mais simples de projetar, codificar e gerenciar o ciclo de vida das telas.
- Flexibilidade na Área de Trabalho: O usuário pode arrastar janelas para monitores diferentes no caso de uso de múltiplos monitores.
- Foco Único: Ideal para telas funcionais e objetivas, como cadastros simples, configurações e caixas de diálogo.

Desvantagens:

- Poluição Visual: Se a aplicação abrir muitas telas ao mesmo tempo, a área de trabalho do usuário e a barra de tarefas do Windows ficam desorganizadas.
- Gestão de Telas Abertas: Se não houver um controle adequado via código, o usuário pode abrir acidentalmente várias cópias da mesma tela (ex: abrir a tela de Cadastro de Clientes 5 vezes).

Exemplo:

frmMain está com a propriedade MDIChild definida como False, ou seja, é uma janela SDI.

Para chamar a janela frmProduto que também é uma janela SDI na opção do submenu Cadastro de produtos temos o seguinte trecho do código:

Form:

```
Private Sub mnuCadProdutos_Click()
  frmProduto.Show vbModal
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FLUXO DE FUNCIONAMENTO DE CHAMADA MODAL, SDI PARA SDI.

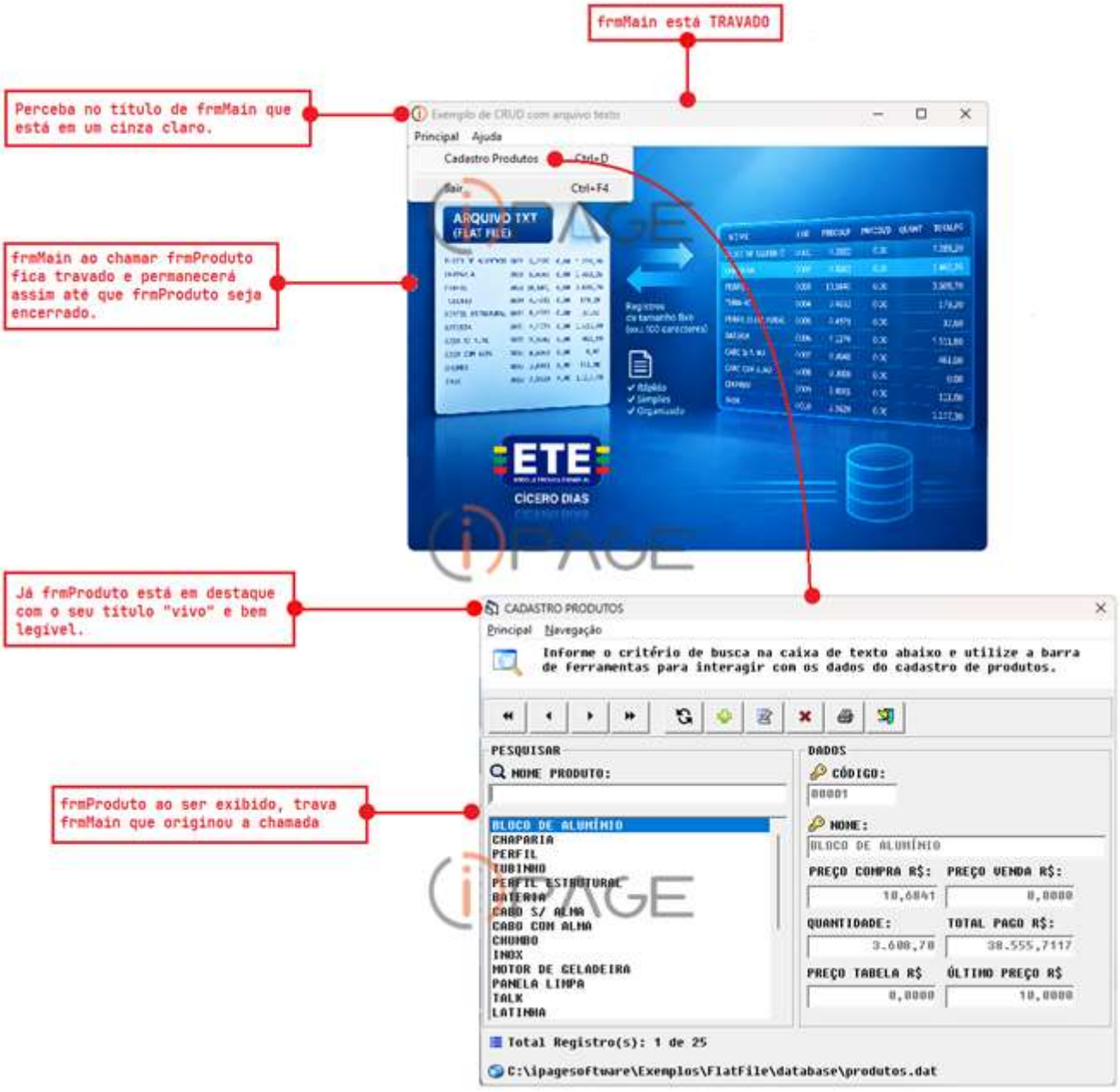


Figura 18

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FLUXO DE FUNCIONAMENTO DE CHAMADA MODELESS, MDI PARA SDI.

Na imagem abaixo temos o funcionamento típico de uma janela MDI(mãe) e uma janela SDI(filha) com a propriedade MDIChild = True.

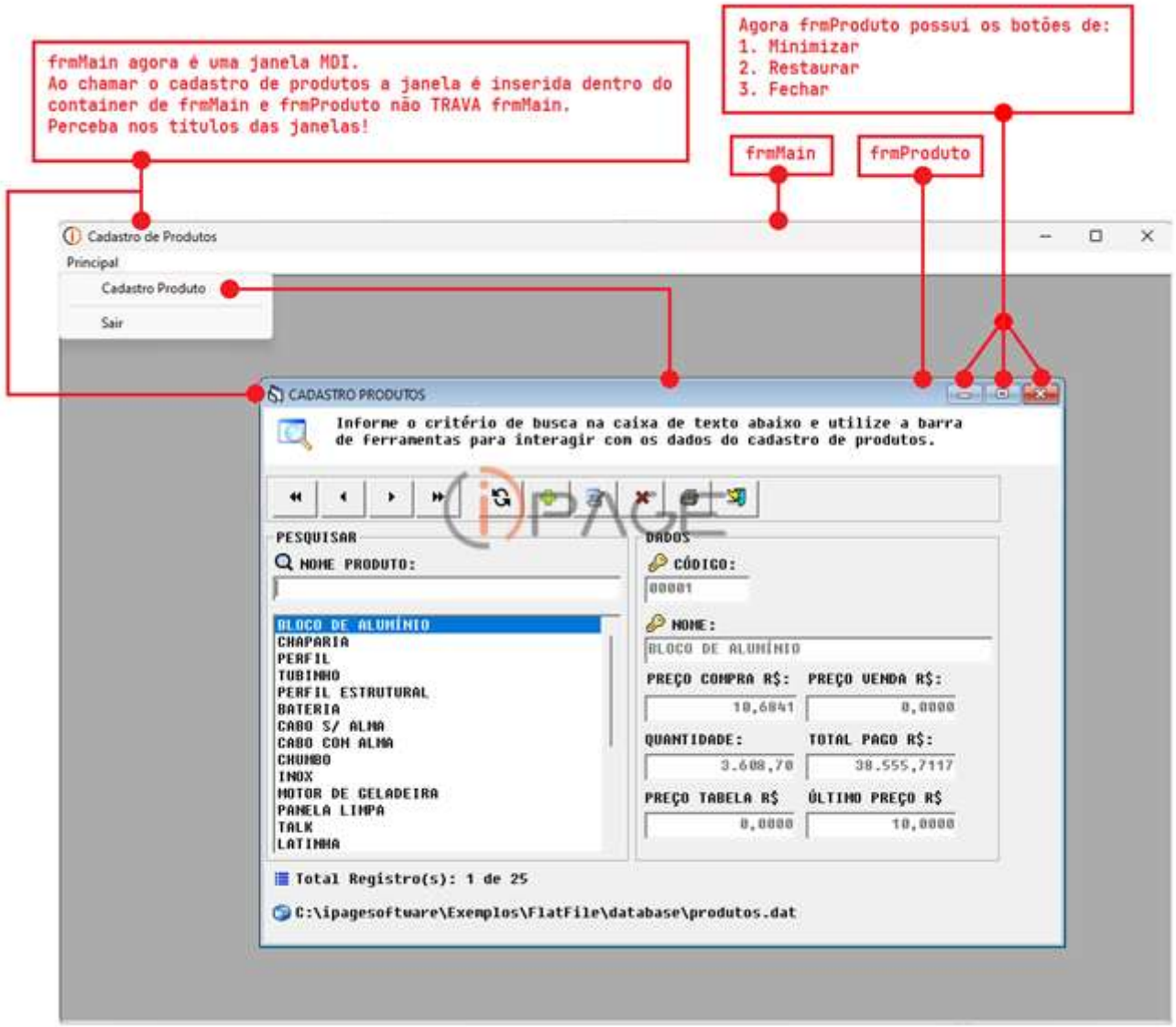


Figura 19

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

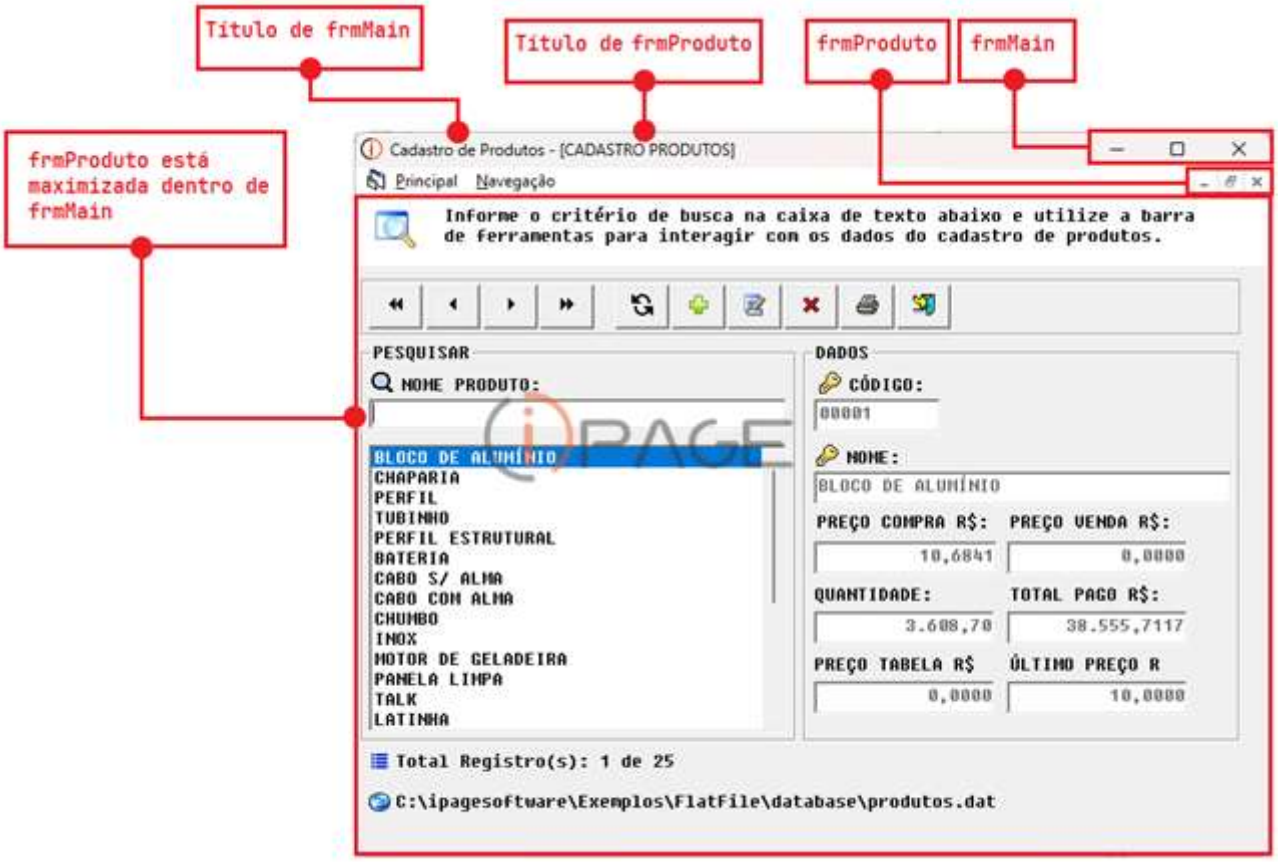


Figura 20

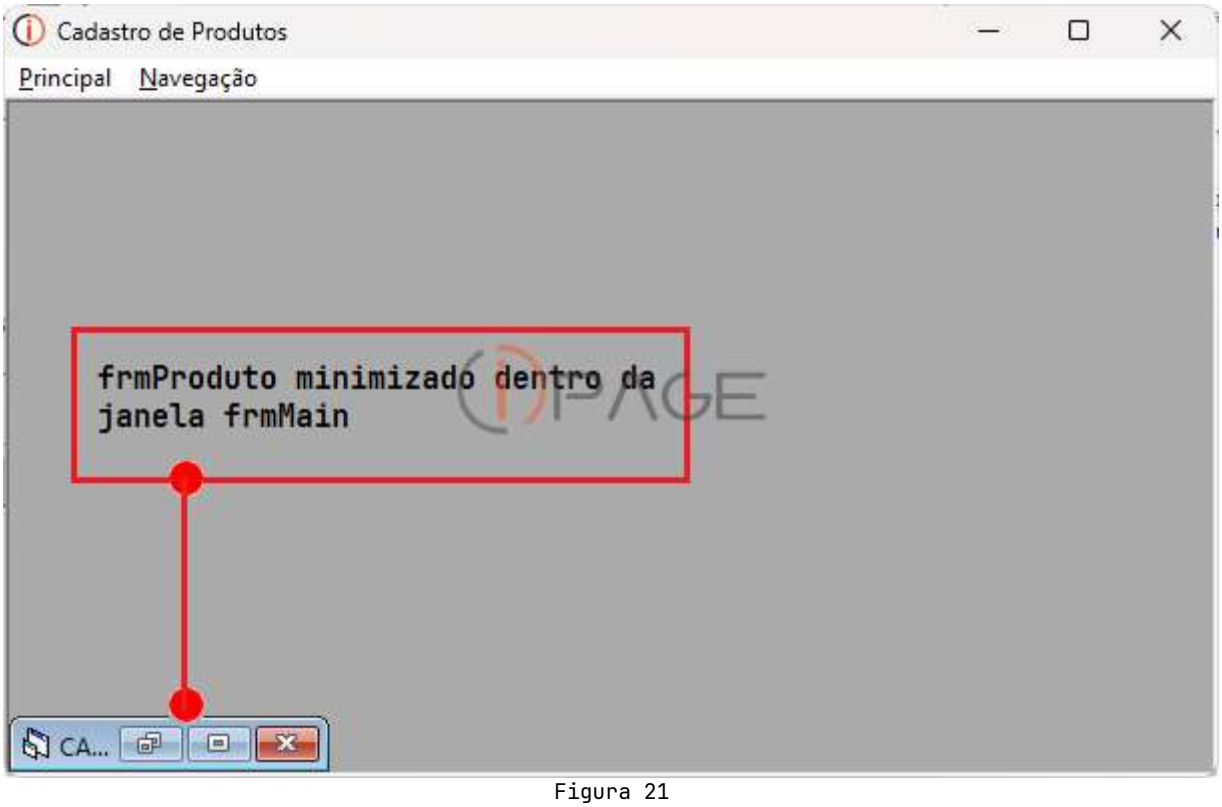


Figura 21

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Para entender o funcionamento de uma janela MDI (Multiple Document Interface) em conjunto com formulários filhos que possuem a propriedade MdiChild = True, vamos usar uma analogia direta e explicar a mecânica no VB 6.0.

A ANALOGIA DIDÁTICA: O ESCRITÓRIO E AS PASTAS

Imagine que o sistema é uma grande mesa de escritório (a janela MDI Parent).

- Janela MDI Principal (MDIForm): É a moldura/limite da mesa de trabalho. Ela ocupa a tela e serve como o "ambiente contenedor".
- Formulário com MdiChild = True: São os papéis ou pastas de trabalho que ficam espalhados estritamente em cima dessa mesa.

O usuário pode abrir vários papéis ao mesmo tempo, mas nenhum deles consegue sair para fora da mesa de escritório. Se você tentar arrastar uma janela filha para fora da janela MDI principal, ela será cortada pelas bordas do container.

COMO FUNCIONA NA PRÁTICA NO VB 6.0?

No ecossistema do Visual Basic 6.0, o relacionamento MDI funciona através de uma hierarquia rígida entre dois tipos específicos de formulários:

O FORMULÁRIO MÃE(MDIForm):

- É criado no menu do projeto como Add MDI Form.
- Funciona como a janela principal da aplicação. Apenas uma janela MDI principal pode existir em um projeto padrão do VB 6.0.
- Ela não serve para colocar botões ou caixas de texto diretamente (com exceção de barras de ferramentas ou menus superiores globais). O fundo dela costuma ser cinza padrão do Windows.

O FORMULÁRIO FILHO(Form com MdiChild = True):

- Qualquer formulário comum (Form) do projeto pode se tornar "filho" alterando a propriedade MdiChild de False para True na janela de Propriedades.
- Restrição Obrigatória: Um formulário com MdiChild = True só pode ser aberto se já houver um MDIForm carregado na memória. Se você tentar dar um Show em um formulário filho sem que a janela MDI mãe esteja ativa, o VB 6.0 gerará um erro em tempo de execução.

COMPORTAMENTO VISUAL E DE COMANDOS

Quando você define a propriedade MdiChild = True em um formulário, ele assume características muito peculiares:

Confinamento Total: O formulário fica preso dentro da área de cliente do MDIForm. Ele não pode ser arrastado para fora da janela principal nem para a barra de tarefas do Windows de forma independente.

Minimização Inteligente: Quando você minimiza uma janela filha dentro de um MDI, ela encolhe para o canto inferior esquerdo da janela mãe (como ícones internos), em vez de ir para a barra de tarefas do sistema operacional.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Menus Compartilhados: Se o MDIForm tiver um menu principal e o formulário filho também tiver, o menu do formulário filho funde-se (merge) automaticamente com o menu do formulário mãe quando o formulário filho recebe o foco.

EM RESUMO

SDI (MdiChild = False): Janelas soltas e independentes, como abas ou janelas avulsas do Bloco de Notas.

MDIChild (MdiChild = True): Janelas presas e gerenciadas por uma janela-mãe central, muito comuns em sistemas ERP legados onde o sistema inteiro roda dentro de uma única grande tela maximizada.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

NOMECLATURA ADOTADA NO PROJETO

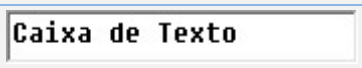
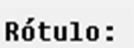
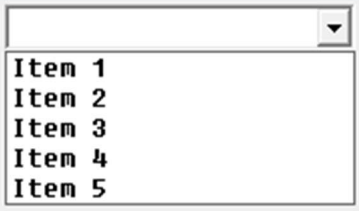
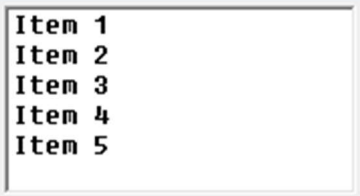
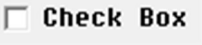
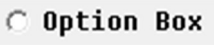
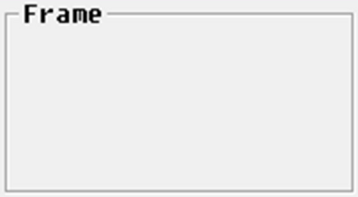
A comunidade de desenvolvedores VB adotou a Convenção de Nomenclatura Padrão (fortemente baseada na Notação Húngara).

Essa prática consiste em utilizar um prefixo de 3 letras em minúsculo para identificar o tipo do objeto ou da variável, seguido pelo nome descritivo com a primeira letra de cada palavra em maiúsculo (CamelCase).

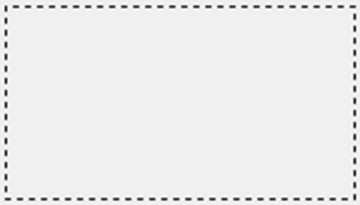
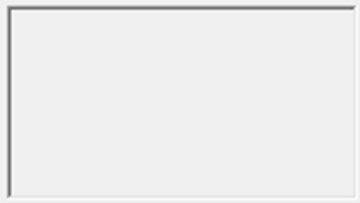
Adotar esse padrão melhora drasticamente a legibilidade, já que o VB 6.0 não possui um ambiente de desenvolvimento moderno com tooltips(balão de dicas) avançados para mostrar o tipo da variável a todo momento.

1. Controles de Formulário (Controles Comuns)

Para os componentes visuais inseridos nos formulários (frm), utilizam-se os seguintes prefixos padrão:

CONTROLE	PREFIXO	TIPO DE CONTROLE	EXEMPLO DE USO
	btn ou cmd	Command Button (Botão)	cmdSalvar, btnCancelar
	txt	Text Box (Caixa de Texto)	txtNomeCliente, txtTotal
	lbl	Label (Rótulo de Texto)	lblInstrucoes, lblResultado
	cbo	Combo Box (Caixa de Listagem Suspensa)	cboEstados, cboTipoPagamento
	lst	List Box (Caixa de Lista)	lstProdutos Selecionados
	chk	Check Box (Caixa de Seleção)	chkAtivo, chkAceitaTermos
	opt	Option Button (Botão de Opção/Radio)	optMasculino, optFeminino
	fra	Frame (Moldura/Agrupador)	fraDadosPessoais

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

	img	Image (Imagem mais leve)	imgLogoEmpresa
	pic	Picture Box (Caixa de Imagem complexa)	picGrafico
	dg ou dbg	DataGrid/DataBound Grid	dbgHistorico

2. Variáveis (Tipos de Dados)

Para as variáveis declaradas no código (Dim, Private, Public), o prefixo indica o tipo de dado que ela armazena:

PREFIXO	TIPO DE DADO	EXEMPLO DE USO
str ou s	String (Texto)	strNome, strCaminhoArquivo
int ou n	Integer (Inteiro curto)	intContador, intIdade
lng ou l	Long (Inteiro longo)	lngCodigoCliente, lngTotalRegistros
bln ou b	Boolean (Verdadeiro/Falso)	blnSucesso, blnPreenchido
cur ou c	Currency (Valores Monetários)	curSalario, curPrecoUnitario
dbl	Double (Ponto flutuante de precisão dupla)	dblPercentual, dblFatorMultiplicador
dt ou dte	Date (Data e Hora)	dtDataNascimento, dteHoraInicio
vnt ou v	Variant (Tipo genérico - evitar se possível)	vntDadosGerais
obj	Object (Objetos genéricos ou de Classes)	objConexao, objExcel

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

3. Escopo das Variáveis

No VB 6.0, também é uma boa prática adicionar um prefixo extra de uma letra antes do tipo de dado para identificar onde a variável foi declarada (o seu escopo):

- Sem prefixo de escopo: Variáveis locais (declaradas dentro de uma Sub ou Function)
- Exemplo: strNome
- m: Variáveis de nível de Módulo ou Formulário (Module/Form level), declaradas na seção General Declarations.
    - Exemplo: mstrUsuarioLogado, mintTentativas
- g ou global: Variáveis Globais, declaradas como Public dentro de um módulo padrão (.bas).
    - Exemplo: gstrCaminhoBanco, gblnSistemaIniciado

4. Componentes e Estruturas do Projeto

Para os arquivos que compõem o seu projeto VB 6.0, utilize as extensões lógicas em seus nomes internos:

- frm para Formulários: frmCadastroClientes, frmMenuPrincipal.
- mdl ou bas para Módulos de Código Padrão: mdlFuncoesGlobais, mdlConexaoBD.
- cls para Módulos de Classe: clsCliente, clsNotaFiscal.
- rpt para Relatórios (Crystal Reports ou Data Report): rptVendasMensais.

REGRAS DE OURO PARA UM CÓDIGO LIMPO

1. Sempre use **Option Explicit**: Coloque isso na primeiríssima linha de todos os formulários, módulos e classes. Isso obriga você a declarar todas as variáveis, evitando bugs invisíveis causados por erros de digitação (onde o VB 6.0 criaria uma variável **Variant** vazia automaticamente).
2. Use Caixa Alta e Baixa (CamelCase): Facilita a leitura rápida. txtCodigoProduto é muito mais legível que txtcodigoproduto.
3. Evite abreviações obscuras: Prefira cmdCalcularTotal a cmdCalcTot.
   Se você estiver mantendo um sistema legado ou iniciando uma refatoração, manter essa consistência tornará o código muito mais fácil de ler e depurar.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMENTÁRIOS

No Visual Basic 6.0, documentar o código utilizando REM ou o apóstrofo (') é uma prática importante para facilitar a manutenção, o aprendizado e a evolução de um sistema. Entretanto, existe uma diferença entre documentar bem um programa e simplesmente colocar comentários em todas as linhas.

Considerando seu projeto em VB 6.0, como o módulo basProdutoReport, que gera relatórios HTML a partir de arquivos de tamanho fixo, vou explicar as vantagens, as desvantagens e as boas práticas.

COMO FUNCIONAM OS COMENTÁRIOS NO VB 6.0?

O VB 6.0 permite inserir comentários de duas maneiras.
Usando o apóstrofo (') — recomendado

```
' Define o caminho padrão do banco de dados.
If Len(dataBaseProduto) = 0 Then
    dataBaseProduto = App.Path & "\database\produtos.dat"
End If
```

É a forma mais utilizada, pois é curta, legível e pode ser usada para comentar linhas inteiras ou trechos após uma instrução.

USANDO O REM (Remark)

```
Rem Define o caminho padrão do banco de dados.
If Len(dataBaseProduto) = 0 Then
    dataBaseProduto = App.Path & "\database\produtos.dat"
End If
```

Também é válido, mas precisa ser usado como instrução de comentário. O apóstrofo costuma ser mais prático no código moderno em VB 6.0.

As duas formas servem para explicar o código e não são executadas como instruções funcionais do programa.

Uma diferença prática: o apóstrofo pode aparecer depois de uma instrução na mesma linha.

```
TotalRegistros = LOF(ArqOrigem) \ Len(Produto) ' Calcula o total de registros
```

Já o REM é normalmente utilizado no início de uma linha própria, não como comentário ao final de uma instrução.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

VANTAGENS DE UMA BOA DOCUMENTAÇÃO

1. Facilita a compreensão do código

Um comentário explica a intenção da rotina, especialmente quando a lógica não é óbvia.

```
' Calcula quantos registros completos existem no arquivo.
TotalRegistros = LOF(ArqOrigem) \ Len(Produto)
```

Quem lê o código entende por que o tamanho do arquivo é dividido pelo tamanho da estrutura TProduto.

2. Facilita a manutenção

Quando você voltar a um módulo depois de meses ou anos, os comentários ajudam a recuperar rapidamente o raciocínio original, sem precisar reconstruir toda a lógica.

3. Ajuda na formação de novos programadores

Em um projeto didático, comentários bem escritos mostram não apenas o que o código faz, mas também por que determinada solução foi escolhida. Isso é especialmente útil para quem está aprendendo programação.

4. Ajuda na depuração e na análise de erros

Comentários que identificam etapas, condições e regras de negócio tornam mais fácil localizar o trecho responsável por um problema.

5. Preserva o conhecimento do projeto

Registrar decisões importantes evita que o conhecimento fique apenas na memória de quem desenvolveu o sistema.

DESVANTAGENS DE DOCUMENTAR EXCESSIVAMENTE

Documentar é importante, mas o excesso pode prejudicar a leitura e a manutenção.
Comentários redundantes

Veja este exemplo:

```
' Incrementa o contador em uma unidade.
Contador = Contador+1
```

O comentário apenas repete o que a instrução já informa. Em geral, não agrega valor.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

UMA DOCUMENTAÇÃO MELHOR EXPLICA A FINALIDADE:

```
' Permite até 30 registros por página.
Const TOTAL_LINHAS_POR_PAGINA As Integer = 27
```

O comentário contradiz o código. Isso pode levar alguém a tomar uma decisão errada durante uma manutenção.

Regra importante: quando alterar o código, revise os comentários relacionados.

POLUIÇÃO VISUAL

Imagine um módulo com centenas de linhas e comentários repetindo operações simples. O programador precisará ler muito mais texto para localizar a lógica realmente importante.

FALSA SENSAÇÃO DE SEGURANÇA

Um código muito comentado não é necessariamente um código bem escrito. Os comentários podem estar incorretos, incompletos ou simplesmente descrever uma implementação confusa.

O ideal é combinar documentação útil com nomes claros, funções bem definidas e uma estrutura coerente.

BOAS PRÁTICAS PARA DOCUMENTAR PROJETOS VB 6.0

Minha recomendação é utilizar o apóstrofo (') como padrão principal e manter uma convenção consistente em todos os módulos.

LOCAL	O QUE DOCUMENTAR?
Cabeçalho do módulo	Finalidade, responsabilidade e dependências
Cabeçalho de uma rotina	O que faz, parâmetros, retorno e efeitos colaterais
Regras de negócio	Por que a regra existe e quais são suas condições
Operações complexas	A lógica que não é evidente pela leitura
Tratamento de erros	O que pode falhar e como o sistema reage
Constantes importantes	Significado e motivo do valor definido
APIs e bibliotecas externas	Dependências, requisitos e particularidades

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

UM MODELO DE CABEÇALHO PARA UMA ROTINA DO SEU PROJETO:

```
'--------------------------------------------------------------------------------
' Módulo      : basProdutoReport
' Rotina      : GerarRelatorio
' Finalidade  : Gera um relatório de produtos em HTML.
'
' Funcionamento:
'   1. Abre o arquivo de dados de produtos.
'   2. Calcula a quantidade de registros.
'   3. Divide os registros em páginas de até 27 linhas.
'   4. Preenche o template HTML.
'   5. Salva o relatório e solicita sua abertura no navegador padrão.
'
' Dependências:
'   CarregarTemplate, GetBody, CriarArquivoTemp, HtmlEncode e SalvarReport.
'
' Observação:
'   Os registros do arquivo devem ser compatíveis com a estrutura TProduto.
'--------------------------------------------------------------------------------
Public Sub GerarRelatorio()
```

Esse cabeçalho oferece uma visão geral da rotina sem obrigar o leitor a examinar todas as linhas para compreender seu propósito.

AFINAL, DEVO USAR REM OU (')?

Prefira o apóstrofo (').

É mais conciso e visualmente discreto.
Permite comentários no final de uma instrução.
É a convenção mais comum em projetos VB 6.0.
Funciona bem com cabeçalhos, blocos explicativos e comentários pontuais.

Reserve REM para casos específicos.
Ele continua válido no VB 6.0, mas não oferece uma vantagem prática que justifique misturá-lo ao apóstrofo em um projeto novo ou em uma padronização de código.

CONCLUSÃO

Uma boa documentação deve responder principalmente a três perguntas:

O que esta rotina faz?

Por que ela foi implementada desta maneira?

Quais condições, limitações ou dependências preciso conhecer antes de modificá-la?

Para seus projetos VB 6.0, especialmente os que também servem como material de estudo, recomendo comentários nos cabeçalhos dos módulos e procedimentos, explicações nas regras importantes e comentários pontuais para operações técnicas menos evidentes.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O melhor comentário não é aquele que explica cada linha: é aquele que economiza o tempo de quem precisará compreender e manter o código no futuro.

VARIÁVEIS DE MEMÓRIA

No Visual Basic 6.0, as variáveis são os espaços reservados na memória RAM para armazenar dados temporários durante a execução do programa.

Definir o tipo correto de variável otimiza o consumo de memória, garante a precisão dos cálculos e evita erros de estouro (overflow).

1. Classificação dos Tipos de Dados

Os tipos de dados no VB 6.0 dividem-se em duas grandes categorias: Tipos Primitivos (ou Fundamentais) e Tipos Especiais/Complexos.

A. Tipos Numéricos Inteiros

Utilizados para armazenar números inteiros (sem casas decimais).

- **Byte**: Ocupa 1 byte de memória. Armazena valores de 0 a 255. Ideal para pequenas contagens ou dados binários.
- **Integer**: Ocupa 2 bytes (16 bits). Armazena valores de -32.768 a 32.767.
- **Long (Long Integer)**: Ocupa 4 bytes (32 bits). Armazena valores de -2.147.483.648 a 2.147.483.647. É o tipo inteiro padrão recomendado em arquiteturas de 32 bits pelo desempenho superior.

B. Tipos Numéricos Decimais (Ponto Flutuante e Monetário)

Utilizados para cálculos que exigem casas decimais.

- **Single**: Ocupa 4 bytes. Números de ponto flutuante de precisão simples (ideal para cálculos científicos aproximados).
- **Double**: Ocupa 8 bytes. Números de ponto flutuante de dupla precisão, com maior capacidade de casas decimais.
- **Currency**: Ocupa 8 bytes. É um tipo de dado fixo otimizado especificamente para cálculos financeiros e monetários, evitando os erros de arredondamento comuns em decimais comuns. Suporta 4 casas decimais fixas à direita da vírgula.

C. Tipos de Texto e Lógicos

- **String**: Armazena sequências de caracteres. Pode ser de tamanho variável (cresce dinamicamente na memória) ou de tamanho fixo (muito utilizado em estruturas de arquivos planos .dat/UDTs).
- **Boolean**: Ocupa 2 bytes. Assume apenas dois valores lógicos: True (-1) ou False (0).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

D. Data e Tipos Especiais

- **Date**: Ocupa 8 bytes. Armazena datas (desde 1 de janeiro de 100 até 31 de dezembro de 9999) e horas simultaneamente.
- **Variant**: É o tipo coringa do VB 6.0. Ocupa 16 bytes (ou mais) na memória porque pode armazenar qualquer tipo de dado (texto, número, data ou até objetos) e adapta-se automaticamente. Deve ser evitar o excesso, pois consome mais processamento e memória RAM.
- **Object**: Ocupa 4 bytes. Armazena referências a ponteiros de objetos na memória (instâncias de classes, formulários ou componentes ActiveX).

2. Exemplos Práticos para Fixação

Abaixo encontra-se um exemplo prático demonstrando a declaração, a atribuição e a exibição de diferentes tipos de variáveis em um procedimento do VB 6.0:

Bas:

```
Public Sub ExemplosVariaveisMemoria()
    ' 1. Declaração explícita de variáveis com a instrução Dim
    Dim idadeCliente As Integer
    Dim codigoProduto As Long
    Dim precoUnitario As Currency
    Dim nomeProduto As String
    Dim dataCadastro As Date
    Dim ativoNoSistema As Boolean
    ' 2. Atribuição de valores
    idadeCliente = 35
    codigoProduto = 102938
    precoUnitario = 154.90          ' Ideal para valores financeiros
    nomeProduto = "Bloco de Aluminio"
    dataCadastro = Now              ' Data e hora atual do sistema
    ativoNoSistema = True
    ' 3. Uso prático e exibição dos dados
    Dim mensagem As String
    mensagem = "Produto: " & nomeProduto & vbCrLf & _
               "Código: " & codigoProduto & vbCrLf & _
               "Preço: R$ " & Format(precoUnitario, "Standard") & vbCrLf & _
               "Cadastrado em: " & Format(dataCadastro, "dd/mm/yyyy hh:nn") & vbCrLf & _
               "Ativo: " & ativoNoSistema

    MsgBox mensagem, vbInformation, "Demonstração de Tipos de Dados"
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

3. Boas Práticas de Declaração de Variáveis

1. Utilizar sempre Option Explicit: Inserida no topo de módulos e formulários, esta diretiva obriga o programador a declarar todas as variáveis antes de usá-las, evitando erros de digitação de nomes de variáveis em tempo de execução.
2. Escolher o menor tipo adequado: Se uma variável vai armazenar apenas o estado de um botão (ligado/desligado), utilize Boolean em vez de Long. Isso otimiza o espaço na memória RAM de sistemas legados.
3. Evitar o uso excessivo de Variant: Embora flexível, o Variant desacelera a aplicação e oculta erros de tipagem que poderiam ser prevenidos em tempo de desenvolvimento.

O QUE É UMA VARIÁVEL UDT?

No VB 6.0, **UDT** é a sigla para **User-Defined Type** (Tipo Definido pelo Usuário).

Trata-se de uma estrutura de dados personalizada que permite agrupar diferentes variáveis de tipos distintos (como **String**, **Integer**, **Double**, **Boolean**) sob um único nome e tipo.

Em outras linguagens de programação, como **C** ou **C++**, o **UDT** equivale a uma **struct** (estrutura) ou a um **Record** no **Pascal/Delphi**.

PARA QUE SERVE UM UDT?

Um **UDT** serve para organizar dados relacionados de forma lógica e estruturada. Em vez de criar e gerenciar várias variáveis avulsas para representar uma entidade (ex: NomeCliente, IdadeCliente, EnderecoCliente), você cria um único tipo chamado Cliente que contém todos esses campos.

SINTAXE DA DECLARAÇÃO.

A declaração de um **UDT** é feita utilizando o bloco Type ... End Type.

Geralmente, os **UDTs** são declarados na seção de declarações de um Módulo (.bas) ou de um Formulário/Módulo de Classe (.frm/.cls):

Bas:

```
' Declaração da estrutura (tipo de dado customizado)
Public Type Cliente
    Codigo As Long
    Nome As String * 50     ' String de tamanho fixo
    CPF As String * 11
    ValorLimite As Double
    Ativo As Boolean
End Type
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Nota:



Em arquivos de código comum (.bas), o UDT pode ser declarado como Public ou Private.
Já dentro de Formulários (.frm) ou Classes (.cls), ele só pode ser declarado como Private.

COMO UTILIZAR NO CÓDIGO?

Depois de definir o tipo, você cria variáveis baseadas nele e acessa seus campos utilizando o operador ponto (.):

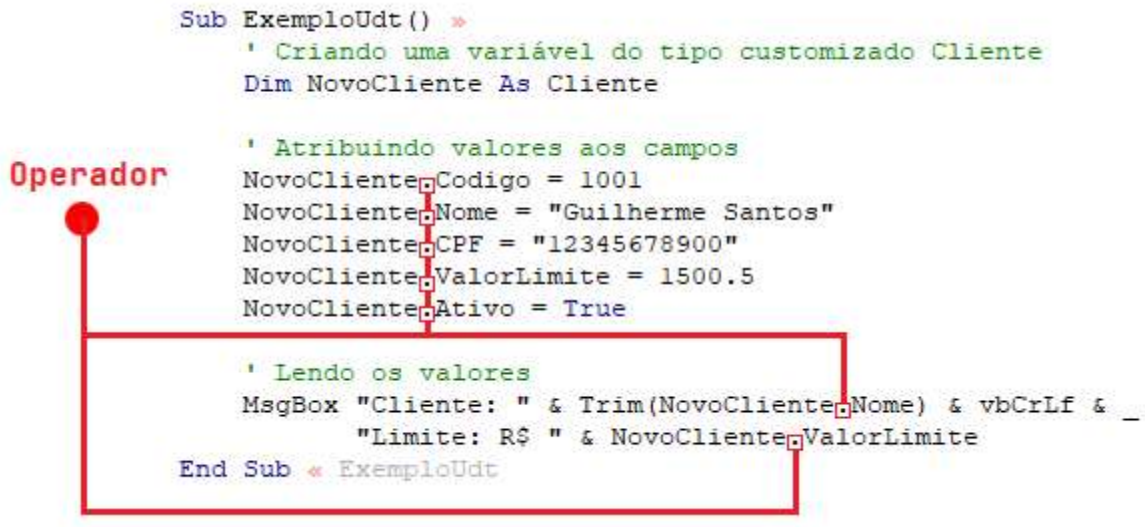


Figura 22

PRINCIPAIS APLICAÇÕES DO UDT NO VB 6.0

Persistência em Arquivos Planos (.dat/Flat Files):
O UDT é amplamente utilizado junto com a instrução Open ... For Random e os comandos Put e Get para gravar e ler registros de tamanho fixo diretamente em arquivos binários sem necessidade de banco de dados.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ARQUIVOS DE TAMANHO FIXO COM VARIÁVEIS UDT

Um arquivo de tamanho fixo ou Flat File (arquivo plano) é um arquivo de texto ou binário simples utilizado para armazenar dados organizados em registros e campos, sem a necessidade de um Sistema Gerenciador de Banco de Dados (SGBD) como SQL Server, MySQL ou Oracle.

Estrutura: Não possui tabelas com relacionamentos dinâmicos, índices complexos ou chaves estrangeiras gerenciadas pelo sistema.

TAMANHO FIXO:

Cada campo do registro possui um número fixo de caracteres (ex: Código ocupa 5 caracteres, Nome ocupa 40 caracteres).

Este formato é muito utilizado em sistemas legados, integrações simples, arquivos de configuração ou aplicações leves que precisam guardar informações diretamente no disco local.

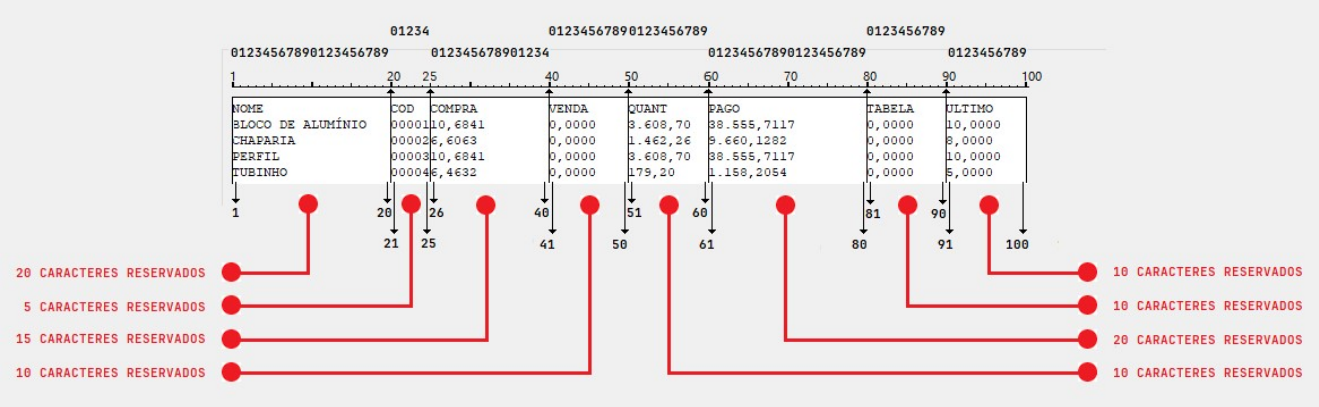


Figura 23

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

A esquerda da coluna vemos a definição uma variável UDT e a direita como é armazenada as informações em uma linha do arquivo.

ESTRUTURA DO ARQUIVO VARIÁVEL UDT	LINHA TEXTO COM CAMPOS FIXOS

```
Public Type TProduto
  Nome         As String * 20
  Codigo       As String * 5
  PrecoCompra  As String * 15
  PrecoVenda   As String * 10
  Quantidade   As String * 10
  TotalPago    As String * 20
  Tabela       As String * 10
  Ultimo       As String * 10
End Type
```

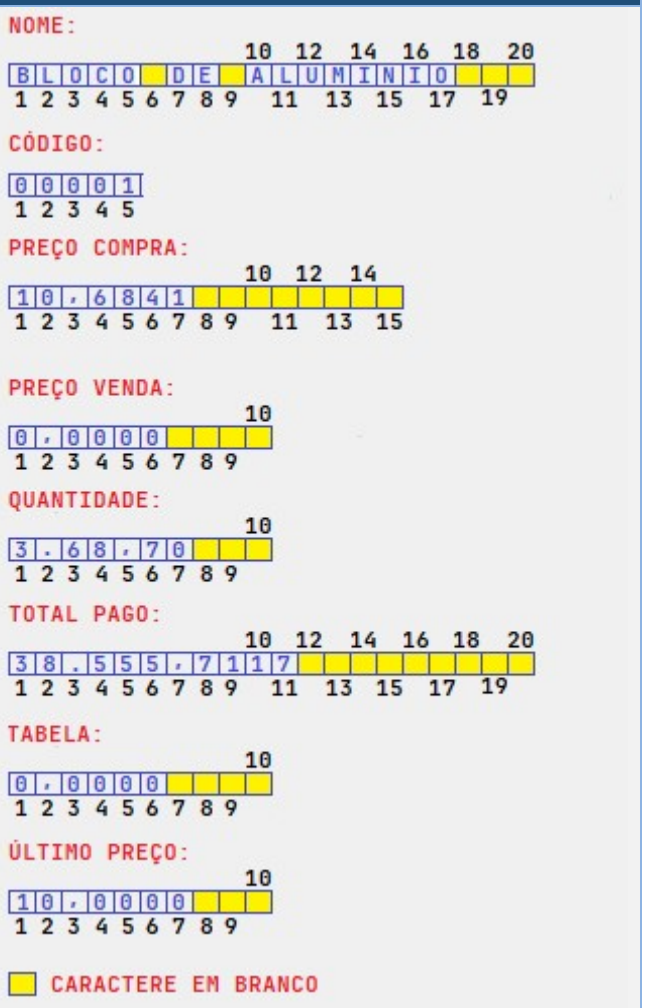


Figura 24

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

SUB ROTINAS

No VB 6.0, as sub-rotinas (frequentemente chamadas de Sub ou Subroutines) são blocos de código autônomos que realizam uma tarefa específica, mas com uma característica fundamental: elas executam uma ação e organizam o fluxo de código, mas não retornam um valor diretamente.

Pense nelas como uma "receita de bolo passo a passo" ou uma "ordem de serviço" que você guarda em um lugar separado e pode chamar a qualquer momento pelo nome.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO.



Imagine que você está a criar um sistema para uma loja e, toda vez que o cliente clica em 'Salvar', o sistema precisa fazer três coisas: limpar os campos da tela, focar o cursor no primeiro item e atualizar a lista de registros. Em vez de reescrever esse bloco de três códigos em cada botão, você coloca tudo dentro de uma Sub-rotina chamada LimparTela(). Sempre que precisar, basta escrever LimparTela no código e o computador executa todo o processo de uma só vez.

ESTRUTURA DE UMA SUB-ROTINA

No VB 6.0, uma sub-rotina começa sempre com a palavra-chave **Sub** e termina obrigatoriamente com **End Sub**. Elas podem ser públicas (**Public Sub**) para serem acessadas de qualquer lugar ou privadas (**Private Sub**) para serem usadas apenas dentro daquele formulário ou módulo.

EXEMPLO PRÁTICO DE UMA SUB-ROTINA SIMPLES:

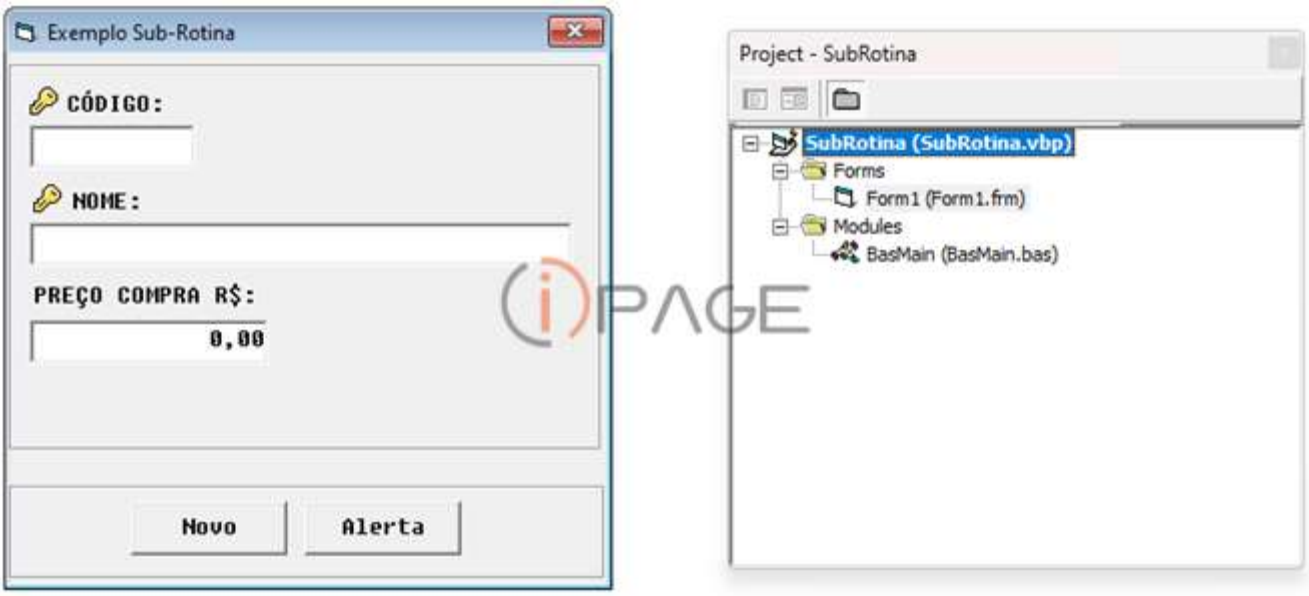


Figura 25

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

BasMain:

```
' Declaração de uma Sub-rotina privada que limpa os campos de cadastro
Private Sub LimparCamposFormulario()
    txtCodigo.Text = ""
    txtNome.Text = ""
    txtPrecoCompra.Text = "0,00"
    txtNome.SetFocus ' Retorna o cursor para a caixa de texto do nome
End Sub
```

Form1:

```
Option Explicit

Private Sub btnNovo_Click()
  ' Chama a sub-rotina que criamos acima
  Call LimparCamposFormulario
  MsgBox "Formulário limpo e pronto para novo cadastro!", vbInformation
End Sub

Private Sub cmdAlerta_Click()
  ExibirAlerta "O estoque deste produto está crítico!", "Atenção de Estoque"
End Sub

' Declaração de uma Sub-rotina privada que limpa os campos de cadastro
Private Sub LimparCamposFormulario()
  txtCodigo.Text = ""
  txtNome.Text = ""
  txtPrecoCompra.Text = "0,00"
  txtCodigo.SetFocus ' Retorna o cursor para a caixa de texto do nome
End Sub
```

COMO CHAMAR (EXECUTAR) A SUB-ROTINA?

Para disparar a ação que está guardada dentro da sub-rotina, basta escrever o nome dela em qualquer evento (como no clique de um botão):

Form:

```
Private Sub btnNovo_Click()
    ' Chama a sub-rotina que criamos acima
    Call LimparCamposFormulario
    MsgBox "Formulário limpo e pronto para novo cadastro!", vbInformation
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

SUB-ROTINAS COM PARÂMETROS (PASSAGEM DE ARGUMENTOS)

Você também pode criar sub-rotinas que recebem informações de fora para executar a tarefa de forma dinâmica, utilizando parâmetros:
Bas:

```
' Sub-rotina que recebe uma mensagem personalizada e exibe na tela
Public Sub ExibirAlerta(ByVal mensagem As String, ByVal tipoTitulo As String)
    MsgBox mensagem, vbExclamation, tipoTitulo
End Sub
```

Para chama-la no código:

```
ExibirAlerta "O estoque deste produto está crítico!", "Atenção de Estoque"
```

SUB-ROTINA (SUB) VS FUNÇÃO (FUNCTION)

Uma dúvida muito comum entre os alunos é a diferença entre as duas principais estruturas de código reutilizável no VB 6.0:

CARACTERÍSTICA	SUB-ROTINA(SUB)	FUNÇÃO(FUNCTION)
Objetivo Principal	Executar uma ação ou conjunto de comandos (ex: limpar tela, gravar dados).	Realizar um cálculo ou processamento e retornar um valor para quem a chamou.
Retorno de Dado	Não retorna valor diretamente.	Retorna um valor (ex: As Double, As String, As Boolean).
Forma de Chamada	Chamada isolada pelo nome (LimparCamposFormulario).	Atribuída a uma variável ou usada em condições (resultado = CalcularImposto()).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FUNÇÕES(Function)

No VB 6.0, uma Function (Função) é um bloco de código reutilizável projetado para realizar uma tarefa específica, processar dados e, diferente da sub-rotina (Sub), retornar obrigatoriamente um valor de volta para quem a chamou.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENSÃO



Imagine que você vai a uma padaria e pede ao atendente para calcular o valor total de 3 pães. Você entrega os dados de entrada (a quantidade e o preço unitário) — esses são os parâmetros. O atendente faz a conta na calculadora e devolve o resultado em dinheiro para você — esse é o valor de retorno da função.
A função é exatamente isso: um bloco de código que faz um trabalho e te devolve uma resposta pronta.

ESTRUTURA DE UMA FUNCTION

No VB 6.0, uma função começa com a palavra-chave Function, define os parâmetros entre parênteses e termina obrigatoriamente declarando o tipo de dado que ela vai retornar (ex: As Double, As String, As Boolean).

O valor de retorno é enviado de volta atribuindo-se o resultado diretamente ao próprio nome da função dentro do bloco de código.

Vamos criar um pequeno programa para um exemplo Prático de uma Function simples:

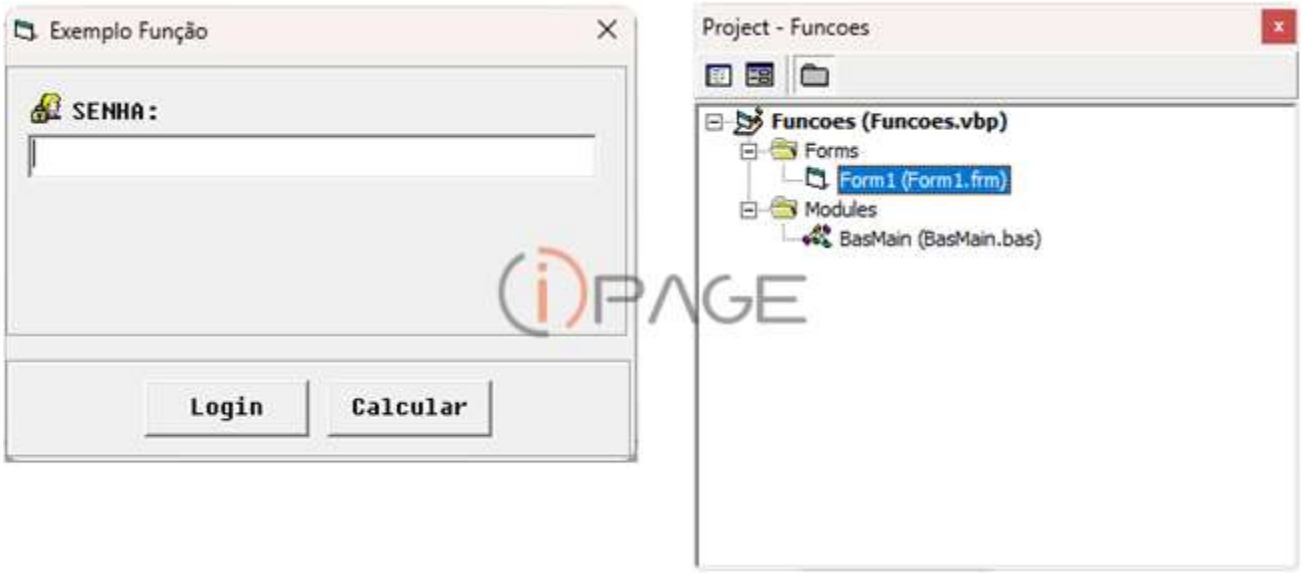


Figura 26

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Bas:

```
' Função que calcula o valor total de uma venda e retorna o valor (As Currency)
Public Function CalcularTotalVenda(ByVal Quantidade As Long, ByVal PrecoUnitario As Cur-
rency) As Currency
    Dim valorTotal As Currency

    ' Realiza o cálculo matemático
    valorTotal = Quantidade * PrecoUnitario

    ' O valor é retornado atribuindo-o ao nome da função
    CalcularTotalVenda = valorTotal
End Function
```

Form:

```
Option Explicit

' Função que calcula o valor total de uma venda e retorna o valor (As Currency)
Public Function CalcularTotalVenda(ByVal Quantidade As Long, _
                                   ByVal PrecoUnitario As Currency) As Currency
  Dim valorTotal As Currency
  ' Realiza o cálculo matemático
  valorTotal = Quantidade * PrecoUnitario
  ' O valor é retornado atribuindo-o ao nome da função
  CalcularTotalVenda = valorTotal
End Function

Public Function ValidarSenhaAdmin(ByVal SenhaDigitada As String) As Boolean
  If SenhaDigitada = "ipage" Then
    ValidarSenhaAdmin = True  ' Senha correta
  Else
    ValidarSenhaAdmin = False ' Senha incorreta
  End If
End Function
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMO CHAMAR E USAR A FUNÇÃO EM UM FORMULÁRIO?

Para utilizar o valor devolvido pela função, você costuma atribuí-lo a uma variável ou usá-lo diretamente em uma condição ou mensagem:

Form:

```
Private Sub btnCalcular_Click()
    Dim qtd As Long
    Dim preco As Currency
    Dim resultadoFinal As Currency

    qtd = 5
    preco = 45.50

    ' Chamando a função e guardando o valor retornado na variável
    resultadoFinal = CalcularTotalVenda(qtd, preco)

    MsgBox "O valor total da compra é: R$ " & Format(resultadoFinal, "Standard"),
vbInformation, "Cálculo Realizado"
End Sub
```

FUNÇÕES QUE RETORNAM VALORES LÓGICOS (BOOLEAN)

As funções são extremamente úteis para validações. Por exemplo, criar uma função que valida se um código ou senha está correto e retorna True ou False:

Bas:

```
Public Function ValidarSenhaAdmin(ByVal SenhaDigitada As String) As Boolean
    If SenhaDigitada = "VB 6.0master" Then
        ValidarSenhaAdmin = True  ' Senha correta
    Else
        ValidarSenhaAdmin = False ' Senha incorreta
    End If
End Function
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Uso prático no botão de acesso:

Form:

```
Private Sub btnLogin_Click()
    If ValidarSenhaAdmin(txtSenha.Text) = True Then
        MsgBox "Acesso autorizado!", vbInformation
    Else
        MsgBox "Senha inválida!", vbCritical
    End If
End Sub
```

SUB-ROTINA (SUB) VS. FUNÇÃO (FUNCTION)

Uma dúvida muito comum entre os alunos é a diferença entre as duas principais estruturas de código reutilizável no VB 6.0:

CARACTERÍSTICA	SUB-ROTINA(SUB)	FUNÇÃO(FUNCTION)
Objetivo Principal	Executar uma sequência de comandos ou ações (ex: limpar campos, abrir tela).	Realizar um cálculo ou validação e retornar um dado.
Retorno de Dado	Não retorna valor.	Retorna obrigatoriamente um valor (As Integer, As String, etc.).
Atribuição	Chamada isolada pelo nome (LimparTela).	Atribuída a uma variável ou usada em expressões (x = CalcularJuros()).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

API’S DO WINDOWS

As APIs do Windows (**Application Programming Interfaces**) representam a ponte direta entre a sua aplicação e o sistema operacional subjacente (Windows).

Embora o VB 6.0 ofereça uma vasta gama de controles visuais e funções prontas, ele é uma camada de abstração. Há recursos avançados do sistema operacional que o VB 6.0 nativamente não consegue enxergar ou manipular. É aí que entram as Windows APIs: bibliotecas nativas do Windows escritas em C/C++ (como user32.dll, kernel32.dll e gdi32.dll) que liberam o "poder total" do sistema para dentro do seu código legado.

A ANALOGIA DIDÁTICA PARA MELHOR COMPREENÇÃO



Imagine que o VB 6.0 é um carro moderno e confortável, mas que vem com a velocidade limitada de fábrica para proteger o motorista. As APIs do Windows são as peças de alto desempenho que você encomenda diretamente na oficina central do fabricante. Quando você as instala, o carro ganha acesso à funções avançadas que não vinham no manual básico.

PARA QUE O VB 6.0 USA AS APIS DO WINDOWS?

Com o uso de APIs, um programa em VB 6.0 consegue realizar tarefas avançadas, tais como:

- Manipular janelas de outros aplicativos (encontrar janelas abertas, fechá-las ou alterar seus títulos).
- Acessar chaves profundas do Registro do Windows com mais flexibilidade.
- Criar efeitos visuais avançados que o VB 6.0 puro não suporta (como janelas translúcidas, cantos arredondados ou barras de tarefas personalizadas).
- Obter informações detalhadas de hardware e rede (espaço em disco de nível físico, endereço MAC, nome exato da máquina).

COMO O VB 6.0 "TIRA PROVEITO" DISSO? (A INSTRUÇÃO DECLARE)

Para usar uma função de uma API do Windows no VB 6.0, você não precisa instalar nenhum componente extra. O VB 6.0 possui a capacidade nativa de importar essas funções externas através da palavra-chave Declare, geralmente colocada no topo de um Módulo Padrão (.bas).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

EXEMPLO PRÁTICO: DESCOBRINDO O NOME DO COMPUTADOR

O Windows possui uma função nativa na biblioteca kernel32.dll chamada **GetComputerNameA**. Veja como o VB 6.0 a consome.

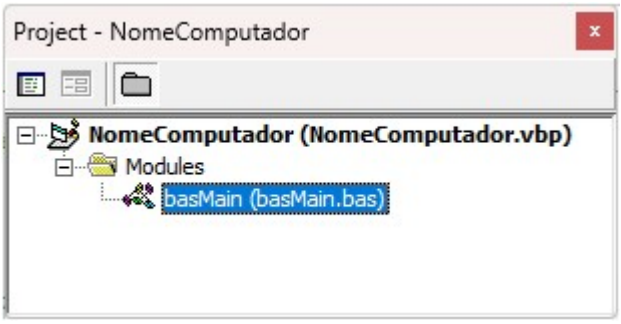


Figura 27

Bas:

```
Option Explicit

' 1. Declaração da API do Windows no topo de um Módulo (.bas)
Public Declare Function GetComputerName Lib "kernel32" Alias "GetComputerNameA"
(ByVal lpBuffer As String, ByRef nSize As Long) As Long

Public Sub Main()
  ExibirNomeDoComputador
End Sub

' 2. Uso prático da API dentro de uma rotina do sistema
Public Sub ExibirNomeDoComputador()
  Dim bufferNome    As String
  Dim tamanhoBuffer As Long
  Dim resultado     As Long
  ' Prepara o espaço de memória para receber o texto do Windows
  bufferNome = Space$(255)
  tamanhoBuffer = 255
  ' Chama a API nativa do Windows
  resultado = GetComputerName(bufferNome, tamanhoBuffer)

  If resultado <> 0 Then
    ' Remove os caracteres vazios extras gerados pelo buffer
    bufferNome = Left$(bufferNome, tamanhoBuffer)
    MsgBox "O nome desta máquina na rede é: " & bufferNome, vbInformation, "API do
Windows"
  Else
    MsgBox "Não foi possível recuperar o nome do computador.", vbCritical
  End If
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

CUIDADOS CRUCIAIS AO USAR APIS NO VB 6.0

Embora poderosas, as APIs exigem atenção redobrada do desenvolvedor, pois operam em baixo nível do sistema:

Mapeamento Rigoroso de Tipos de Dados: Um erro de conversão entre o tipo de dado do C/C++ da API e o VB 6.0 (por exemplo, confundir um Integer de 16 bits do VB 6.0 com um int de 32 bits, que no VB 6.0 deve ser Long) causa estouro de pilha e fecha a IDE do VB 6.0 instantaneamente sem aviso prévio.

Gerenciamento de Ponteiros e Memória: Como muitas APIs exigem ponteiros de memória (ByRef ou alocação de buffers com Space$), um deslize na manipulação de strings pode corromper a memória RAM e gerar erros de Access Violation.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FLUXOGRAMA DO PROJETO

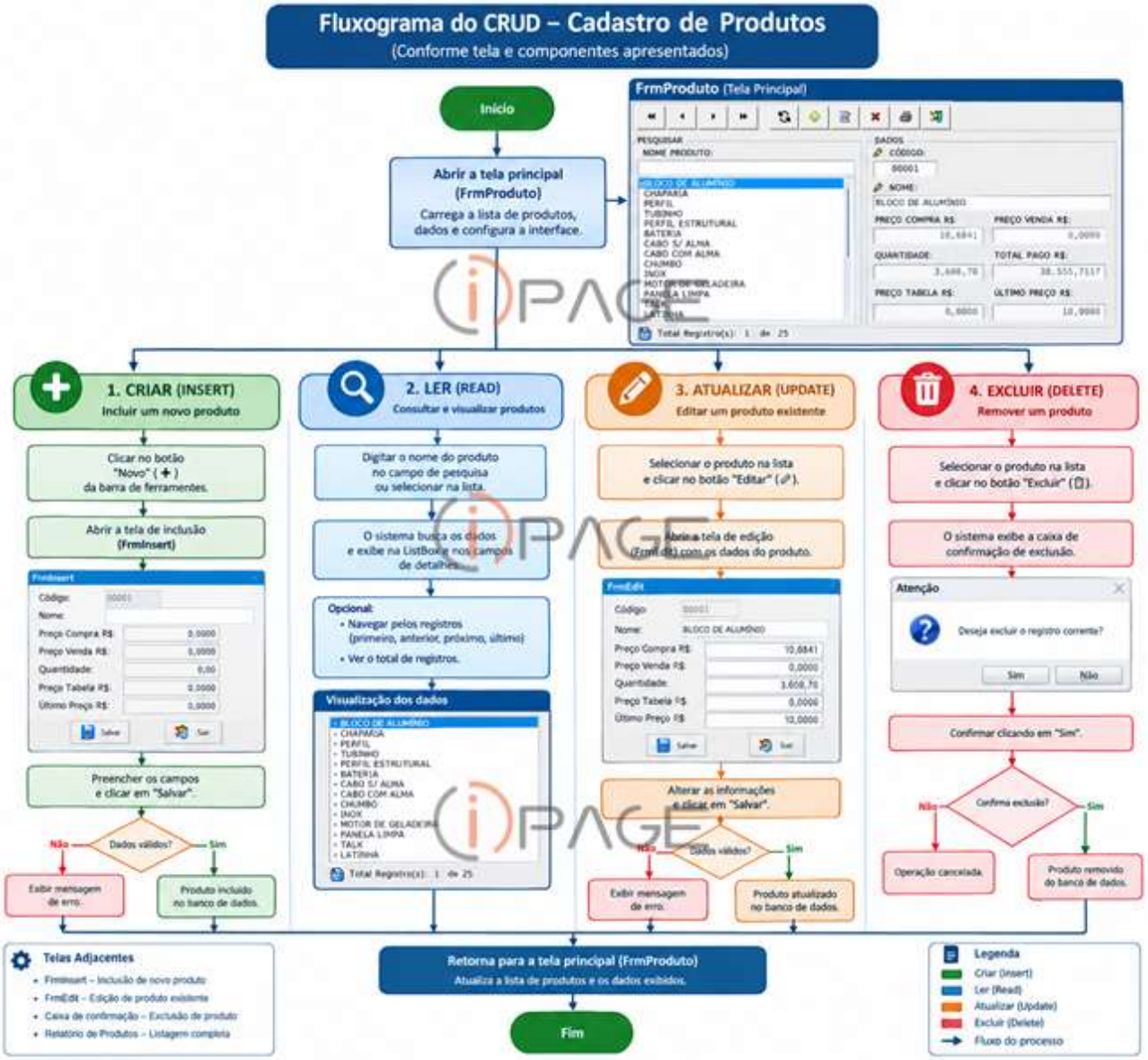


Figura 28

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O PONTO DE INÍCIO DO PROJETO

No VB 6.0, o ponto de partida de uma aplicação é o local onde o programa começa sua execução. É a partir dele que o VB 6.0 determina qual código será executado primeiro e qual formulário será apresentado ao usuário.

Na imagem abaixo, estamos visualizando as propriedades do projeto FlatFile, especificamente a opção Startup Object, que define esse comportamento.

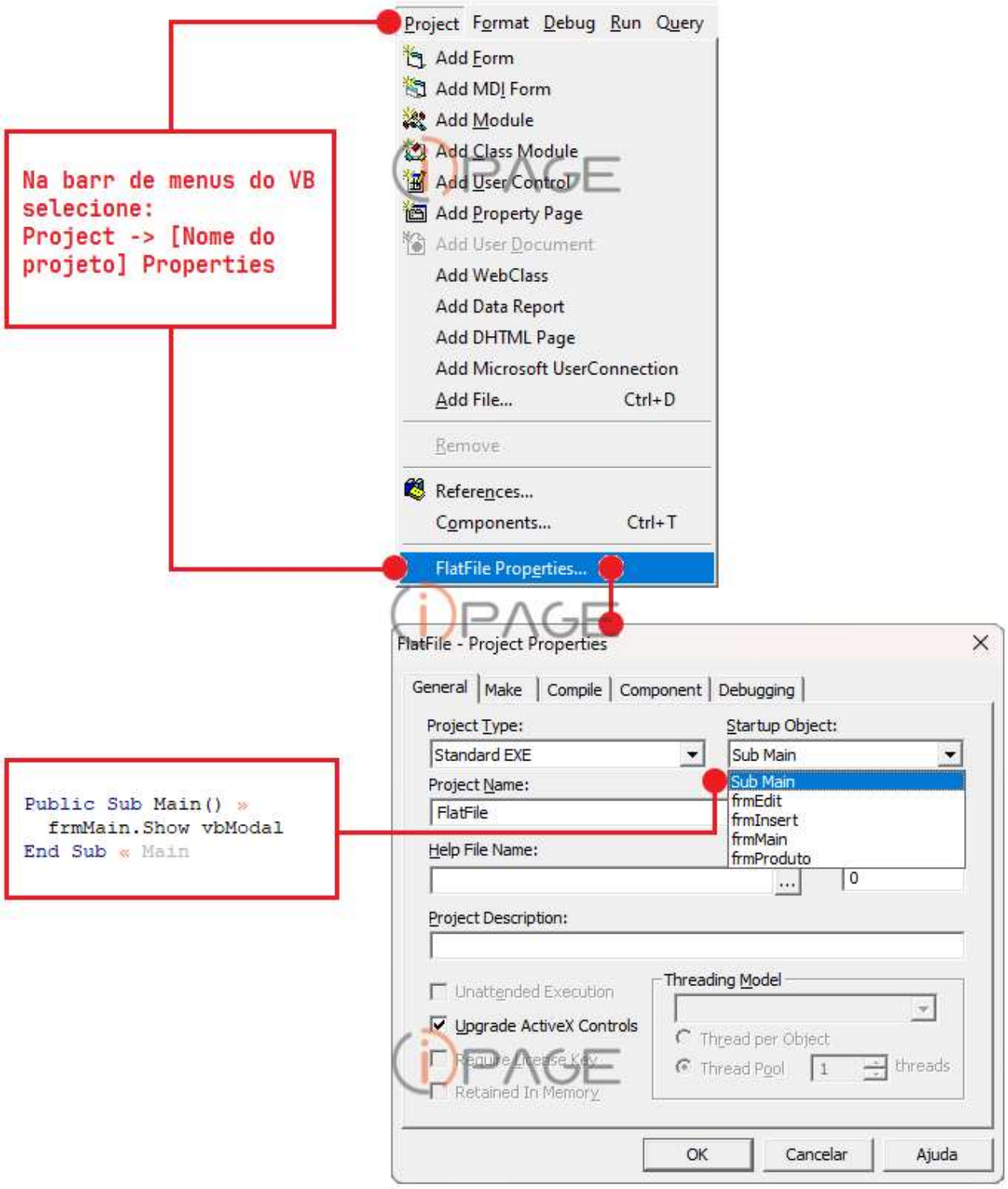


Figura 29

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

PARA ACESSAR ESSA CONFIGURAÇÃO NO VB 6.0:

1. Abra o projeto no Visual Basic 6.0.
2. No menu superior, clique em Project.
3. Selecione FlatFile Properties... (ou o nome do seu projeto).
4. Na aba General, localize a opção Startup Object.
5. Escolha o formulário inicial ou a opção Sub Main.

Sub Main deve ser obrigatório e não poderá ser repetido em nenhum outro módulo. O VB 6.0 entende automaticamente que esta sub deve ser executada como ponto de partida. Sub Main chama uma form SDI de forma Modal.

Caso não deseje que o ponto inicial seja uma sub-rotina, você poderá escolher qualquer um dos formulários deverá ser executado.

NO PROJETO ESTÁ DEFINIDO ASSIM:

1. Sub-Rotina Main
2. frmMain
3. Cadastro de Produtos

Essa abordagem é simples e adequada quando o formulário escolhido deve ser a primeira tela da aplicação.

INICIANDO DIRETAMENTE POR UM FORMULÁRIO

Imagine que você selecione frmProduto em Startup Object.

Quando o programa for executado, o VB 6.0 iniciará diretamente pelo formulário frmProduto, que será exibido ao usuário.

Por exemplo, você pode ter um formulário com uma lista de produtos, campos de pesquisa e botões para incluir, editar e excluir registros.

Nesse caso, a configuração seria:

```
Startup Object: frmProduto
```

Essa abordagem é simples e adequada quando o formulário escolhido deve ser a primeira tela da aplicação.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

INICIANDO POR MEIO DA SUB MAIN

Uma alternativa é selecionar Sub Main como ponto de partida.

Nesse caso, o VB 6.0 executa primeiro uma rotina chamada Main, normalmente localizada em um módulo padrão (.bas).

Exemplo:

Bas:

```
Public Sub Main()
    frmMain.Show vbModal
End Sub
```

Nesse exemplo, a execução começa em Main, que abre o formulário frmMain como uma janela modal.

A opção vbModal significa que o usuário precisa fechar esse formulário antes de interagir com outras janelas do mesmo aplicativo.

A diferença fundamental é que, usando Sub Main, você pode executar instruções antes de apresentar a primeira tela.

Por exemplo:

```
Public Sub Main()
    ' Inicialização da aplicação
    ' Verificação de arquivos
    ' Carregamento de configurações
    frmMain.Show
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O QUE ACONTECE EM CADA ALTERNATIVA?

PONTO DE PARTIDA	FUNÇÃO(FUNCTION)
frmMain	Inicia diretamente pelo formulário principal.
frmProduto	Abre diretamente o cadastro de produtos.
frmInsert	Abre diretamente a tela de inclusão.
frmEdit	Abre diretamente a tela de edição.
Sub Main	Executa primeiro o procedimento Main, que controla a inicialização.

Atenção!



iniciar diretamente por frmInsert ou frmEdit é possível, mas geralmente essas telas dependem de um contexto, como um produto selecionado ou um modo de operação previamente definido.

POR QUE UTILIZAR SUB MAIN EM UM PROJETO CRUD?

No seu projeto de cadastro de produtos com arquivos de tamanho fixo, eu recomendaria utilizar Sub Main como ponto de partida quando você quiser centralizar a inicialização da aplicação.

Isso permite organizar etapas como:

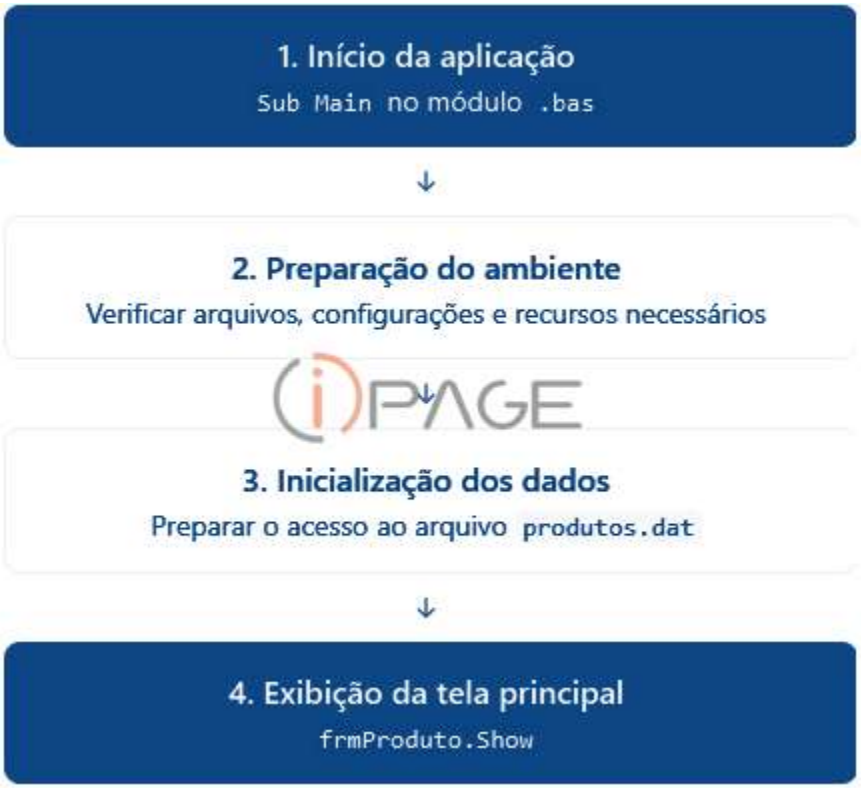


Figura 30

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Como o VB 6.0 se comporta depois da inicialização?

É importante distinguir o ponto de partida do restante da execução.

Quando você seleciona um formulário como objeto inicial, o VB 6.0 carrega esse formulário e executa seus eventos de ciclo de vida, como Form_Load.

Quando você seleciona Sub Main, o programa executa essa rotina primeiro. Ela pode abrir um formulário, preparar dados e decidir como a aplicação deve prosseguir.

Portanto, Sub Main funciona como um ponto central de inicialização, mas não substitui os eventos dos formulários nem o código responsável pelas operações do CRUD.

Disto isto, o projeto tem como ponto de partida do formulário frmMain.

ÁRVORE DO PROJETO

ESTRUTURA DA ÁRVORE DO PROJETO FLATFILE – VB 6.0

A árvore do projeto FlatFile apresenta a organização dos formulários, módulos padrão e módulos de classe que compõem a aplicação de cadastro de produtos. Essa estrutura permite separar as responsabilidades do sistema, facilitando a manutenção, a reutilização de código e a compreensão do funcionamento do CRUD.

No Visual Basic 6.0, os componentes são organizados principalmente em três grupos: Forms, Modules e Class Modules.

VISÃO GERAL DA ESTRUTURA

Com base na imagem, a estrutura do projeto é a seguinte:

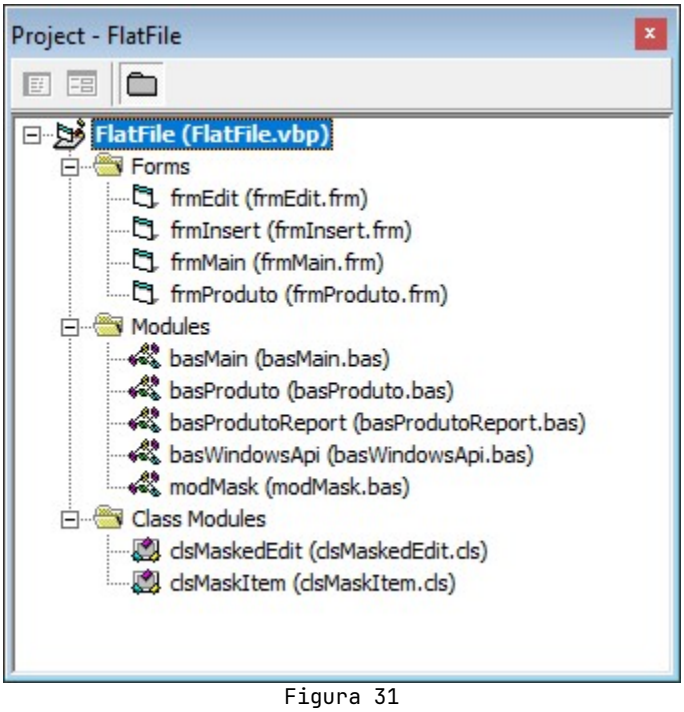


Figura 31

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O arquivo FlatFile.vbp é o arquivo de projeto do Visual Basic 6.0. Ele reúne as referências aos componentes utilizados e as configurações necessárias para abrir, compilar e executar a aplicação.

EVENTOS

Eventos são ações ou ocorrências reconhecidas por um objeto (como um formulário ou um botão) que disparam a execução de um bloco de código programado pelo desenvolvedor. Em termos simples: são os gatilhos que dizem ao sistema "o usuário fez isso, agora faça aquilo".

Aqui estão os principais tipos e conceitos de eventos no VB 6.0, de forma bem direta:

EVENTOS DE TECLADO (KEYBOARD EVENTS):

No Visual Basic 6.0, os eventos de teclado permitem que uma aplicação detecte quando o usuário pressiona, solta ou utiliza teclas enquanto interage com um formulário ou controle, como TextBox, CommandButton, ComboBox ou ListBox.

Esses eventos são muito úteis para criar atalhos, validar entradas, controlar a navegação entre campos e desenvolver interfaces mais profissionais para sistemas comerciais.

OS TRÊS PRINCIPAIS EVENTOS DE TECLADO DO VB 6.0 SÃO:

- **KeyDown** – quando uma tecla é pressionada.
- **KeyPress** – quando uma tecla que gera um caractere é pressionada.
- KeyUp – quando uma tecla é solta.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ENTENDENDO A SEQUÊNCIA DOS EVENTOS



Figura 32

Essa é a sequência típica para uma tecla que gera um caractere. Ela pode variar conforme a tecla, o controle e o processamento do sistema operacional. Por exemplo, as teclas de função e as setas normalmente não geram um evento **KeyPress**.

EVENTO KEYDOWN

O evento **KeyDown** é acionado quando uma tecla é pressionada. Ele permite identificar teclas especiais, teclas de função e combinações como Ctrl+S.

Exemplo: detector Tecla F1.

Form:

```
Private Sub Text1_KeyDown(KeyCode As Integer, Shift As Integer)
    If KeyCode = vbKeyF1 Then
        MsgBox "Você pressionou F1!"
    End If
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

A constante vbKeyF1 representa a tecla F1. O VB 6.0 também disponibiliza constantes como vbKeyEscape, vbKeyReturn, vbKeyDelete e vbKeyUp.
Exemplo: detectar Ctrl+S

Form:

```
Private Sub Text1_KeyDown(KeyCode As Integer, Shift As Integer)
    If KeyCode = vbKeyS And (Shift And vbCtrlMask) <> 0 Then
        MsgBox "Atalho para salvar!"
    End If
End Sub
```

O operador And, nesse contexto, permite verificar se o bit correspondente ao Ctrl está presente no parâmetro Shift.

As máscaras utilizadas são:

CONSTANTE	VALOR	TECLA
vbShiftMask	1	Shift
vbCtrlMask	2	Ctrl
vbAltMask	4	Alt

KeyDown e KeyUp são especialmente úteis para reconhecer teclas de navegação, teclas de função e modificadores.

EVENTO KEYPRESS

O evento KeyPress é indicado quando você deseja trabalhar com o caractere digitado, em vez de identificar apenas a tecla física.

Exemplo:

O parâmetro KeyAscii contém o código do caractere recebido pelo controle.
Exemplo: aceitar somente números

Form:

```
Private Sub Text1_KeyPress(KeyAscii As Integer)
    If KeyAscii < 48 Or KeyAscii > 57 Then
        If KeyAscii <> vbKeyBack Then
            KeyAscii = 0
        End If
    End If
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

No Script Acima:

- Os códigos de 48 a 57 correspondem aos dígitos de 0 a 9.
- **vbKeyBack** permite usar Backspace para apagar caracteres.
- **KeyAscii** = 0 cancela a entrada do caractere.

Esse código serve para demonstrar a filtragem de teclas, mas não valida sozinho números decimais, valores negativos ou dados colados da área de transferência. Para dados financeiros, também é necessário validar o conteúdo final.

Ver estudo sobre Tabela ASCII no link abaixo:
https://github.com/ipagesoftware/o-que-codigo-ascii

Exemplo: converter letras para maiúsculas

Form:

```
Private Sub Text1_KeyPress(KeyAscii As Integer)
    KeyAscii = Asc(UCase$(Chr$(KeyAscii)))
End Sub
```

Ao digitar letras, o controle recebe suas versões maiúsculas.
Esse método é apropriado para entradas comuns de texto, mas deve ser adaptado se o campo aceitar caracteres especiais ou entradas de outras formas.

Nota:



O KeyPress não é o evento indicado para detectar setas, F1 ou outras teclas que não geram caracteres.

EVENTO KEYUP

O evento KeyUp ocorre quando o usuário solta uma tecla que estava pressionada.

Exemplo: identificar quando F2 é liberada

Form:

```
Private Sub Text1_KeyUp(KeyCode As Integer, Shift As Integer)
    If KeyCode = vbKeyF2 Then
        Debug.Print "A tecla F2 foi liberada."
    End If
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Esse evento pode ser útil para detectar o término de uma interação com o teclado ou atualizar um estado visual depois que uma tecla é liberada.

Uma diferença importante: se você mantiver uma tecla pressionada, KeyDown poderá ocorrer repetidamente. Portanto, não presuma que cada chamada representa uma nova pressão independente.

PROPRIEDADE KEYPREVIEW DO FORMULÁRIO

Essa é uma das propriedades mais úteis para quem desenvolve aplicações comerciais em VB 6.0.

Por padrão, o controle que tem o foco recebe os eventos de teclado. Se você configurar:

Form:

```
Private Sub Form_Load()
    Me.KeyPreview = True
End Sub
```

O formulário receberá os eventos de teclado antes do controle que tem o foco. Assim, você pode implementar atalhos centralizados sem precisar repetir o mesmo código em cada TextBox.

Exemplo: usar F1 para abrir a ajuda

No formulário FrmPrincipal:

Form:

```
Private Sub Form_Load()
    Me.KeyPreview = True
End Sub

Private Sub Form_KeyDown(KeyCode As Integer, Shift As Integer)
    Select Case KeyCode
        Case vbKeyF1
            MsgBox "Sistema de ajuda"
        Case vbKeyF2
            MsgBox "Consultar registro"
        Case vbKeyF3
            MsgBox "Pesquisar"
    End Select
End Sub
```

Com KeyPreview = True, os atalhos podem ser tratados no formulário enquanto o foco estiver em controles que encaminham normalmente esses eventos.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Atenção!



KeyPreview não é um mecanismo global para capturar todas as teclas do Windows. Algumas teclas, combinações, botões padrão e comportamentos específicos dos controles podem exigir tratamento adicional.

DIFERENÇAS ENTRE OS TRÊS EVENTOS

CARACTERÍSTICAS	KEYDOWN	KEYPRESS	KEYUP
Quando ocorre	Pressiona a tecla	Recebe um caractere	Solta a tecla
Parâmetro principal	KeyCode	KeyAscii	KeyCode
Detecta F1 e setas	Sim	Não, normalmente	Sim
Filtra caracteres digitados	Não é o mais indicado	Sim	Não
Detecta Ctrl, Shift e Alt	Sim, via Shift	Não da mesma maneira	Sim, via Shift
Cancela a entrada alterando o parâmetro	Não pelo método usado em KeyPress	Sim, com KeyAscii = 0	Não pelo método usado em KeyPress

QUAL EVENTO DEVO UTILIZAR?

- Para detectar F1, F2, setas ou combinações como Ctrl+S: use KeyDown.
- Para permitir somente números, converter caracteres ou rejeitar uma tecla digitada: use KeyPress.
- Para executar uma ação quando o usuário soltar uma tecla: use KeyUp.
- Para centralizar atalhos em um formulário: configure KeyPreview = True e use Form_KeyDown.

UMA ÚLTIMA RECOMENDAÇÃO:

Os eventos de teclado ajudam a controlar a interface, mas não substituem a validação dos dados nem as regras de negócio. Em um sistema de vendas, por exemplo, bloquear letras em um campo de quantidade não garante que o usuário tenha informado uma quantidade válida.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Para aprofundar o assunto, consulte a documentação de referência do Visual Basic: Eventos KeyDown e KeyUp — Microsoft Learn.

CONSTANTE	VALOR	SIGNIFICADO
vbLeftButton	1	Botão esquerdo
vbRightButton	2	Botão direito
vbMiddleButton	4	Botão do meio

No MouseDown e no MouseUp, o parâmetro indica qual botão provocou o evento.
No MouseMove, ele informa o estado atual dos botões.

EVENTOS DE MOUSE (MOUSE EVENTS):

OS PRINCIPAIS EVENTOS DO MOUSE NO VB 6.0

No VB 6.0, os eventos mais importantes para controlar o mouse são MouseDown, MouseUp, MouseMove, Click e DblClick. Cada um representa uma etapa diferente da interação do usuário com um formulário ou controle

MouseDown — pressionou o botão
Ocorre quando o usuário pressiona um botão do mouse, mesmo antes de soltá-lo.
Exemplo: iniciar a seleção de um item.

MouseUp — soltou o botão
Ocorre quando o usuário solta o botão do mouse.
Exemplo: finalizar uma seleção ou operação de arraste.

MouseMove — movimentou o ponteiro
Ocorre quando o ponteiro se movimenta sobre o formulário ou controle.
Exemplo: desenhar, arrastar objetos ou atualizar coordenadas.

Click — realizou um clique
É o evento normalmente utilizado para executar uma ação após um clique.
Exemplo: abrir uma consulta ou confirmar uma operação.

DblClick — realizou um clique duplo
Ocorre quando o usuário realiza um duplo clique reconhecido pelo sistema.
Exemplo: abrir o cadastro de um produto selecionado.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ENTENDENDO OS PARÂMETROS DE MOUSEDOWN, MOUSEUP E MOUSEMOVE

Esses três eventos recebem os mesmos quatro parâmetros no VB 6.0:

Form:

```
Private Sub Form_MouseDown( _
    Button As Integer, _
    Shift As Integer, _
    X As Single, _
    Y As Single)

End Sub
```

EXEMPLO PRÁTICO: IDENTIFICAR O BOTÃO CLICADO

No formulário, adicione este código:

Form:

```
Private Sub Form_MouseDown( _
    Button As Integer, _
    Shift As Integer, _
    X As Single, _
    Y As Single)

    Select Case Button
        Case vbLeftButton
            Debug.Print "Botão esquerdo"

        Case vbRightButton
            Debug.Print "Botão direito"

        Case vbMiddleButton
            Debug.Print "Botão do meio"
    End Select

    Debug.Print "X: "; X
    Debug.Print "Y: "; Y

End Sub
```

Execute a aplicação e clique em diferentes áreas vazias do formulário. Abra a janela Immediate do editor VB 6.0 com Ctrl+G para visualizar os resultados.

Observação: o evento do formulário ocorre quando o clique acontece em uma área que pertence ao próprio formulário. Se você clicar sobre um TextBox, por exemplo, o evento correspondente normalmente será o do controle.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Exemplo: detectar o botão direito

Você pode usar o botão direito para exibir uma mensagem ou iniciar uma operação contextual.

Form:

```
Private Sub Form_MouseDown(Button As Integer, Shift As Integer, X As Single, Y As Single)
    If Button = vbRightButton Then
        MsgBox "Você clicou com o botão direito."
    End If
End Sub
```

Em aplicações comerciais, esse mecanismo pode ser adaptado para abrir menus de contexto com opções como editar, excluir ou consultar um registro.

EVENTO	QUANDO USAR?
Initialize	Inicializar o estado da instância do formulário.
Activate	Atualizar informações quando a janela se torna ativa.
Deactivate	Reagir quando outra janela se torna ativa.
Resize	Reposicionar ou redimensionar controles.
Paint	Executar desenho personalizado quando a área precisa ser redesenhada.
Click	Tratar cliques na área do formulário.
DblClick	Tratar duplo clique na área do formulário.
MouseDown	Detectar o pressionamento de um botão do mouse.
MouseMove	Detectar movimento do ponteiro.
MouseUp	Detectar a liberação do botão do mouse.
KeyDown	Detectar teclas e atalhos, respeitando o foco e KeyPreview.
KeyPress	Tratar caracteres digitados.
KeyUp	Detectar a liberação de teclas.
GotFocus	Detectar ganho de foco, quando aplicável.
LostFocus	Detectar perda de foco, quando aplicável.
Terminate	Identificar a destruição efetiva da instância.

MINHA RECOMENDAÇÃO PARA SEUS PROJETOS EM VB 6.0

Para manter o código organizado, recomendo esta divisão de responsabilidades:

- Initialize: inicialização do objeto.
- Load: configuração inicial dos controles e da interface.
- Activate: atualização de dados que precisam ser renovados ao retornar à tela.
- Resize: adaptação do layout.
- QueryUnload: validação da saída e confirmação de alterações pendentes.
- Unload: limpeza final antes de descarregar a tela.
- Terminate: ações relacionadas à destruição do objeto, sem depender dele continuar disponível.

Em um sistema de depósito de bebidas, por exemplo, a abertura do cadastro de produtos pode preparar os campos no Load, atualizar informações no Activate e verificar alterações não salvas no QueryUnload.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

O princípio fundamental é: cada evento deve ter uma responsabilidade clara. Isso facilita a manutenção, reduz a duplicação de código e torna a aplicação mais previsível.

EVENTOS DE CICLO DE VIDA E JANELA (FORM & OBJECT EVENTS):

No Visual Basic 6.0, um formulário (Form) é a janela que serve de base para a interface de uma aplicação. Ele pode conter caixas de texto, botões, listas, menus, imagens e outros controles.

Os eventos de formulário são procedimentos executados automaticamente quando alguma ação acontece durante o ciclo de vida da janela, como sua abertura, carregamento, ativação, redimensionamento ou fechamento.

Por exemplo, em um sistema comercial para depósito de bebidas, podemos utilizar eventos do formulário para:

- Inicializar os dados quando a tela for carregada.
- Configurar o tamanho e a aparência da janela.
- Posicionar o cursor no primeiro campo.
- Detectar teclas de atalho.
- Atualizar informações quando a janela for redimensionada.
- Confirmar o fechamento de uma tela.
- Liberar recursos antes de encerrar a aplicação.

OS PRINCIPAIS EVENTOS DE UM FORMULÁRIO

Initialize – inicialização do objeto
Ocorre quando a instância do formulário é criada. É apropriado para inicializar variáveis e estados internos do objeto.
Exemplo: definir valores iniciais de variáveis privadas.

Load – carregamento do formulário
Ocorre quando o formulário é carregado na memória. É um dos eventos mais utilizados para preparar a tela antes de sua exibição.
Exemplo: carregar configurações e preparar campos.

Activate – ativação
Ocorre quando o formulário se torna a janela ativa.
Exemplo: atualizar informações quando o usuário retorna à tela.

Resize – redimensionamento
Ocorre quando as dimensões da janela mudam.
Exemplo: ajustar a largura de uma grade ou reposicionar controles.

QueryUnload – consulta antes de descarregar
Permite verificar se o formulário pode ser fechado e, se necessário, cancelar o fechamento.
Exemplo: perguntar se o usuário deseja descartar alterações.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Unload – descarregamento
Ocorre quando o formulário está sendo removido da memória. É adequado para executar ações finais de limpeza.
Exemplo: liberar recursos e finalizar operações associadas à tela.

Esses eventos não são os únicos disponíveis. Um formulário também possui eventos como Deactivate, Paint, GotFocus, LostFocus, KeyDown, KeyPress, KeyUp e os eventos do mouse.

ENTENDENDO O CICLO DE VIDA DE UM FORMULÁRIO

Um dos conceitos mais importantes no VB 6.0 é compreender a ordem em que os eventos acontecem.



Figura 33

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Essa é uma sequência simplificada. Activate pode ocorrer várias vezes, e Deactivate acontece quando o formulário perde a ativação. Se um formulário já estiver carregado, exibi-lo novamente não significa que Initialize e Load serão executados de novo.

Além disso, Unload não significa necessariamente que o objeto foi destruído definitivamente: isso depende de ainda existirem referências à instância. Terminate ocorre quando o objeto é efetivamente destruído.

EVENTO FORM_LOAD

É normalmente utilizado para preparar o formulário antes de sua primeira exibição. Exemplo de uma tela de cadastro de produtos:

Form:

```
Private Sub Form_Load()
    Me.Caption = "Cadastro de Produtos"
    txtCodigo.Text = ""
    txtDescricao.Text = ""
    txtPreco.Text = "0,00"
    txtCodigo.SetFocus
End Sub
```

Nesse exemplo, configuramos o título, limpamos os campos e posicionamos o cursor no código do produto.

Atenção!



SetFocus pode gerar erro se o controle estiver invisível, desabilitado ou ainda não puder receber foco. Em telas mais complexas, verifique essas condições.

EVENTO FORM_QUERYUNLOAD

Esse evento é especialmente importante em sistemas comerciais porque permite impedir o fechamento quando existem alterações não salvas.

Form:

```
Private Sub Form_QueryUnload(Cancel As Integer, UnloadMode As Integer)
    If MsgBox( _
        "Deseja realmente fechar esta tela?", vbQuestion + vbYesNo, _
        "Confirmação") = vbNo Then
        Cancel = 1
    End If
End Sub
```


© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Quando o usuário tenta fechar a tela, o VB 6.0 pergunta se ele deseja continuar.

- Se responder Sim, o fechamento continua.
- Se responder Não, Cancel = 1 cancela o fechamento.

O parâmetro UnloadMode informa como o fechamento foi solicitado, por exemplo, pelo botão X, pelo código ou pelo encerramento do Windows. Você pode utilizá-lo para adaptar o comportamento.

EVENTO FORM_UNLOAD

O evento Unload é utilizado durante o descarregamento do formulário.

Form:

```
Private Sub Form_Unload(Cancel As Integer)

    Debug.Print "Formulário sendo descarregado."

End Sub
```

Você pode utilizá-lo para salvar preferências da interface ou executar procedimentos de limpeza.

Entretanto, tenha cuidado ao salvar dados importantes nesse evento: operações críticas devem ser confirmadas e tratadas antes de chegar ao fechamento, para que falhas não causem perda de informações.

O Unload também possui o parâmetro Cancel, que pode cancelar o descarregamento. Para perguntar ao usuário se deve fechar, normalmente é mais organizado usar QueryUnload.

OUTROS EVENTOS IMPORTANTES DO FORMULÁRIO

FORMS — FORMULÁRIOS DA APLICAÇÃO

Os formulários são responsáveis pela interface gráfica e pela interação com o usuário.

frmEdit.frm — Edição de produtos

Responsável por apresentar os dados de um produto já cadastrado e permitir a alteração de suas informações, como nome, preço de compra, preço de venda e quantidade.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

DESCRIÇÃO DOS CONTROLES - frmEdit

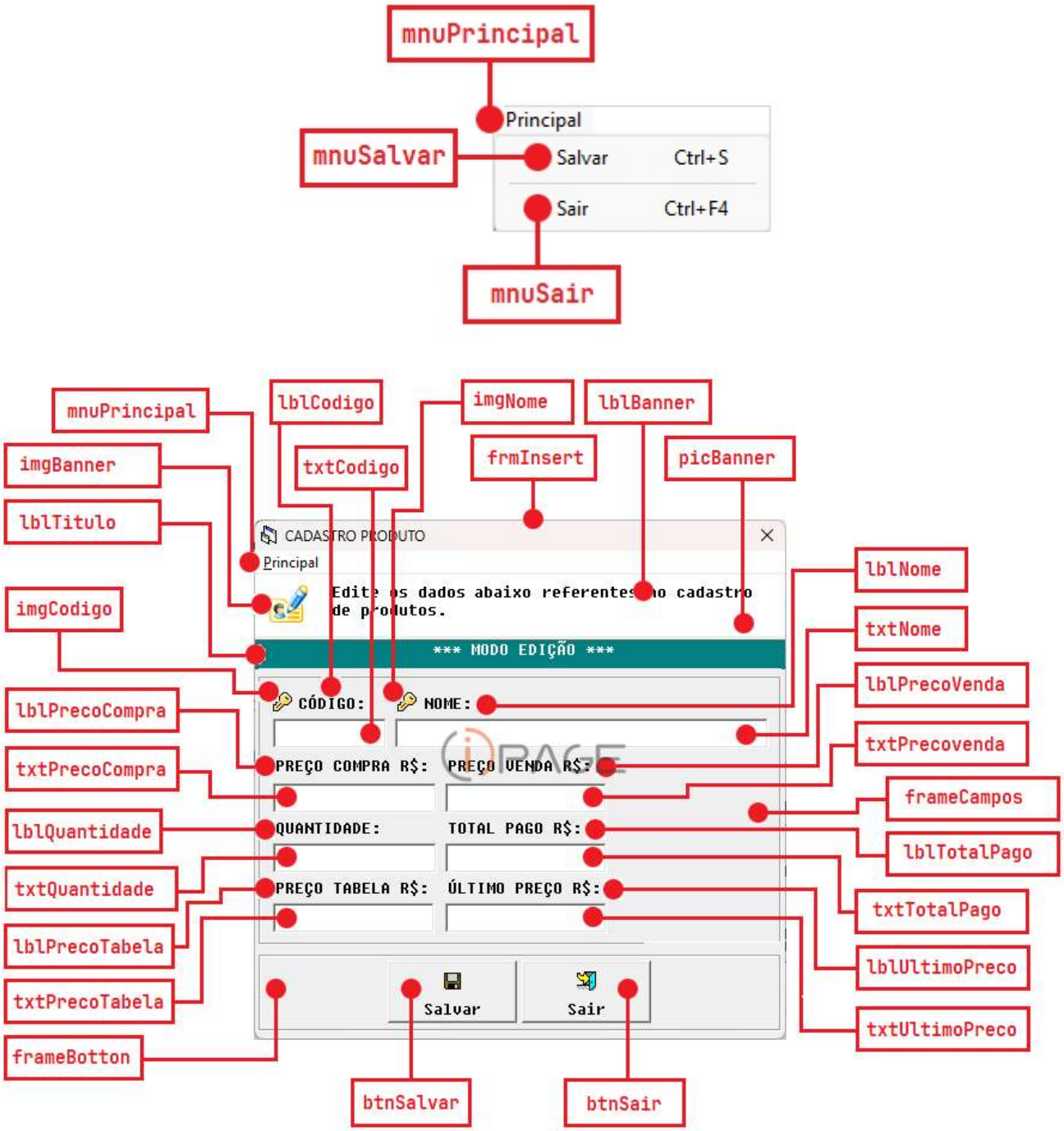


Figura 34

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

INCLUSÃO DE PRODUTOS – frmInsert.frm

Responsável pelo cadastramento de novos produtos. Recebe os dados informados pelo usuário e disponibiliza a operação para salvar o novo registro.

DESCRIÇÃO DOS CONTROLES - frmInsert

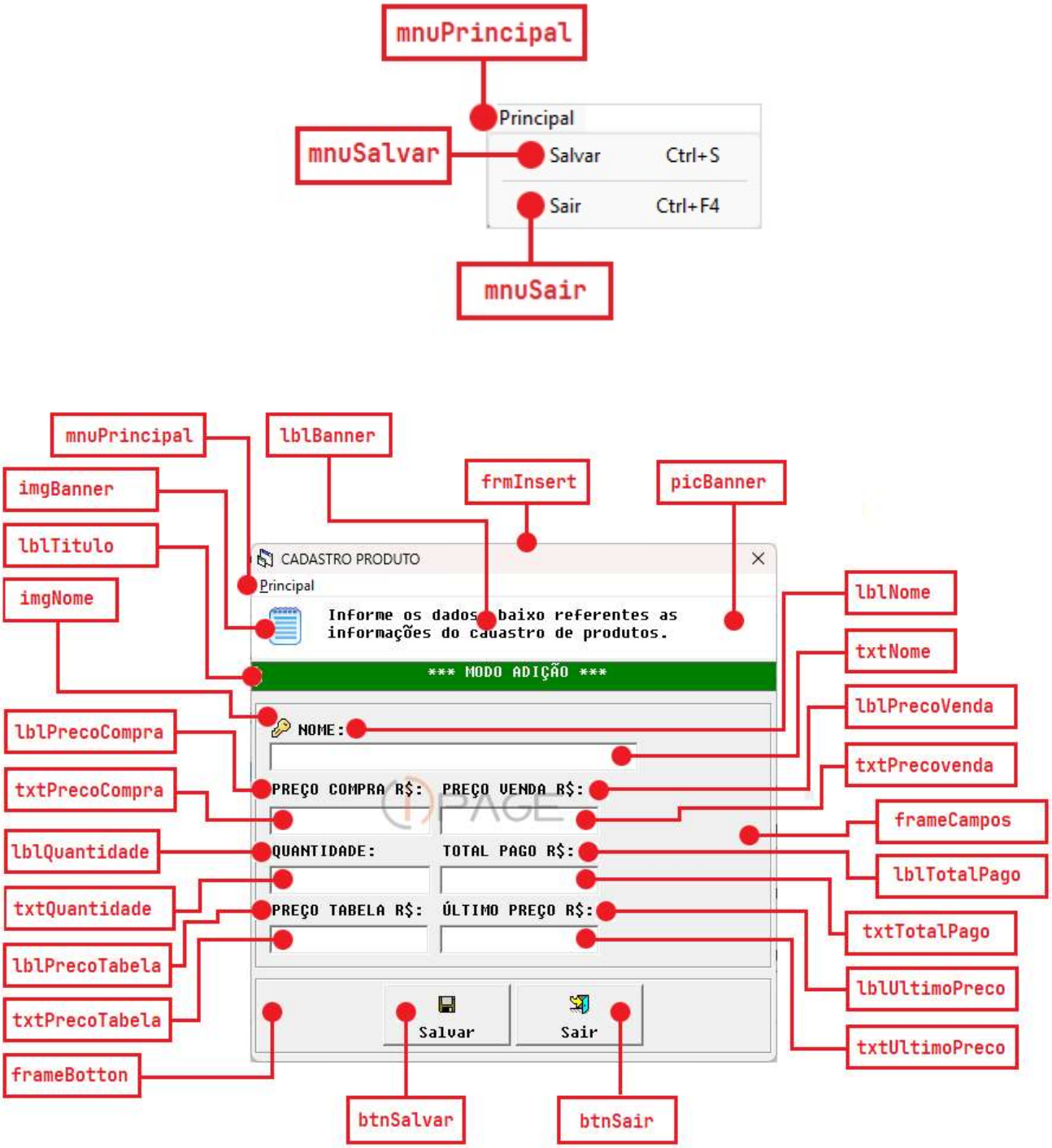


Figura 35

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FORMULÁRIO INICIAL – frmMain.frm

É o formulário definido como tela inicial da aplicação, conforme a organização indicada na imagem. Atua como ponto de entrada visual e pode direcionar o usuário às funcionalidades do sistema.

DESCRIÇÃO DOS CONTROLES – frmMain.frm

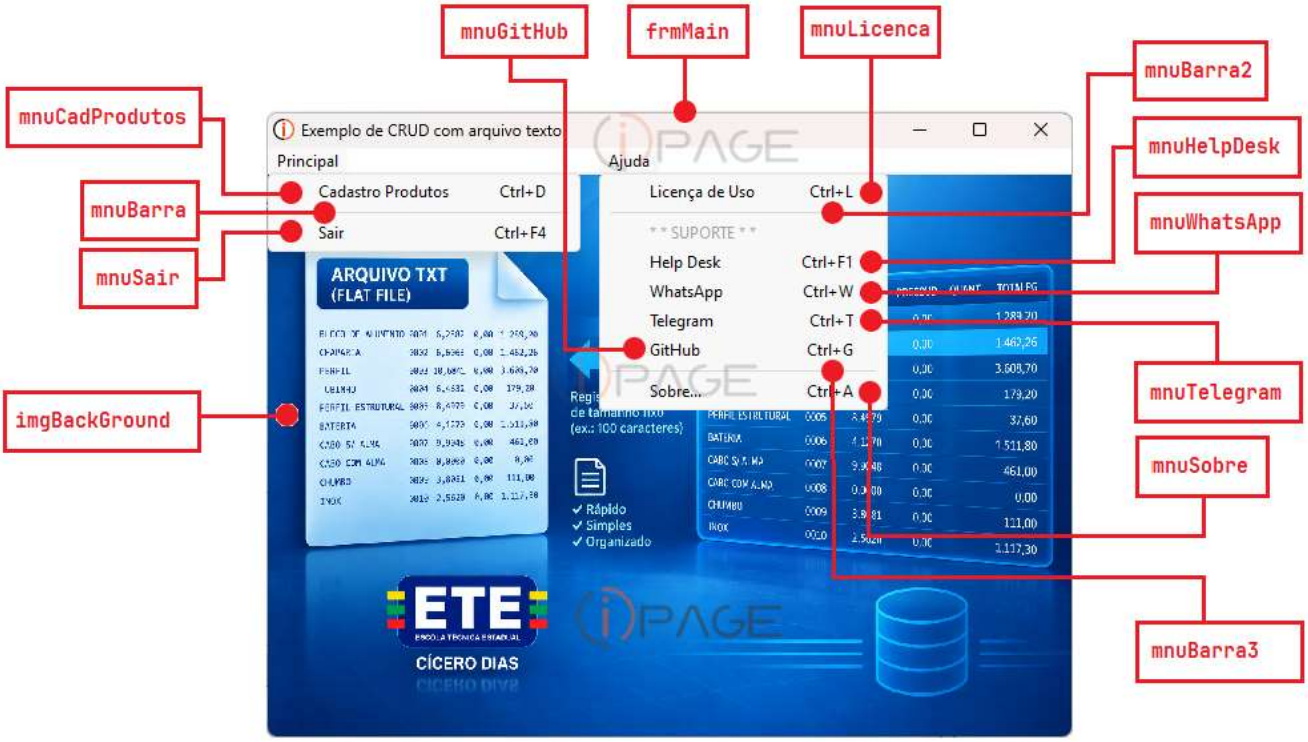


Figura 36

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

GERENCIAMENTO DE PRODUTOS – frmProduto.frm

É a tela principal do módulo de produtos. Reúne a pesquisa, a listagem, a navegação entre registros e os comandos para incluir, editar, excluir e emitir o relatório do cadastro.

REPRESENTAÇÃO VISUAL

MENUS



Figura 37

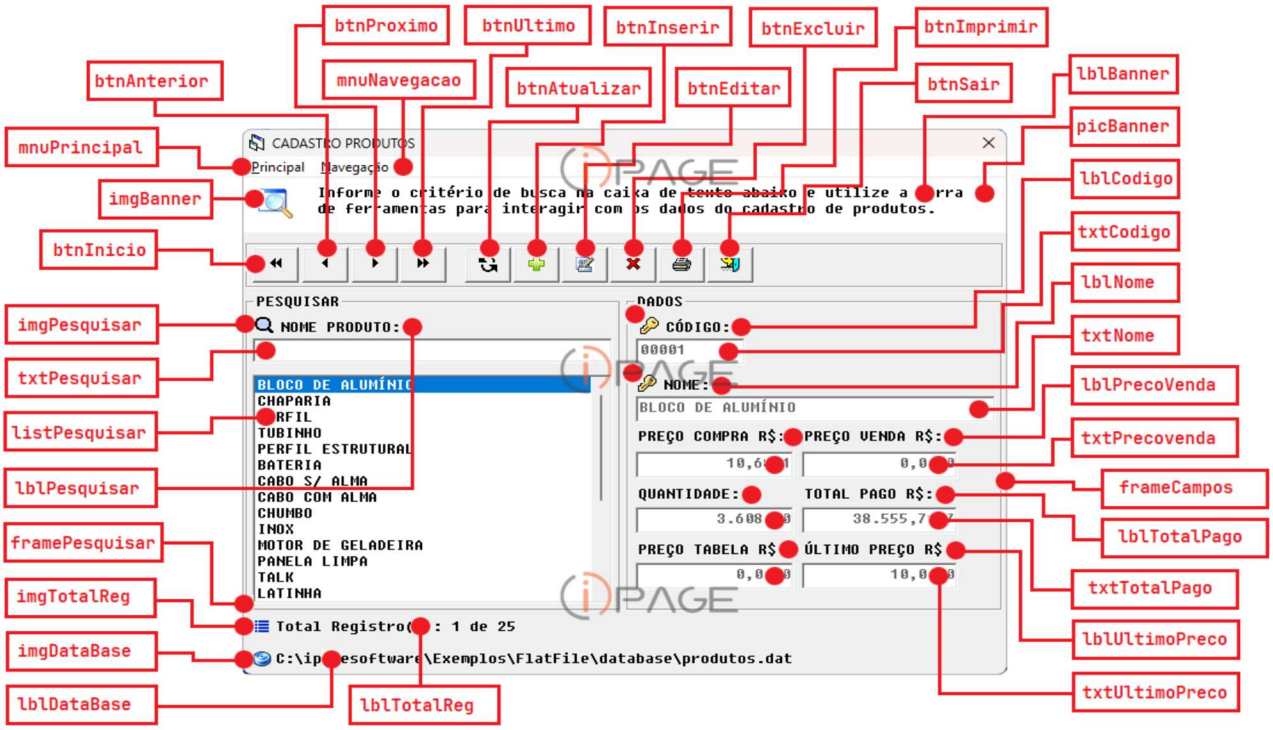


Figura 38

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

MODULES — MÓDULOS PADRÃO

Os módulos padrão (.bas) agrupam procedimentos e funções que podem ser utilizados por diferentes partes da aplicação, sem precisar estar vinculados a um formulário específico.

MÓDULO	RESPONSABILIDADE
basMain.bas	Módulo principal da aplicação. Pode conter o procedimento Sub Main, responsável por executar as rotinas de inicialização e abrir o formulário inicial.
basProduto.bas	Reúne as rotinas relacionadas à gestão dos produtos, como operações de leitura, gravação, alteração e exclusão nos arquivos de dados.
basProdutoReport.bas	Concentra as rotinas relacionadas à geração e emissão do relatório de produtos.
basWindowsApi.bas	Agrupa declarações e funções que permitem acessar recursos de baixo nível do Windows por meio da API do sistema operacional.

A separação das rotinas em módulos torna o código mais organizado e permite que os formulários utilizem funções compartilhadas.

Por exemplo, o formulário frmProduto pode chamar procedimentos de basProduto.bas para carregar a lista de produtos, enquanto basProdutoReport.bas pode ser utilizado para produzir o relatório.

CLASS MODULES — MÓDULOS DE CLASSE

dsMaskEdit.cls

Esse módulo de classe é responsável pelo tratamento de máscaras de entrada em caixas de texto, conforme indicado na imagem.

As máscaras permitem orientar ou restringir a digitação de valores em determinados formatos. Dependendo de sua implementação, a classe pode ser utilizada para campos numéricos, valores monetários, datas ou outros dados que exijam uma apresentação padronizada.

Diferentemente de um módulo padrão, uma classe permite encapsular comportamentos e manter uma organização orientada a objetos.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

COMO OS COMPONENTES SE RELACIONAM?

A estrutura apresentada separa a interface gráfica das rotinas de processamento e dos recursos auxiliares.

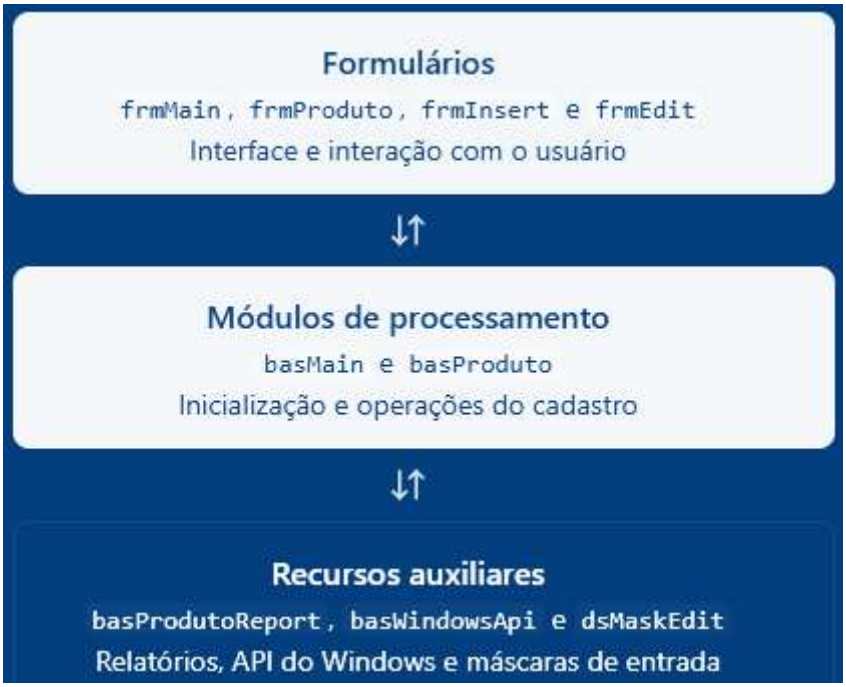


Figura 39

No caso do projeto FlatFile, o armazenamento utiliza arquivos de dados de tamanho fixo, conforme a proposta da aplicação. As rotinas de manipulação desses arquivos devem ficar organizadas na camada responsável pelos dados, evitando concentrar todo o processamento nos eventos dos formulários.

CONCLUSÃO

A árvore do projeto FlatFile demonstra uma organização modular típica de aplicações desenvolvidas em Visual Basic 6.0. Os formulários concentram a interação com o usuário, os módulos padrão agrupam as rotinas de processamento e os módulos de classe encapsulam comportamentos específicos.

Essa divisão facilita a compreensão do CRUD de produtos e ajuda a identificar onde cada funcionalidade deve ser implementada ou modificada. Também constitui um bom exemplo didático de organização de projetos, separação de responsabilidades e reutilização de código em aplicações desktop.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

VISÃO GERAL DO frmMain E SEUS MENUS

MENU PRINCIPAL

O menu principal pertence ao formulário FrmMain ("Exemplo de CRUD com arquivo texto") e está estruturado da seguinte forma:

- Menu Superior Principal: Na barra superior da janela FrmMain, encontram-se as opções de menu Principal e Ajuda.
- Submenu "Principal": Ao clicar na opção Principal, abre-se uma lista suspensa contendo dois itens principais:
    - Cadastro Produtos: Opção com o atalho de teclado Ctrl+D, que serve para abrir a janela de gerenciamento e listagem de produtos (frmProduto).
    - Sair: Opção com o atalho Ctrl+F4, responsável por encerrar a aplicação, exibindo uma caixa de diálogo de confirmação ("Deseja sair desta aplicação?") com os botões Sim e Não.

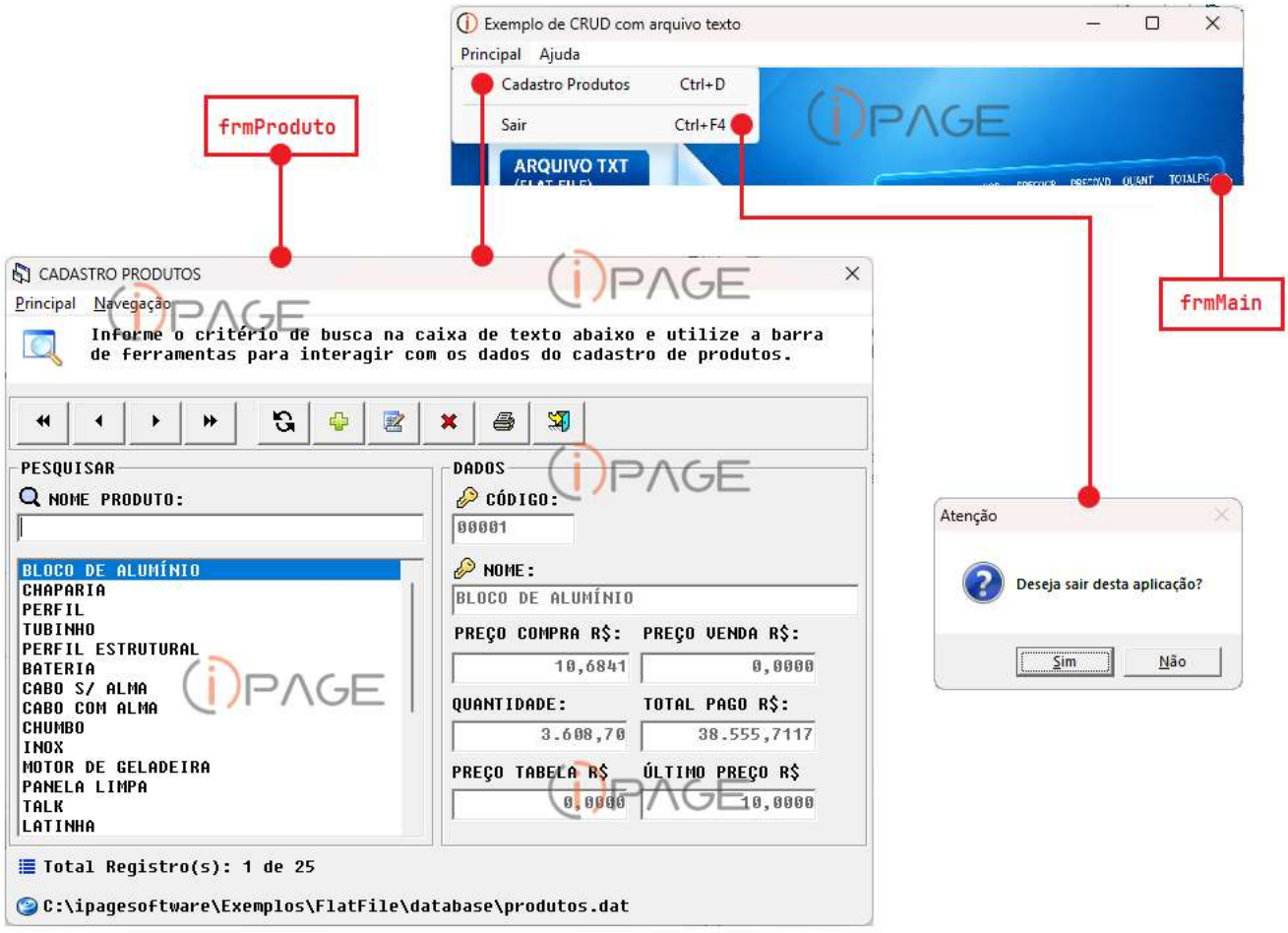


Figura 40

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

MENU AJUDA

O menu Ajuda do formulário FrmMain abre um submenu suspenso com opções de suporte e documentação interligadas a canais externos e informativos. Abaixo estão descritos cada um dos itens apresentados:

- Licença de Uso (Atalho Ctrl+L): Abre o contrato ou documento PDF de licença de uso do software da IPAGE Software.
- Help Desk (Atalho Ctrl+F1): Redireciona para o painel web de suporte técnico, permitindo criar novas demandas ou visualizar chamados existentes.
- WhatsApp (Atalho Ctrl+W): Abre o canal de atendimento direto via WhatsApp Web para comunicação rápida com a equipe de suporte.
- Telegram (Atalho Ctrl+T): Abre a página ou grupo oficial da IPAGE no Telegram para avisos e interação com membros.
- Sobre... (Atalho Ctrl+A): Exibe uma caixa de diálogo informativa detalhando os dados do sistema, como a versão do IPAGE SOFTWARE (Ver. 1.0.0 FlatFile), informações de licenciamento para o usuário e dados do ambiente Windows.

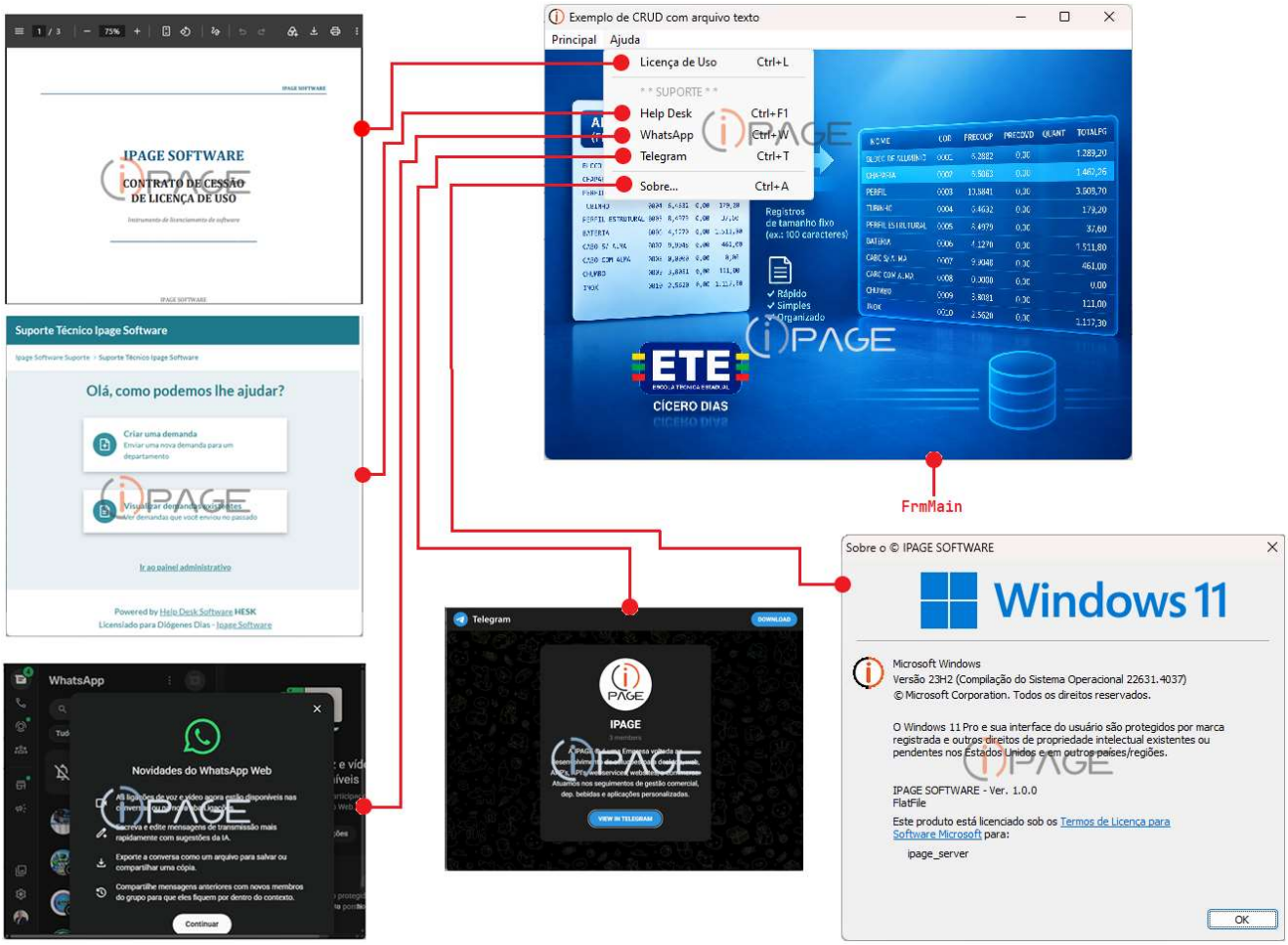


Figura 41

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

CADASTRO DE PRODUTOS

DESCRIÇÃO FUNCIONAL DOS COMPONENTES DE UM CRUD DE PRODUTOS.

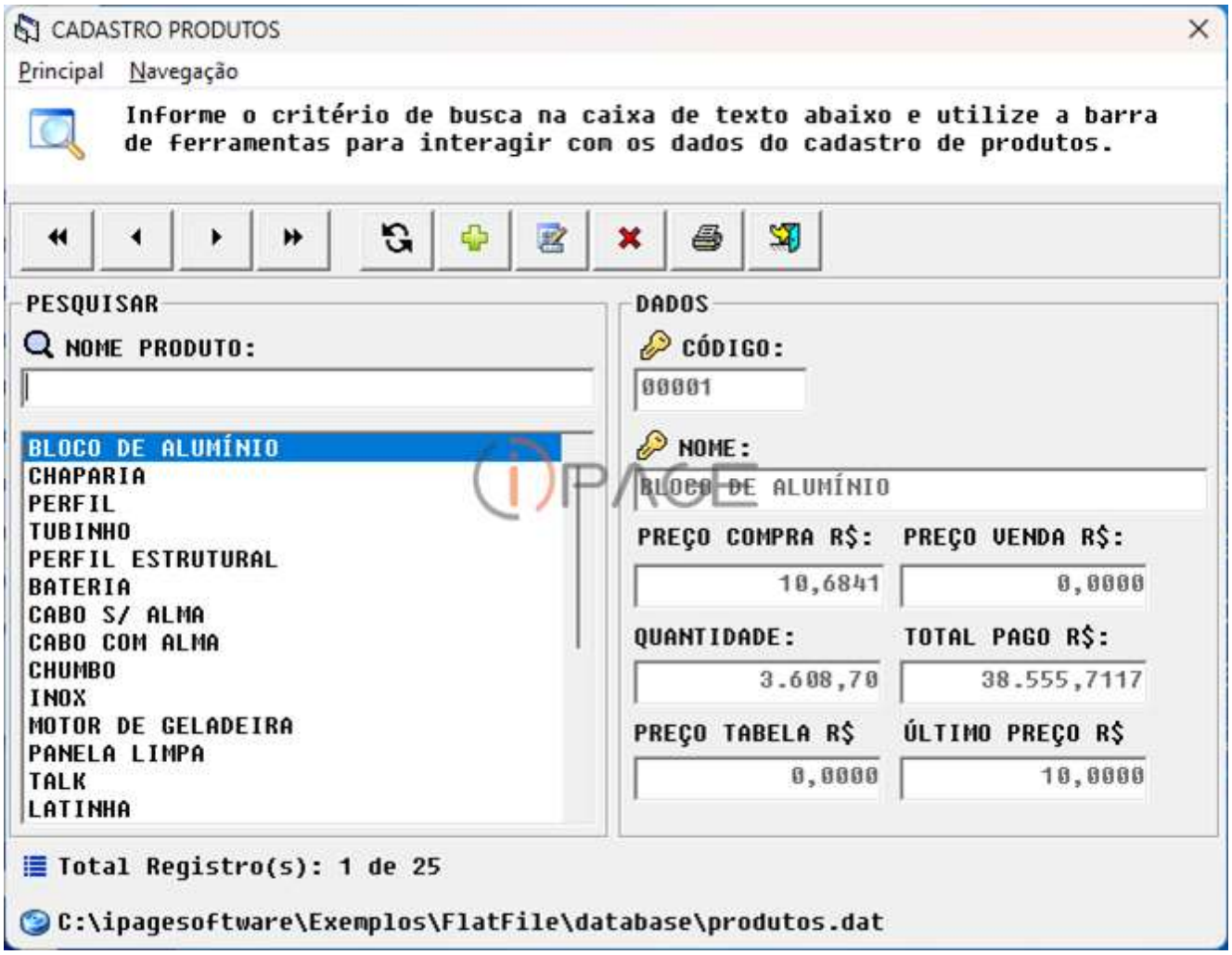


Figura 42

OBJETIVO DA TELA

A tela apresentada é um CRUD (Create, Read, Update e Delete) destinado ao cadastro e à manutenção de produtos. Ela permite localizar registros existentes, navegar entre os registros, incluir novos produtos, alterar dados, excluir registros, imprimir informações e encerrar a operação.

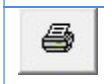
© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

IDENTIFICAÇÃO GERAL DOS COMPONENTES

Nº	COMPONENTE	IDENTIFICAÇÃO	DESCRIÇÃO
1	Formulário principal	Janela CADASTRO PRODUTOS	Contêiner principal de toda a aplicação. Organiza menus, barra de ferramentas, área de pesquisa, dados do produto e informações de status.
2	Menu Principal	Principal	Menu superior destinado às operações gerais do cadastro e às funções principais do módulo.
3	Menu Navegação	Navegação	Menu relacionado à navegação e localização dos registros do cadastro.
4	Área de orientação	Ícone de pesquisa + texto explicativo	Informa ao usuário que a caixa de texto deve ser utilizada para pesquisar e que a barra de ferramentas permite interagir com os dados.
5	Barra de ferramentas	Toolbar horizontal	Concentra as operações de navegação, atualização, inclusão, alteração, exclusão, impressão e saída.
6	Grupo de pesquisa	PESQUISAR	Área destinada à localização de produtos.
7	Campo de pesquisa	NOME PRODUTO	Caixa de texto utilizada para digitar o nome ou parte do nome do produto a ser localizado.
8	Lista de produtos	ListBox	Exibe os produtos encontrados. O usuário pode selecionar um item para visualizar seus dados no painel à direita.
9	Contador de registros	Total Registro(s): 1 de 25	Indica a quantidade de registros disponíveis e a posição do registro atualmente selecionado.
10	Grupo de dados	DADOS	Área que apresenta os campos do produto selecionado e permite sua manutenção.
11	Código	Campo numérico	Identificador do produto. Normalmente corresponde à chave primária do registro.
12	Nome	Campo de texto	Descrição/nome comercial do produto.
13	Preço de compra	Campo monetário	Valor de aquisição ou custo unitário do produto.
14	Preço de venda	Campo monetário	Valor utilizado para venda do produto.
15	Quantidade	Campo numérico	Quantidade de unidades ou estoque associado ao produto.
16	Total pago	Campo monetário	Valor total pago, normalmente relacionado à quantidade adquirida multiplicada pelo preço de compra ou ao valor registrado na operação.
17	Preço tabela	Campo monetário	Preço de referência/tabela do produto.
18	Último preço	Campo monetário	Último preço registrado para o produto.
19	Barra de status	Caminho do arquivo	Exibe o local da base de dados utilizada pela aplicação. Na imagem, é mostrado o arquivo produtos.dat.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

BARRA DE FERRAMENTAS – FUNÇÕES

BOTÃO	OPERAÇÃO	FUNÇÃO
	Primeiro registro	Move o cursor para o primeiro produto do cadastro.
	Registro anterior	Volta um registro na sequência atual.
	Próximo registro	Avança um registro na sequência atual.
	Último registro	Move o cursor para o último produto do cadastro.
	Atualizar	Atualiza os dados exibidos, relendo ou sincronizando os registros da fonte de dados.
	Incluir	Inicia o cadastramento de um novo produto, normalmente limpando ou preparando os campos para edição.
	Editar	Coloca o produto atual em modo de edição para permitir alterações.
	Excluir	Remove o produto selecionado, normalmente após uma confirmação do usuário.
	Imprimir	Executa ou prepara a impressão dos dados/relatório relacionado ao cadastro.
	Sair	Encerra a tela atual ou retorna ao módulo anterior.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FLUXO DE UTILIZAÇÃO DO CRUD

1. **Pesquisar:** O usuário informa o nome do produto no campo NOME PRODUTO.
2. **Localizar:** A lista apresenta os produtos compatíveis com o critério informado.
3. **Selecionar:** O usuário seleciona um produto na lista.
4. **Visualizar:** Os campos da área DADOS são preenchidos com as informações do registro selecionado.
5. **Navegar:** Os botões Primeiro, Anterior, Próximo e Último permitem percorrer os registros.
6. **Incluir:** O botão Novo inicia o cadastro de um produto.
7. **Alterar:** O botão Alterar permite modificar o produto selecionado.
8. **Excluir:** O botão Excluir remove o registro, preferencialmente com confirmação.
9. **Atualizar:** O botão Atualizar/recarregar sincroniza a tela com a fonte de dados.
10. **Imprimir:** O botão Imprimir gera a saída impressa ou relatório correspondente.

ESTRUTURA LÓGICA SUGERIDA PARA O CRUD

A tela pode ser organizada conceitualmente em quatro grupos de operações:

CREATE — Criar — Novo cadastro de produto.

READ — Ler — Pesquisa, listagem, seleção e visualização dos produtos.

UPDATE — Atualizar — Alteração dos dados do produto selecionado.

DELETE — Excluir — Exclusão do produto selecionado.

OBSERVAÇÕES SOBRE A IMPLEMENTAÇÃO

A imagem indica que a aplicação utiliza uma base de dados em arquivo, pois a barra de status apresenta o caminho: C:\ipagesoftware\Exemplos\FlatFile\database\produtos.dat.

A lista de produtos funciona como mecanismo de seleção dos registros, enquanto o painel DADOS apresenta os campos do registro selecionado.

Em uma implementação VB 6.0, o ItemData da ListBox pode ser utilizado para armazenar o código/ID do produto correspondente a cada item da lista. Dessa forma, o texto exibido pode ser o nome do produto, enquanto ItemData mantém a identificação numérica necessária para localizar, alterar ou excluir o registro correto.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

VISÃO GERAL DA ARQUITETURA DE JANELAS DO CADASTRO DE PRODUTOS

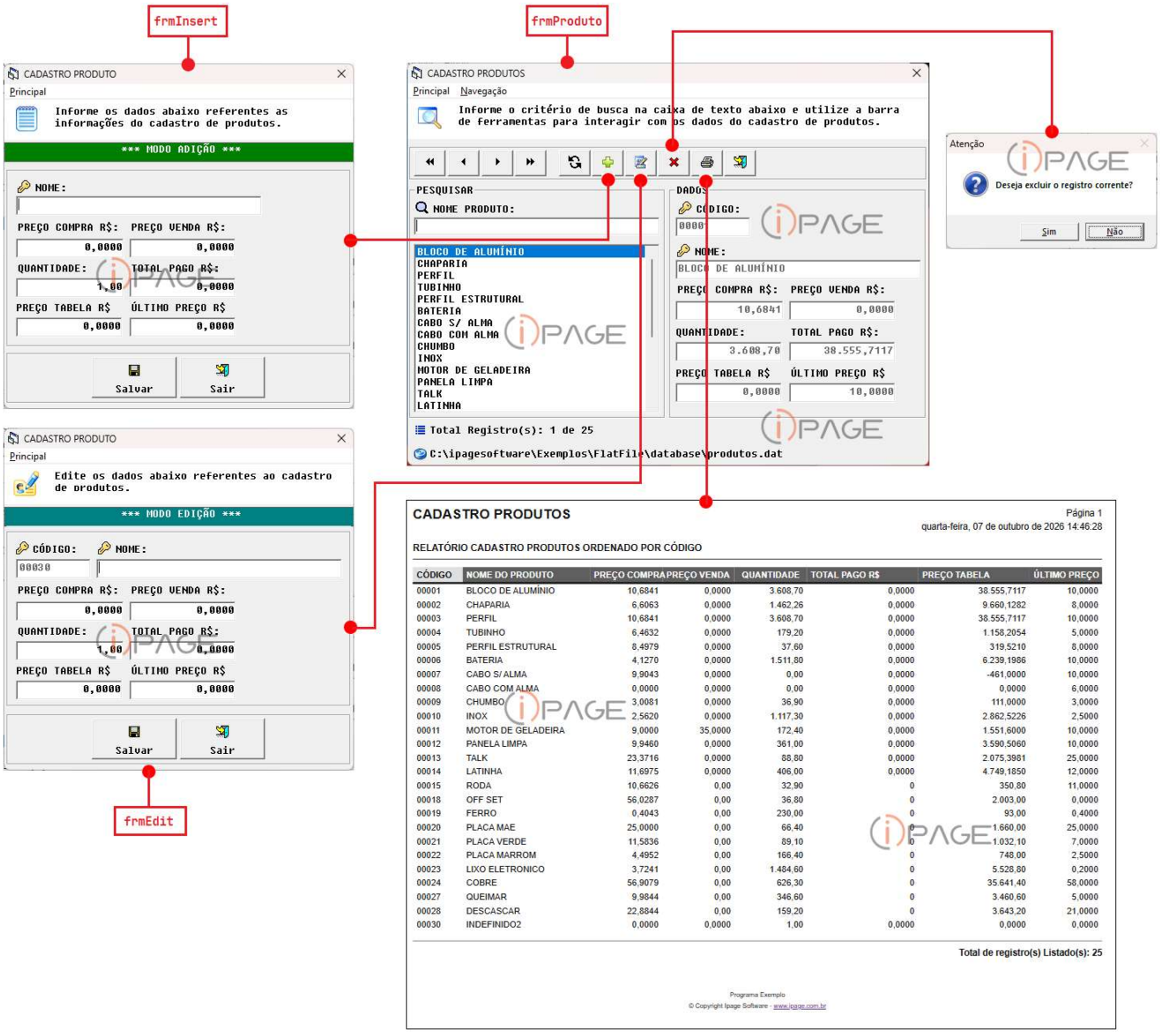


Figura 43

IDENTIFICAÇÃO GERAL DOS COMPONENTES

O módulo **Cadastro de Produtos** foi construído segundo o padrão de gerenciamento de dados **CRUD (Create, Read, Update, Delete)**, composto por uma tela principal de navegação e consulta (**frmProduto**), duas janelas modais de operação (**FrmInsert** e **FrmEdit**), uma caixa de diálogo para confirmação de exclusão e um gerador de relatório para impressão e exportação.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

TELA PRINCIPAL “FRMPRODUTOS” (NAVEGAÇÃO E CONSULTA)

A tela **frmProduto** atua como o painel central de controle do cadastro.

Ela possui uma estrutura dividida em duas colunas principais: Painel de Busca/Listagem (à esquerda) e **Painel de Detalhes do Registro** (à direita).

BARRA DE FERRAMENTAS (TOOLBAR SUPERIOR)

A barra superior conta com controles sequenciais de navegação e botões de ação:

NAVEGAÇÃO DE REGISTROS:

	(Primeiro Registro)
	(Registro Anterior)
	(Próximo Registro)
	(Último Registro)

AÇÕES DO SISTEMA:

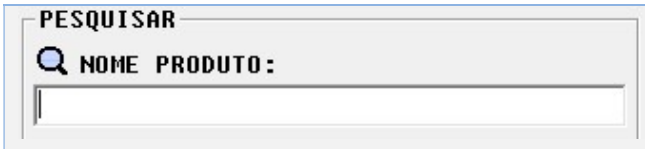
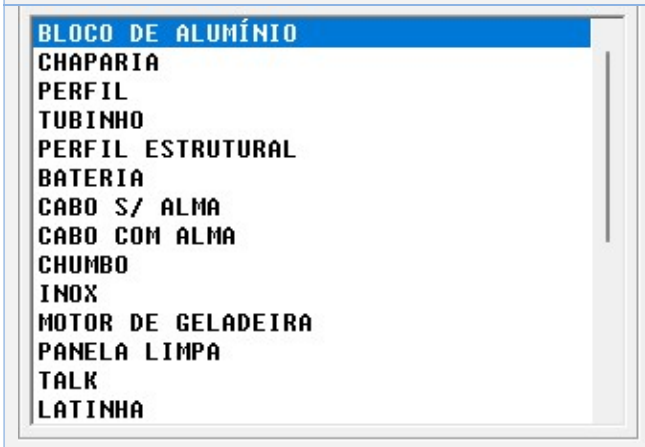
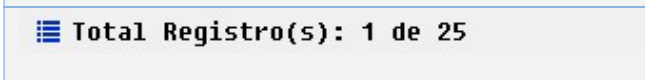
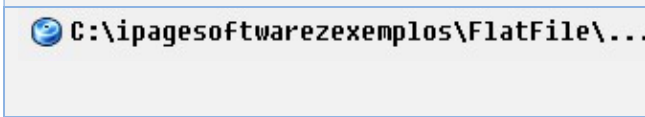
	Recarrega a lista de produtos a partir da base de dados.
	Abre o formulário **FrmInsert** para inclusão de novos produtos.
	Abre o formulário **FrmEdit** carregando os dados do registro selecionado.
	Dispara a caixa de confirmação para remoção do registro corrente.
	Abre a tela de pré-visualização de impressão do relatório geral de produtos.
	Fecha a janela de cadastro de produtos.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

PAINEL ESQUERDO: PESQUISA E LISTAGEM

A seção de pesquisa do formulário de cadastro de produtos está estruturada da seguinte forma:

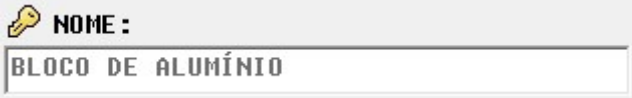
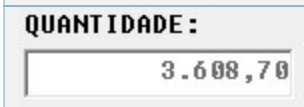
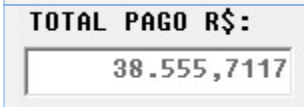
- Campo de Entrada (NOME PRODUTO:): Uma caixa de texto superior precedida por um ícone de lupa, destinada a permitir que o operador digite os critérios para filtrar ou procurar produtos específicos.
- Lista de Resultados (ListBox): Uma caixa de listagem abaixo do campo de busca que exibe os itens cadastrados (como "BLOCO DE ALUMÍNIO", "CHAPARIA", "PERFIL", "TUBINHO", entre outros), permitindo a seleção visual de um registro.
- Rodapé de Contagem e Caminho:
    - Exibe o total de registos encontrados e a posição atual (por exemplo, "Total Registro(s): 1 de 25").
    - Apresenta o caminho físico onde o arquivo de dados plano (.dat) está armazenado no sistema ex: C:\ipagesoftware\Exemplos\FlatFile\database\.

	Permite busca incremental/filtrada pelo nome do item.
	Exibe a listagem dos itens cadastrados (ex: **BLOCO DE ALUMÍNIO, CHAPARIA, PERFIL, TUBINHO**, etc.).
	Total Registro(s): X de Y (Contador de navegação).
	C:\ipagesoftware\Exemplos\FlatFile\database\produtos.dat (Caminho do arquivo de banco de dados/FlatFile).

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

PAINEL DIREITO: EXIBIÇÃO DOS DADOS DO REGISTRO SELECIONADO

Exibe de forma legível (modo somente leitura) os campos do produto selecionado na lista:

	Identificador único numérico (ex: 00001).
	Descrição do produto (ex: BLOCO DE ALUMÍNIO).
	Valor de custo/compra unitário (ex: 10,6841).
	Valor praticado para venda ao cliente (ex: 0,0000).
	Estoque físico atual (ex: 3.608,70).
	Valor acumulado pago pelo estoque/compra (ex: 38.555,7117).
	Preço de tabela de referência (ex: 0,0000).
	Histórico da última precificação registrada (ex: 10,0000)

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FORMULÁRIO DE INCLUSÃO: FRMINSERT (MODO ADIÇÃO)

Acessado via botão **Novo**  na tela principal. Abre uma janela modal destinada ao cadastramento de um novo produto no banco de dados.

CAMPOS DO FORMULÁRIO

NOME:	Caixa de texto livre para inserção do nome/descrição do produto. Propriedade MaxLenght = 20
PREÇO COMPRA R$: 0,0000	Campo numérico para o valor de aquisição (Padrão: 0,0000). Propriedade MaxLenght = 15
PREÇO VENDA R$: 0,0000	Campo numérico para a definição do preço de venda (Padrão: 0,0000). Propriedade MaxLenght = 10
QUANTIDADE: 1,00	Campo numérico para definição da quantidade inicial em estoque (Padrão: 1,00). Propriedade MaxLenght = 10
TOTAL PAGO R$: 0,0000	Campo numérico para registro do custo total do lote/estoque (Padrão: 0,0000). Propriedade MaxLenght = 20
PREÇO TABELA R$ 0,0000	Campo numérico para valor de tabela (Padrão: 0,0000). Propriedade MaxLenght = 10
ÚLTIMO PREÇO R$ 0,0000	Campo numérico para histórico de preço prévio (Padrão: 0,0000). Propriedade MaxLenght = 10

BOTÕES DE AÇÃO

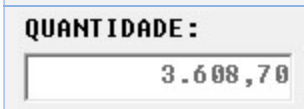
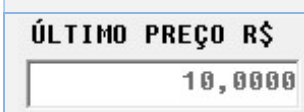
Salvar	Valida os campos preenchidos, insere o registro na base produtos.dat, atualiza a lista da tela principal e fecha o formulário.
Sair	Cancela a operação de adição sem salvar nenhuma alteração e fecha a janela.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FORMULÁRIO DE EDIÇÃO: FRMEDIT (MODO EDIÇÃO)

Acessado via botão Editar  na tela principal. Permite a alteração dos dados de um produto previamente cadastrado.

CAMPOS E COMPORTAMENTO

	Exibe o código do produto selecionado (ex: 00030), porém mantido como **somente leitura/desabilitado**, impedindo a alteração da chave primária do registro. Propriedade MaxLenght = 5
	Editável para correção de descrição do produto. Propriedade MaxLenght = 20
	Campo numérico para o valor de aquisição (Padrão: 0,0000). Propriedade MaxLenght = 15
	Campo numérico para a definição do preço de venda (Padrão: 0,0000). Propriedade MaxLenght = 10
	Campo numérico para definição da quantidade inicial em estoque (Padrão: 1,00). Propriedade MaxLenght = 10
	Campo numérico para registro do custo total do lote/estoque (Padrão: 0,0000). Propriedade MaxLenght = 20
	Campo numérico para valor de tabela (Padrão: 0,0000). Propriedade MaxLenght = 10
	Campo numérico para histórico de preço prévio (Padrão: 0,0000). Propriedade MaxLenght = 10

BOTÕES DE AÇÃO

	Grava as modificações no registro existente na base de dados, atualiza a exibição na tela **frmProduto** e fecha o formulário.
	Descarta as edições feitas e retorna à tela principal sem alterar o banco de dados.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

CAIXA DE CONFIRMAÇÃO DE EXCLUSÃO (ATENÇÃO)

Disparada ao clicar no botão **Excluir**  da barra de ferramentas da tela **frmProduto**.



Figura 44

O botão **Sim:** Remove permanentemente o registro selecionado do arquivo produtos.dat e atualiza a listagem.

Já botão **Não:** Cancela a exclusão e mantém o registro intacto.

TELA/PREVIEW DE RELATÓRIO: RELATÓRIO CADASTRO PRODUTOS

Disparada ao clicar no botão **Imprimir**  da barra de ferramentas da tela **frmProduto**.

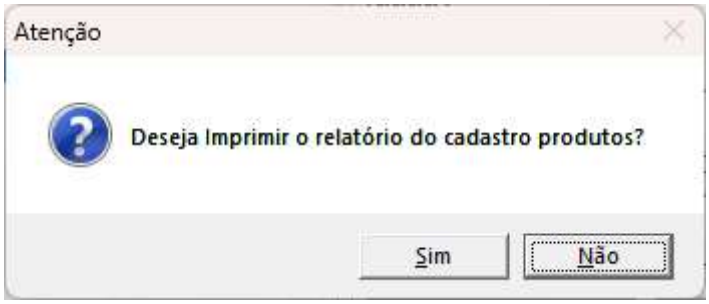


Figura 45

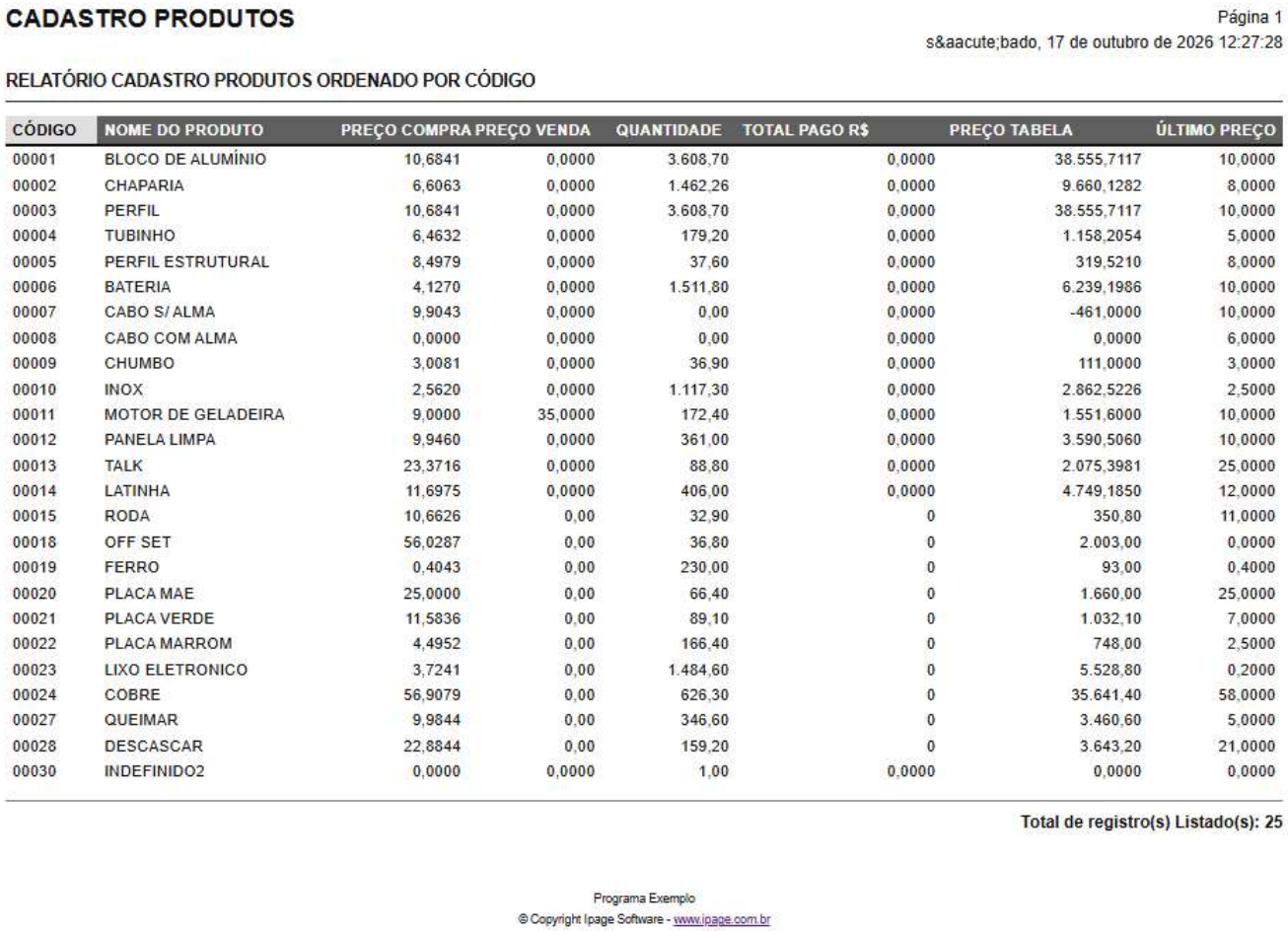
O botão **Sim:** Exibe no navegador(browser) do Windows a listagem dos produtos cadastrados no arquivo **produtos.dat**.

Já botão **Não:** Cancela a operação sem emitir o relatório.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

ESTRUTURA DO RELATÓRIO

Acessada via botão de **Impressão/Relatório** da tela **frmProduto**. Gera um documento formatado para visualização em tela ou impressão física/HTML que pode ser salvo em PDF.



CADASTRO PRODUTOS

Página 1
sábado, 17 de outubro de 2026 12:27:28

RELATÓRIO CADASTRO PRODUTOS ORDENADO POR CÓDIGO

CÓDIGO	NOME DO PRODUTO	PREÇO COMPRA	PREÇO VENDA	QUANTIDADE	TOTAL PAGO R$	PREÇO TABELA	ÚLTIMO PREÇO
00001	BLOCO DE ALUMÍNIO	10,6841	0,0000	3.608,70	0,0000	38.555,7117	10,0000
00002	CHAPARIA	6,6063	0,0000	1.462,26	0,0000	9.660,1282	8,0000
00003	PERFIL	10,6841	0,0000	3.608,70	0,0000	38.555,7117	10,0000
00004	TUBINHO	6,4632	0,0000	179,20	0,0000	1.158,2054	5,0000
00005	PERFIL ESTRUTURAL	8,4979	0,0000	37,60	0,0000	319,5210	8,0000
00006	BATERIA	4,1270	0,0000	1.511,80	0,0000	6.239,1986	10,0000
00007	CABO S/ ALMA	9,9043	0,0000	0,00	0,0000	-461,0000	10,0000
00008	CABO COM ALMA	0,0000	0,0000	0,00	0,0000	0,0000	6,0000
00009	CHUMBO	3,0081	0,0000	36,90	0,0000	111,0000	3,0000
00010	INOX	2,5620	0,0000	1.117,30	0,0000	2.862,5226	2,5000
00011	MOTOR DE GELADEIRA	9,0000	35,0000	172,40	0,0000	1.551,6000	10,0000
00012	PANELA LIMPA	9,9460	0,0000	361,00	0,0000	3.590,5060	10,0000
00013	TALK	23,3716	0,0000	88,80	0,0000	2.075,3981	25,0000
00014	LATINHA	11,6975	0,0000	406,00	0,0000	4.749,1850	12,0000
00015	RODA	10,6626	0,00	32,90	0	350,80	11,0000
00018	OFF SET	56,0287	0,00	36,80	0	2.003,00	0,0000
00019	FERRO	0,4043	0,00	230,00	0	93,00	0,4000
00020	PLACA MAE	25,0000	0,00	66,40	0	1.660,00	25,0000
00021	PLACA VERDE	11,5836	0,00	89,10	0	1.032,10	7,0000
00022	PLACA MARROM	4,4952	0,00	166,40	0	748,00	2,5000
00023	LIXO ELETRONICO	3,7241	0,00	1.484,60	0	5.528,80	0,2000
00024	COBRE	56,9079	0,00	626,30	0	35.641,40	58,0000
00027	QUEIMAR	9,9844	0,00	346,60	0	3.460,60	5,0000
00028	DESCASCAR	22,8844	0,00	159,20	0	3.643,20	21,0000
00030	INDEFINIDO2	0,0000	0,0000	1,00	0,0000	0,0000	0,0000

Total de registro(s) Listado(s): 25

Programa Exemplo
© Copyright Ipage Software - www.ipage.com.br

Figura 46

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

FLUXOGRAMA DE EXECUÇÃO: GERAÇÃO DE RELATÓRIO HTML

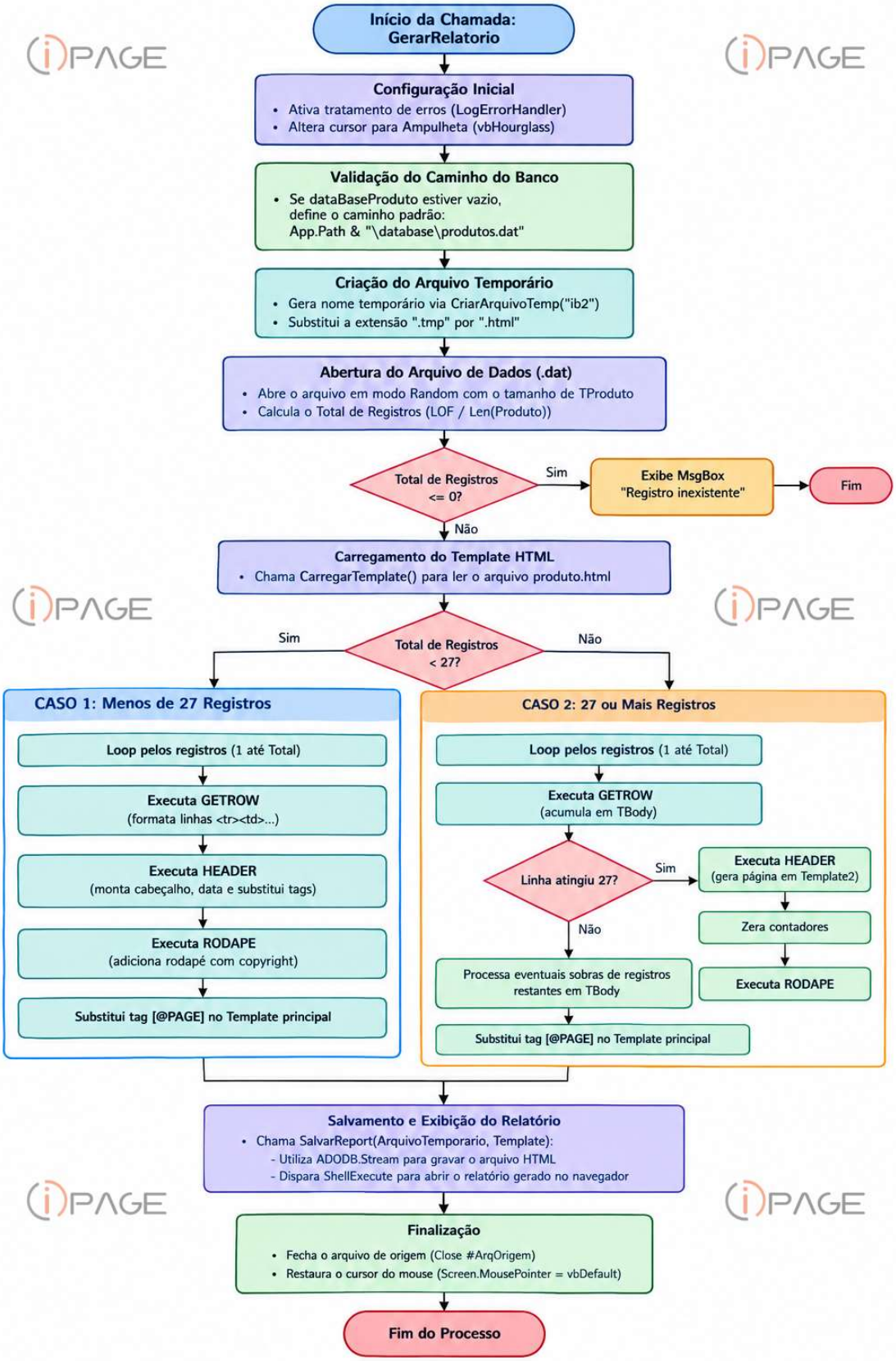


Figura 47

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Anotações

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Anotações

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Anotações

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

Anotações

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352

IPAGE SOFTWARE

ETE ESCOLA TÉCNICA ESTADUAL

Microsoft® Visual Basic 6.0

OBRIGADO
POR CHEGAR ATÉ AQUI!

Seu empenho e dedicação em acompanhar todos os conteúdos desta apostila demonstram o seu compromisso com o aprendizado e com o desenvolvimento das suas habilidades.

CONTINUE EXPLORANDO, PRATICANDO E DESENVOLVENDO NOVOS PROJETOS.

O conhecimento que você adquiriu aqui é a base para criar soluções e transformar ideias em realidade.





Microsoft® Visual Basic 6.0

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA
Fone/whatsApp: (081) 98615-2352

IPAGE SOFTWARE
Soluções em Software para o seu negócio.

© Copyright Diógenes Dias 1995-2026
IPAGE UNIDADE DE TECNOLOGIA EM INFORMÁTICA.
Fone-WhatsApp: +55 (81) 98615-2352